ST. DOMINIC COLLEGE OF ASIA
REVOLUTIONIZINGEDUCATION™
St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

Cloud-Based Sales and Marketing Portal with AI Chatbot Support, Data Analytics Dashboard for Unicom Technologies

Presented to the faculty of the College of Information Technology

St. Dominic College of Asia

Submitted By

Carlos, Dennis V.

Lindog, Maria Rollainne R.

Los Baños, Christian Ivan A.

Lopez, Justine M.

Professor: Dr. Pastor Arguelles Jr.

St. Dominic College of Asia

City of Bacoor, Cavite, Philippines

December 2025

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

Table of contents

Chapter I: Background of the study 5

1.1 Introduction 5

1.2 Objectives of the Study 5

1.3 Significance of the Study 6

1.4 Scope and Delimitation of the Study 7

Chapter II – Methodology of the Study (SDLC) 7

2.1 Software Development Methodology 7

2.2 Agile Process Phases 8

2.3 Development Tools and Environment 9

Chapter III – Data Gathering Procedures and Output 10

3.1 Introduction to the data gathering procedure 10

3.2 Stakeholders / Respondents 10

3.3 Data Collection Methods 11

3.4 Data Analysis Plan 12

3.5 Output: Defined Software Requirements 15

Chapter IV – Documentation of the Current System 16

4.1 Organizational Chart 16

4.2 Duties and Responsibilities of Personnel 17

4.3 Current System Policies and Procedures 17

4.4 Conceptual Framework and Context Data flow Diagram 19

Chapter V: Software Project Estimation 20

5.1 Schedule of activities 20

5.2 Resource Requirements 21

5.3 Cost Estimation 22

Chapter VI: Requirements Analysis and Specification Model 24

6.1 Process Specification Modelling 24

6.1.1 Conceptual Design 24

6.1.2 Data Flow Diagrams 24

6.1.3 Logical DFD 26

6.1.4 Physical DFD 27

6.2 Data Specification Modeling 28

6.2.1 Entity Relationship Diagram (ERD) 28

6.2.2 Data Dictionary 29

6.2.3 Use Case Diagram 32

Chapter VII: System Design Specification 33

7.1 Software Design Specification 33

7.2 Database Design 35

7.3 System Interface Design 37

7.4 Program Module Specification 41

Chapter VIII: Software Implementation 44

8.1 Programming Considerations, issue, and Tools. 44

8.2 System Requirements Specification 45

Chapter IX: Summary, Conclusion, and Recommendations 48

9.1 Software Maintenance Plan 48

9.1.1 Release Cadence 49

9.1.2 Environment Strategy 49

9.2 Summary 51

9.3 Conclusion 51

9.4 Recommendation 52

Appendices 53

Appendix A 53

Appendix B 54

Appendix C 55

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

Appendix D 56

Appendix E 57

Appendix F 58

Appendix G 64

Bibliography 188

Resource Persons 189

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

Chapter I: Background of the study

1.1 Introduction

Unicom Technologies is a system integrator and corporate reseller of computer information technology solutions. The company primarily services small and medium-sized businesses and does not have a physical retail store. They provide information technology services to other businesses for their products. These services entail looking through various suppliers and partnering distributors to procure the necessary hardware and software components and therefore having a comprehensive platform to find those components is paramount in delivering the best quality.

During the pandemic, the company was a member of Shopee and Lazada, but the application usage fees, and tax deductions prompted them to look for alternative ways to distribute their items. Since simply creating a similar catalogue system would induce too much cost and labor when it comes to usage and maintenance, modifications and upgrades must then be employed to the new system.

Some of these include basing the catalogue system on clouds and introducing AI to aid customer support. Cloud computing offers security and scalability, whilst AI chatbots give 24/7 support, reduce manual labor, and improve marketing and sales strategy.

1.2 Objectives of the Study

1.2.1 General Objective

The primary objective of this study is to develop a comprehensive cloud-based catalog system with AI chatbot support, a quote generator, and data analytics, with the aim of addressing the current challenges and gaps faced by the company.

1.2.2 Specific Objectives

Specifically, this study seeks to

- Reduce order inquiry and processing time by at least 30-50%.
- Provide immediate customer support or after-sales handling and solutions.
- Increase remote company staff efficiency and effectiveness.

- Expand marketing opportunities by 10-30% initially, to reach 80-90% sales increase within 3 quarters.

1.2.3 Software Quality Standard (ISO/IEC 25010)

To ensure that the system meets global software development standards, this study adopts the ISO/IEC 25010 Software Quality Model as the benchmark for evaluating system quality and performance. This model defines eight key characteristics that guide both the development and assessment of the proposed system:

- **Functional Suitability** The system must deliver essential functions such as quotation generation, chatbot responses, and analytics visualization to fulfill user requirements.
- **Performance Efficiency** It should maintain fast response times (target: under 2 seconds per user request) and handle concurrent operations efficiently.
- **Compatibility** The system must operate smoothly across multiple devices and browsers, ensuring accessibility for all users.
- **Usability** The interface should prioritize simplicity and intuitive navigation for users with varying technical backgrounds.
- **Reliability** The application should provide stable, error-free operation with consistent uptime and robust recovery mechanisms.
- **Security** The system must safeguard user and company data through encryption, authentication, and secure access protocols.
- **Maintainability** Built using **Next.js** and **React**, the system's modular structure should allow easy debugging, scalability, and enhancements.
- **Portability** The solution should support deployment on various cloud platforms such as **Vercel**, ensuring flexibility in future migrations or upgrades.

1.3 Significance of the Study

This study is significant to customers, SMB clients, the sales and marketing team, administrators, and stakeholders.

- Customers benefit from faster responses via AI chatbot support, making the platform more accessible and user-friendly.

- The sales and marketing team gains from data-driven insights provided by the analytics dashboard, which helps improve campaign strategies.
- Administrators benefit from real-time cloud accessibility, streamlining operations and reducing delays.
- More broadly, this project contributes to the modernization of Unicom Technologies' business model and supports the digital transformation of customer service in the IT industry.

1.4 Scope and Delimitation of the Study

Scope:

- A web-based application accessible on every device
- Online interactive query and assistance, and solutions
- Offline AI-based support assistance for common problems
- A basic proposal/quote generator
- Technical support that is accessible online
- A link to handle financial/sales transactions as a future enhancement of this application
- A procedure for warranty support updates in real time
- A procedure for sales/order updates in real time
- A procedure for delivery support updates in real time
- A quick product comparison

Delimitation:

- Hardware constraints, like servers, firewall appliances, are currently not considered.
- Budget constraints, the application must be run only on free cloud-based systems, and security until proven effective before investing in.

Chapter II – Methodology of the Study (SDLC)

2.1 Software Development Methodology

Older methods that follow a fixed process are not a good fit for this project, as it is not flexible and does not include working closely with the stakeholders. The Agile method is chosen as it focuses on being flexible, teamwork, building the project step by step, and allowing the delivery of improvements regularly.

Why Agile is Chosen:

- Agile adapts well to changes, which fits well with this project.
- Agile is quick to respond to feedback and suggestions and apply them in the project, which ensures that the product meets user needs.
- Agile is all about teamwork and working closely with stakeholders to avoid misunderstandings and share responsibility.

These qualities make Agile a strong choice for the project, as it helps maintain flexibility, improve quality, and keep the development process in line with what stakeholders expect.

2.2 Agile Process Phases

2.2.1 Planning

The team will begin by planning the system and setting clear goals for it, defining its scope, and listing its expected deliverables. A product backlog will be created with the use of ClickUp, with the inclusion of features such as a product catalog, a quote submission feature, an analytics dashboard, and an AI chatbot. As a user story, each task will be written based on its priority, complexity, and importance to ensure smooth development.

2.2.2 Requirements Analysis & Design

To understand the needs of Unicom Technologies staff and system users, the team will conduct meetings and gather important information from them. Design materials such as flow diagrams, wireframes, and UML diagrams will be created by the team to visualize the system's structure and functions. Tools were chosen for the system, such as Next.js App Router for page organization, React.js for building components, and MongoDB for managing the database.

2.2.3 Iteration / Development (Sprints)

For the development process, work will be divided into short sprints lasting for six weeks, two weeks each. The focus for each sprint is on creating specific features with the use of TypeScript, Next.js, React components, Tailwind CSS, and Shadcn/Radix UI elements. For the version control, Git and GitHub will be used, and for the completed sprint outputs, they will be deployed using Vercel for easy preview and feedback.

2.2.4 Testing

Tests will occur with each sprint's output before being marked complete. There will be unit testing to check individual React components and TypeScript functions, while integration testing will ensure that data flows properly between components and the database. Lastly, browser-based responsiveness tests will also be performed to ensure that the user interface works correctly on different devices.

2.2.5 Review & Feedback

The deployed version on Vercel will be presented to the stakeholders after each sprint. The product backlog will then contain the stakeholders' feedback on the system's design, usability, and performance. Ensuring that the development stays aligned with user needs and expectations, this feedback will be used as a guide for the next sprint.

2.2.6 Deployment

The stable version of the system will be deployed through Vercel's continuous deployment that is connected to GitHub once the core features are fully developed and tested. To manage user data, quotes, and other important records securely and efficiently, MongoDB Atlas will be set up.

2.2.7 Maintenance & Continuous Improvement

The Agile approach will continue to guide system updates and improvements even after deployment. The team will work on adding new features such as enhancing the chatbot's intelligence and refining the React component structure. All these updates will be based on user feedback, bug reports, and performance data that is gathered after the system's release.

2.3 Development Tools and Environment

Programming Language:

- TypeScript
- JavaScript (via TS transpilation)
- CSS (Tailwind utility classes)

Tools:

- Vercel CLI – for deployments
- Git + GitHub – for version control
- pnpm – for package management
- Lucide icons (lucide-react)
- Shadcn/Radix UI – component generator and primitives

Framework:

- Next.js 14 (App Router)
- React 18
- Tailwind CSS
- Radix UI / Shadcn UI components (buttons, sheets, dropdowns, etc.)
- Custom hooks and context (e.g., useAuth)

Database:

- MongoDB
- Browser localStorage (for lightweight auth persistence)

Project Management Tool:

- Clickup for project management
- GitHub for version control

Chapter III – Data Gathering Procedures and Output

3.1 Introduction to the data gathering procedure

For this chapter, the course of action for gathering data to specify the system requirements for the Cloud-Based Sales and Marketing Portal with AI Chatbot Support and Data Analytics Dashboard for Unicom Technologies will be presented.

Identifying the current problems, user needs, and desired features is the main focus of this phase, which will guide the system's design and development, which is discussed in Chapter II.

To make this possible, the team gathered data by using surveys from selected stakeholders.

3.2 Stakeholders / Respondents

The respondents were carefully chosen based on their involvement in the company's sales and marketing. Their input was crucial to understanding how the current manual processes work and what improvements are needed.

Table 3.1

Role	No. of Respondents	Reason for Selection
Sales & Marketing	2	- Handles Inquiries - Cold Calling - Client DBF Buildup - Sales Funnel Report - Prepares Quote - Client Meetings - Close Deal/Project - Create Marketing Campaigns
Accounting	1	- Discount Approval - Terms of Payment - Receivables - Payables
Project Manager	1	- Oversees Projects - Team Creation and Deployment - Subcontractor Hiring and Management - Intermediary between End Clients and Company Administrative Staff, Both Technical and Otherwise.

The company operates with three key departments: Sales & Marketing (2 staff) handles inquiries, cold calling, client database buildup, sales funnel reporting, quotation preparation, client meetings, deal closure, and marketing campaigns; Accounting (1 staff) manages discount approvals, payment terms, receivables, and payables; and Project Management (1 staff) oversees projects, creates and deploys teams, hires and manages subcontractors, and serves as the intermediary between end clients and company administrative staff for both technical and non-technical matters.

3.3 Data Collection Methods

The group used one main method to collect data

- **Surveys / Questionnaires**

  Google Forms were distributed to internal staff. Questions used Likert scales

to measure satisfaction and asked about preferred features such as Analytics, Used of quotation, and chat support.

Instruments / Tools

- **Survey Questionnaire** — Google Form with 12 questions including open ended questions on current experience, pain points, and preferred system features.
- **Interview Guide** — A list of 4 open-ended questions (e.g., "What features do you want in a new sales and marketing port?").

Example interview questions:

- What are the biggest problems you face in handling client quotations?
- How do you currently process sales or quotation requests?
- What kind of reports or analytics would help improve your work?

3.4 Data Analysis Plan

The data gathered from the survey was analyzed using both quantitative and qualitative approaches. Closed-ended questions were presented in a Likert scale format (1 = Strongly Disagree to 5 = Strongly Agree), which allowed the researchers to compute agreement percentages. Open-ended questions were coded into themes and summarized to highlight recurring challenges and requirements mentioned by the respondents.

3.4.1 Likert-Scale Analysis

The survey included closed-ended questions using a Likert scale ranging from 1 (Strongly Disagree) to 5 (Strongly Agree). This section aimed to measure the respondents' perceptions of quotation delays, data management challenges, and openness to adopting digital solutions.

Table 3.2. Likert Scale Summary

Questions	Percentage %	Notes
It takes a long time to respond to client inquiries.	33.3%	Only 1 out of 3 agreed; majority disagreed.
Preparing and sending quotations is time-consuming.	33.3%	Only one respondent found it time-consuming; others did not

It is hard to track which products are available or out of stock.	33.3%	Mixed — only one respondent strongly agreed.
We do not have enough data or reports to guide our sales/marketing strategies.	66.7%	Most agreed — analytics and reports are lacking.
We often lose or misplace important client quotation records.	0%	No one agreed; losing records is not the main issue.
We are open to using an AI chatbot to answer basic customer questions.	100%	All respondents agreed — strong support for chatbot use.
We need a central system to manage products, client information, and quotations.	100%	All respondents agreed — strongest consensus.

Note: As seen in the table above, respondents stressed that the reason for quotation delays is due to supply issues such as out of stock and order-basis items as well as client-specific configurations. Moreover, updates given on time from out-of-office replies made the client's communication slower.

3.4.2 Open-Ended Analysis

To get more in-depth responses from respondents, the survey also contained a number of open-ended questions in addition to Likert-scale questions. The purpose of these inquiries was to identify difficulties with handling quotations, existing procedures, characteristics that would be ideal in a suggested system, and further recommendations for enhancement.

Table 3.3. Open-Ended Responses Summary

Question	Response Summary (Keywords)	Notes
Biggest problems	Incomplete client info, manual processes, inconsistent pricing, outdated data, delays due to no	Core issues: slow/manual processes, lack of

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

	available agent, out-of-stock items, special client configs, IT infra requirements, tech support delays, out-of-office responses	updated data, and communication delays.
How quotations are prepared	(1) Standard template → manual input → internal review → send PDF → manual follow-up. (2) Understand client needs → calculate costs (products, delivery, manpower, margins) → send quotation. (3) Client request → purchasing sources items → approvals (sales, finance) → negotiations, meetings, demos → closing.	All respondents follow manual, multi-step workflows requiring approvals and manual tracking.
Reports/analytics needed	Win/loss rates, quote turnaround time, conversion rates, top quoted items, pricing history, margin analysis, marketing strategy gaps, social media engagement, sales analytics, market segment analysis, quotation & tech support tracking, inquiry tracking.	Strong emphasis on analytics dashboards, especially for pricing, sales performance, and market insights
Features wanted	(1) Centralized client & lead management, CRM integration, real-time reporting. (2) Online chatbot for 24/7 inquiries. (3) Online quote generator, tech support tracking, AI-assisted sales inquiries.	Top priorities: centralization, automation, AI support, and real-time reporting.
Feedback/suggestions	Centralized database, secure data storage, sales performance improvements via technology, open to further enhancements.	General openness to modernization and digital transformation.

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

Stakeholders identified core problems including incomplete client information, manual processes, inconsistent pricing, outdated data, and communication delays. All respondents follow manual, multi-step quotation workflows requiring multiple approvals and manual tracking, involving client request intake, cost calculations, internal reviews, and follow-ups. There is strong emphasis on the need for analytics dashboards covering win/loss rates, quote turnaround time, conversion rates, pricing history, margin analysis, and market insights. Top feature priorities include centralized client and lead management, CRM integration, real-time reporting, 24/7 AI chatbot support, online quote generation, and tech support tracking. Overall, stakeholders expressed openness to digital transformation and modernization through secure, centralized data systems and technology-driven sales performance improvements.

3.5 Output: Defined Software Requirements

Functional Requirements

- Product catalog with categories, search, and product details.
- Quotation generator (select products → auto compute totals → export PDF).
- AI chatbot for FAQs, product info, and inquiries.
- Admin panel for managing products, users, and viewing analytics.
- Data analytics dashboard showing sales.

Non-Functional Requirements

- Accessible from any device via a web browser.
- System response time under 2 seconds for common operations.
- Secure login using MongoDB Authentication.
- 99.9% uptime availability.
- Data encryption at rest and in transit.
- Responsive UI for desktop and mobile screens.

Data Requirements

- **Product:** id, name, description, price, category, image, inStock, specifications, brand, rating, reviews
- **User:** id, email, name, role
- **Quote:** id, customerName, customerEmail, company, phone, items, totalAmount, status, notes, createdAt, expiresAt, adminNotes

- **QuoteItem**: productId, productName, quantity, unitPrice, customSpecs
- **Chat logs:** id, text, isBot, timestamp, products

User Interface / Experience Requirements

- Clean, modern layout using Tailwind CSS and Shadcn UI components.
- Easy navigation using a fixed navbar and search bar.
- Mobile-friendly responsive design.
- Dashboard view with charts and KPIs for admin users.
- Chatbot widget is accessible only from client and visitor side.

Constraints / Assumptions / Dependencies

- Must operate on free cloud platforms initially (Vercel, MongoDB Atlas free tier).
- Requires a stable internet connection to use.
- Requires users to have an existing email address for account creation.
- LLM-based chatbot will be introduced only after system is stable.

Chapter IV – Documentation of the Current System

4.1 Organizational Chart

The organizational chart below represents the structure of Unicom Technologies. It highlights only the departments and personnel directly involved in the sales, marketing and administrative process relevant to this study.

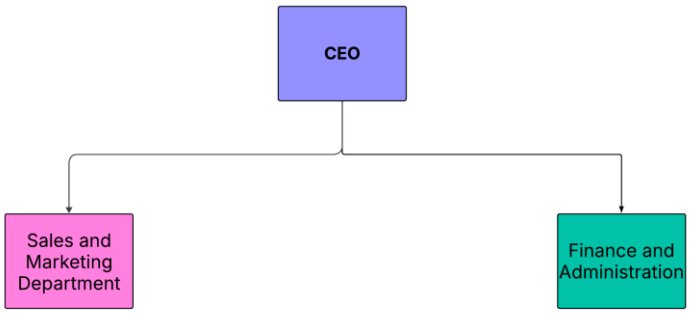


Figure 4.1. Organizational Chart of Unicom Technologies

This figure presents the internal organizational structure of Unicom Technologies, highlighting the departments involved in the company's sales, quotation, and customer support processes. It provides an overview of hierarchical roles and communication flow within the organization.

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

4.2 Duties and Responsibilities of Personnel

Table 4.1

Position	Duties and Responsibilities	Relation to the Study
CEO / Project Development Manager	Oversees business operations, approves strategic sales and IT-related projects Reviews and approves client quotations and pricing before sending to clients.	Authorizes implementation of the cloud-based portal Directly manages quotation flow and system validation.
Corporate Account Executives	Cold Calling, Client DBF Management, Sales Funnel Reporting, Handles client inquiries, prepares quotations using Excel, and coordinates with purchasing for Hardware and Software pricing and availability Promotes products and services, tracks market trends, and assists in client long term relations and building rapport.	Primary users of the quotation and AI chatbot features. Will use analytics dashboard for marketing insights
Financial Officer	Verifies costing, payment terms, and client account status	Ensures financial data integration with the system

Note: This table summarizes the main roles within the Unicom project team including Project Manager/Scrum Master, System Analyst, Backend Developer, and Frontend Developer along with their specific functions and contributions to the development of the system.

4.3 Current System Policies and Procedures

Currently, Unicom Technologies follows a manual, email-based process for handling sales and quotations. The process lack automation and centralized tracking, which results in delayed responses and inconsistent data records.

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

Table 4.2

Policy / Procedure	Description
Client Inquiry Handling	Clients send quotation or product inquiries through email or messaging applications. Requests are manually checked by the sales team for completeness.
Quotation Review and Preparation	The sales agent reviews requests, verifies product availability, and coordinates with the purchasing department for item sourcing and costing. Quotations are prepared manually using Excel.
Costing and Approval Process	The sales supervisor reviews and approves the costing of each quotation before it is finalized.
Quotation Delivery	Approved quotations are manually sent to clients via email.
Follow-Up and Feedback Tracking	The sales team manually monitors client responses, using spreadsheets or chat logs to track communication.
Payment Coordination	Once a quotation is approved, the sales team coordinates with the finance department for payment processing and confirmation.
Product Procurement	The purchasing department sources and orders products as requested by the sales team once payment or terms are confirmed.
Delivery Scheduling	Delivery arrangements are made manually through coordination between sales, sales manager, and logistics staff.
Record Keeping and Documentation	Transaction records and communication details are stored individually by each department, leading to inconsistent documentation.

Note: This table outlines the existing manual workflow of Unicom Technologies. It details how quotations are processed, approvals are handled, and data is shared among departments through traditional methods such as email and spreadsheets.

4.4 Conceptual Framework and Context Data flow Diagram

Conceptual Framework:

The proposed cloud-based system aims to automate and centralize the sales and marketing workflow. It connects clients, sales staff through a single platform that provides a real time quotations, data analytics, and chatbot assistance.

Table 4.3

Category	Description
Input	Client inquiries, quotation requests, and product details are submitted through the web-based portal. These inputs include customer information, product specifications, and pricing parameters.
Process	The system automatically generates quotations using predefined product data, performs analytics computations for insights, and interacts with users through the integrated AI chatbot for real-time responses and support.
Output	The system produces approved quotations, sales and analytics reports, and automated customer responses or insights for the sales and marketing team.

Note: The system receives client inquiries, quotation requests, and product details through the web portal, automatically generates quotations using predefined product data while providing AI chatbot support and analytics computations, and produces approved quotations, sales reports, and automated customer responses for the sales and marketing team.

Figure 4. Conceptual Framework of the Proposed System

This diagram illustrates the conceptual structure of the proposed Cloud-Based Sales and Marketing Portal. It identifies key elements — input, process, and output — and shows how automation, AI chatbot integration, and analytics transform the company's quotation and communication workflow.

Chapter V: Software Project Estimation

5.1 Schedule of activities

The project follows an agile software development life cycle (SDLC), which divides the development process into iterative sprints, Each sprint delivers a functional output that can be tested and reviewed before moving to the next phase.

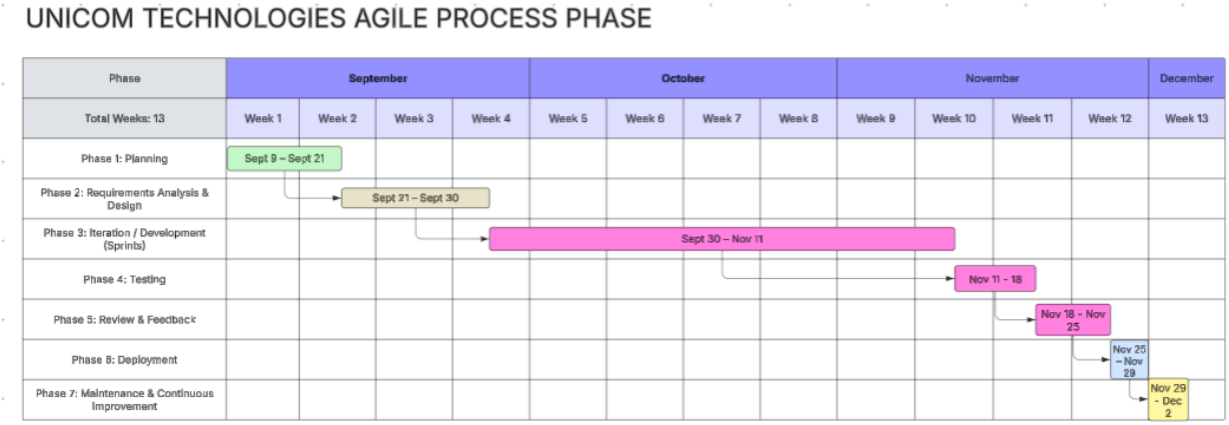


Figure 5. Agile Development Timeline for Unicom Technologies

This Gantt-style timeline represents the 13-week Agile development schedule of the system. It includes the major phases such as planning, design, development sprints, testing, deployment, and maintenance, showing the overlapping and iterative nature of the workflow.

Agile development Detailed flow Process

Phase	Description	Duration	Start Date	End Date	Responsible Team	Expected Output
Phase 1: Planning	Identify objectives, define scope, gather initial requirements, and set sprint goals using Agile methodology.	2 weeks	Sept 9, 2025	Sept 21, 2025	**Scrum Master & System Analyst** – facilitate sprint planning, define scope, and create backlog.	Approved system plan, sprint backlog
Phase 2: Requirements Analysis & Design	Analyze system requirements, design DFD, ERD, and wireframes. Define architecture for front-end and back-end.	1 week	Sept 21, 2025	Sept 30, 2025	**System Analyst & Front-end Developer** – analyze requirements, design diagrams, and prototype UI.	System design diagrams and requirement specs
Phase 3: Iteration / Development (Sprints)	Implement key modules — quotation builder, AI chatbot integration, product management, and analytics dashboard.	6 weeks	**Sept 30, 2025**	Nov 11, 2025	**Front-end & Back-end Developers** – implement client interface, API endpoints, and database logic.	Functional system prototype per sprint
Phase 4: Testing	Perform unit and integration testing. Validate system flow, database operations, and chatbot accuracy.	1 week	Nov 11, 2025	Nov 18, 2025	**All Members (Scrum Master oversees) –** perform tests and bug fixes collaboratively.	Test results, bug reports, and revisions
Phase 5: Review & Feedback	Conduct sprint review and retrospective. Present progress to Unicom stakeholders for feedback.	1 week	Nov 18, 2025	Nov 25, 2025	**Scrum Master & System Analyst** – collect feedback, prepare retrospectives, and document improvements.	Review report and improvement log
Phase 6: Deployment	Deploy the system via Vercel and MongoDB Atlas. Ensure production readiness.	4 days	Nov 25, 2025	Nov 29, 2025	**Back-end & Front-end Developers** – perform deployment and production testing.	Fully deployed cloud-based system
Phase 7: Maintenance & Continuous Improvement	Monitor live performance, apply bug fixes, and optimize chatbot and analytics features.	3 days	Nov 29, 2025	Dec 2, 2025	**All Members** – oversee live system and improve modules continuously.	Final report and maintenance plan

Figure 6. Detailed Agile Sprint Workflow Diagram

This diagram explains the process of the Agile development approach used by the team. It shows how the team plans sprints, develops modules, performs testing, gathers feedback, and iterates improvements in a continuous loop to ensure progressive system enhancement

5.2 Resource Requirements

The project uses modern, cost-effective tools and free-tier cloud services to minimize expenses. Below are the resources and tools required:

Table 5.1

Category	Tools / Resources	Purpose
Software Tools	Visual Studio Code, GitHub, Vercel, MongoDB Atlas,	Development and deployment
Communication Tools	ClickUp, Google Meet	Collaboration and project tracking
Hardware	Laptops/PCs with stable internet	Coding and testing

Human Resources	Scrum Master, System Analyst, Frontend/Backend Developer	Agile sprint team roles

Note: The table lists the tools, communication platforms, hardware, and human resources required to complete the Unicom Cloud-Based Sales and Marketing Portal development.

5.3 Cost Estimation

Since the project is developed using **free-tier platforms** and **open-source tools**, the total project cost is kept minimal. However, a small budget is considered for miscellaneous expenses, such as internet usage, documentation, and cloud upgrades.

Table 5.2

Category	Item / Description	Estimated Cost (₱)
Direct Costs (Development & Services)	Developer Allowance – 4 developers (₱2,000 each)	₱8,000.00
	Web Hosting / Cloud Deployment – Vercel upgraded tier for testing	₱1,176.11
	Database Hosting – MongoDB Atlas upgraded tier for storage	₱1,764.16
	Data Encoder – To build up and update the product database	₱13,440.00
	AI Subscription – For a more intuitive AI support system -Monthly subscription	₱3,500.00
	Internet / Data Usage – for development, research, and testing	₱1,299.00
	Subtotal (Direct Costs)	**₱ 29,179.27**
Indirect Costs (Documentation & Administration)	Printing & Documentation – hard copies of reports and research papers	₱500.00

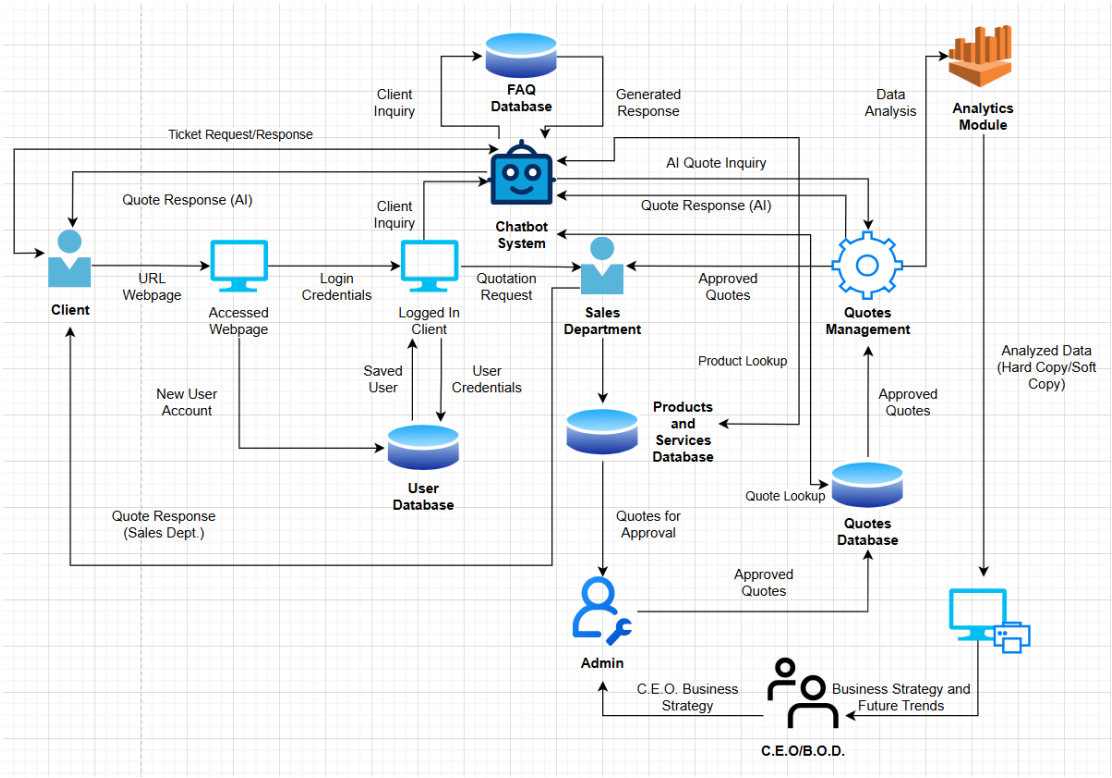
	Administrative Expenses – communication, transportation, coordination, snack allowance	₱1000.00
	Miscellaneous – office supplies, paper, flash drives, and other materials	₱1,900.00
	Subtotal (Indirect Costs)	**₱3,400.00**
Contingency and Buffer	Contingency Fund (₱29,179.27 + ₱3,400.00 = ₱32,579.27 × 0.10)	
	Subtotal (Contingency)	**₱3,257.93**
Summary of Project Costs	Direct Costs	₱29,179.27
	Indirect Costs	₱3,400.00
	Contingency & Buffer	**₱3,597.93**
	Total Estimated Project Cost	**₱32,579.27**

Note: The total estimated project cost is ₱32,579.27, covering both development and administrative expenses. Direct costs include developer allowances, hosting, and internet usage, while indirect costs cover documentation, administrative needs, and supplies. A 10% contingency fund is added to ensure flexibility and preparation for unexpected project-related expenses.

Chapter VI: Requirements Analysis and Specification Model

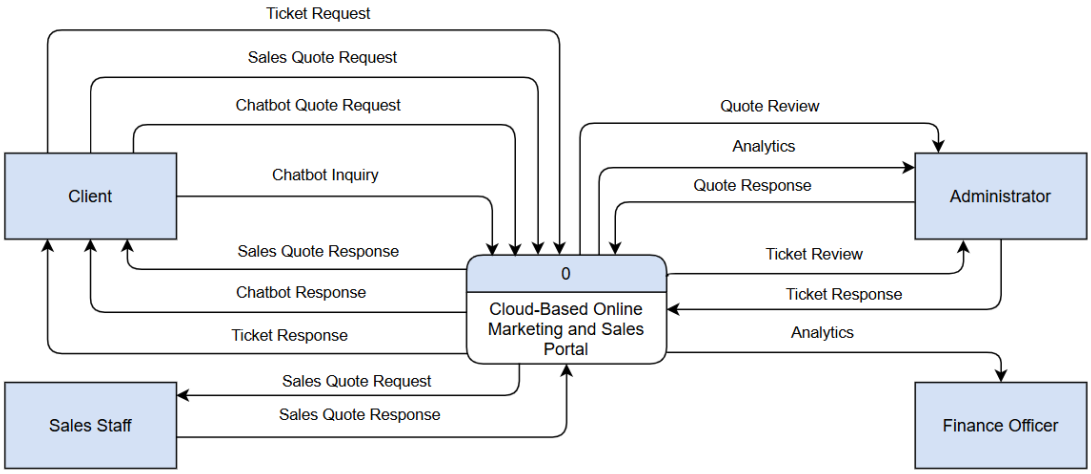
6.1 Process Specification Modelling

6.1.1 Conceptual Design



The proposed system automates the company's manual quotation and client communication processes. It provides a **centralized web-based portal** where clients can request quotations, communicate with an AI chatbot, and where the sales and marketing team can manage data, analytics, and updates.

6.1.2 Data Flow Diagrams



Context Diagram (Level 0)

Figure 3. Data Flow Diagram (Level 0) of the Proposed System

This figure presents the overall interaction between external entities (clients, AI chatbot API, admin) and the system's core processes. It shows how inquiries or quotation requests from clients are handled by the system, authenticated, processed, and stored in MongoDB, with admin oversight for all data transactions.

Level 1 DFD (Top-Down View)

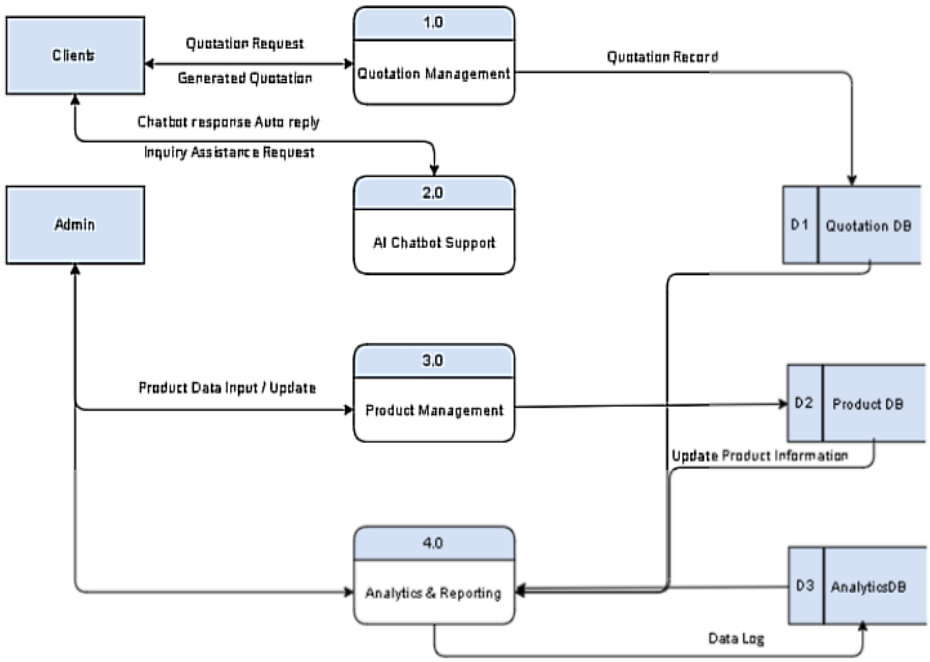


Figure 4. Data Flow Diagram (Level 1) of the Proposed System

The Level 1 DFD breaks down the system into its main processes — authentication, product management, quotation generation, chatbot inquiry handling, and analytics computation. Each process interacts with the MongoDB database for storage and retrieval while returning responses to the admin and clients accordingly.

6.1.3 Logical DFD

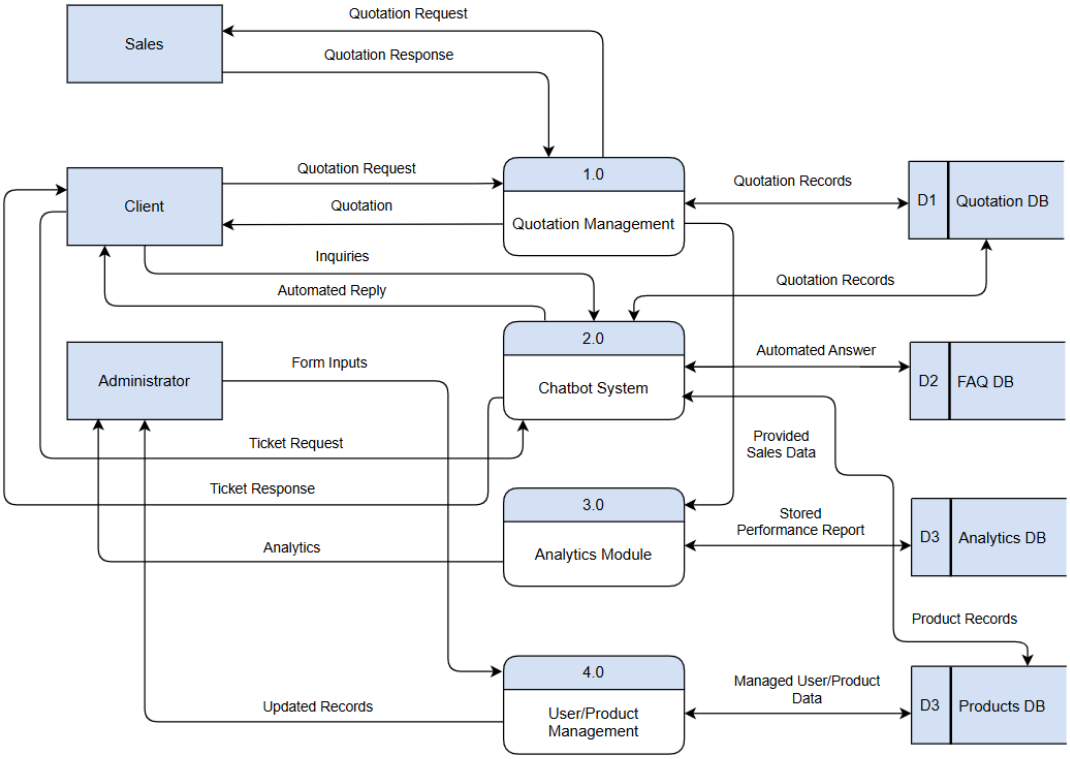


Figure 5. Logical Data Flow Diagram of the Proposed System

The logical DFD outlines the flow of information between the system's core modules without specifying technical details. It focuses on the logical relationships among the processes such as login authentication, product retrieval, quotation creation, and chatbot query handling.

6.1.4 Physical DFD

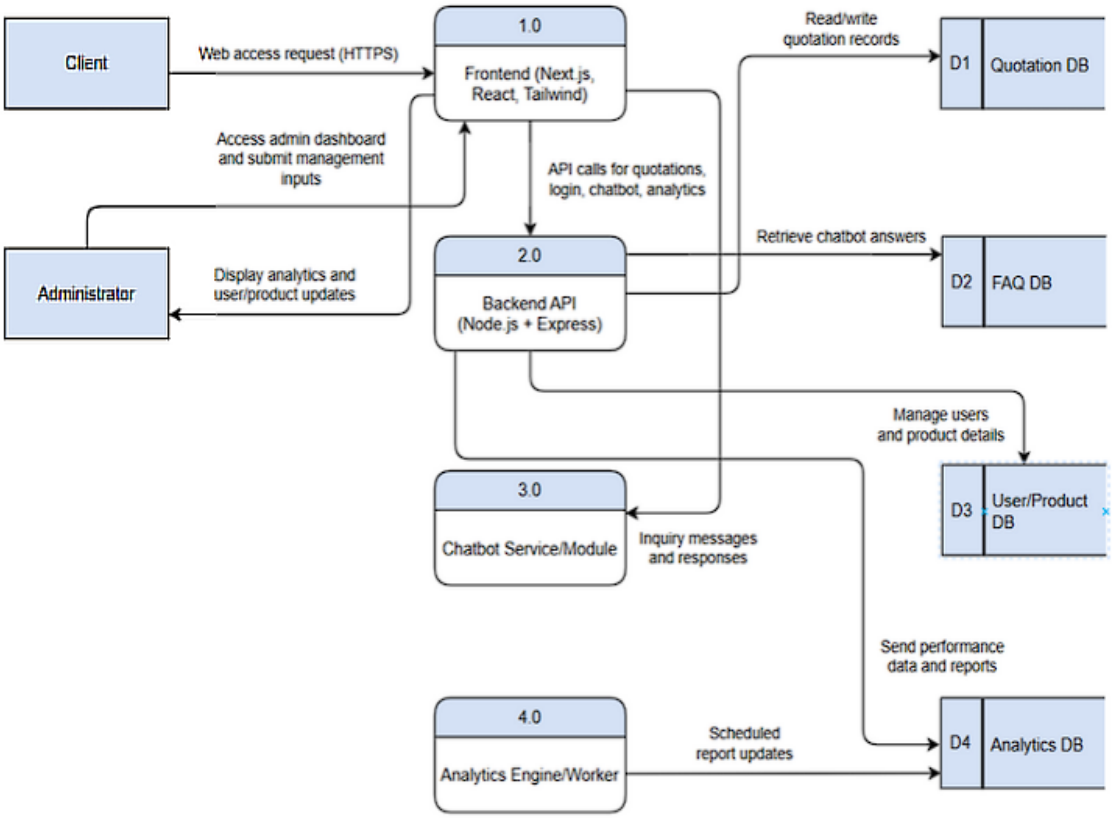


figure 6. Physical Data Flow Diagram of the Proposed System

The physical DFD represents the actual implementation of processes and data movement. It includes the real storage components such as MongoDB Atlas, the API routes handled by Express.js, and the interactions between frontend (Next.js) and backend servers, ensuring secure and efficient data transactions.

6.2 Data Specification Modeling

6.2.1 Entity Relationship Diagram (ERD)

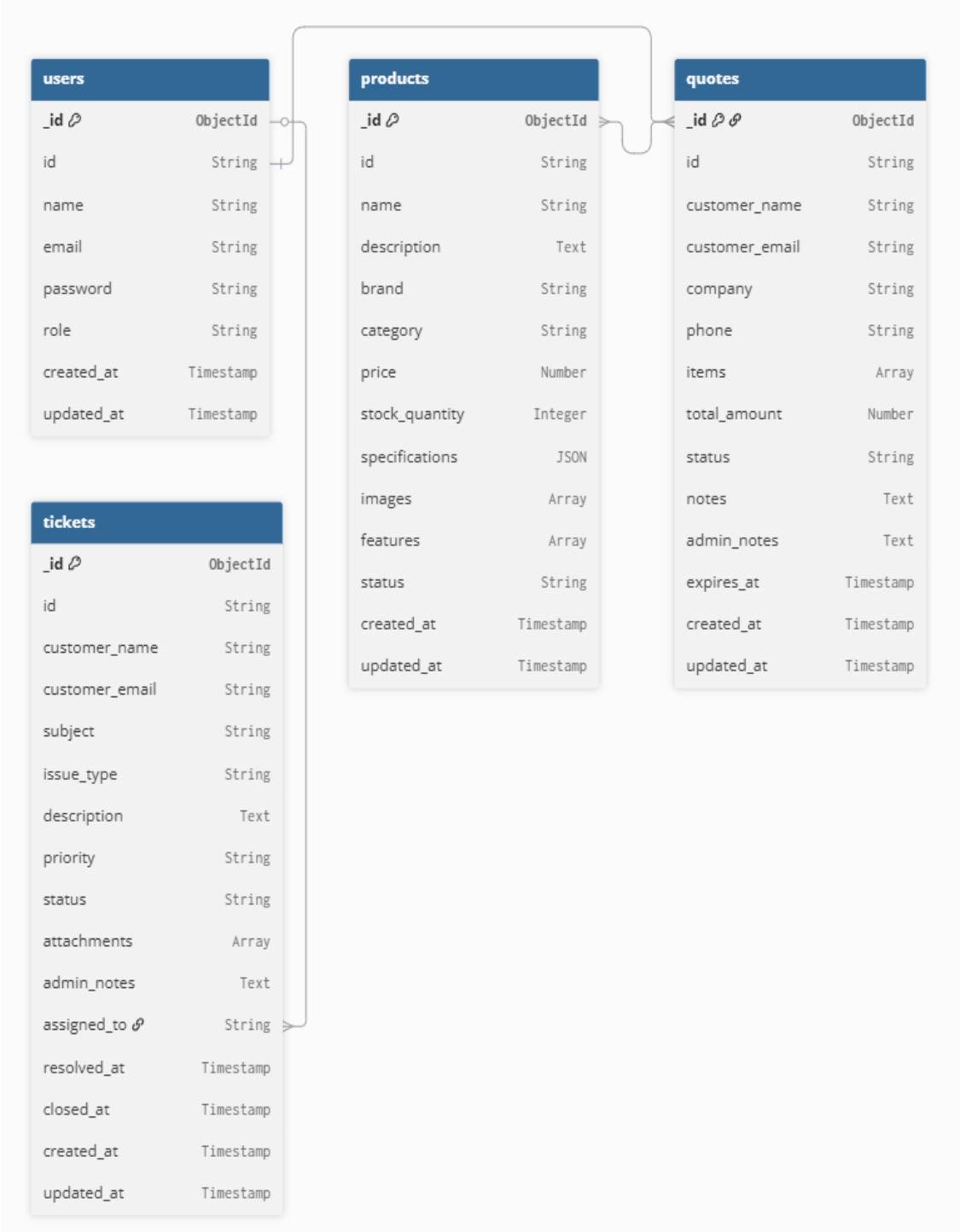


Figure 7. Entity Relationship Diagram (ERD) of the Proposed System

The ERD displays the relationships between entities in the system — Clients, Users, Quotations, and Products. Each entity holds attributes necessary for managing quotation requests, user authentication, and analytics computation. Relationships show that each client can have multiple quotations, and each quotation can include multiple products.

6.2.2 Data Dictionary

Table 6.1 User Collection

Field Name	***Data Type***	***Size***	***Description***
_id	*ObjectId*	–	*MongoDB primary key (automatically generated)*
id	*String*	*UUID*	*Unique ID for external system references*
name	*String*	–	*User's full name*
email	*String*	–	*Unique email used for login*
password	*String*	–	*Hashed password stored using bcrypt*
role	*String*	–	*User role (e.g., admin, customer)*
created_at	*Timestamp*	–	*Date and time when the record was created*
updated_at	*Timestamp*	–	*Date and time of the latest update*

Note: Data dictionary for the ***Users*** *collection. This table outlines the structure, data types, and purpose of each field stored within MongoDB. It provides developers with a clear reference for how user records are formatted and maintained inside the system.*

Table 6.2 Product Collection

Field Name	***Data Type***	***Size***	***Description***
_id	*ObjectId*	–	*MongoDB primary key (automatically generated)*
id	*String*	*UUID*	*Unique ID for product references*
name	*String*	–	*Product name*

description	*Text*	–	*Full product description*
brand	*String*	–	*Product brand*
category	*String*	–	*Product category*
price	*Number*	–	*Product price (in USD or local currency)*
stock_quantity	*Integer*	–	*Available stock quantity*
specifications	*JSON*	–	*Key–value object describing product specifications*
images	*Array*	–	*Array of image URLs associated with the product*
features	*Array*	–	*List of product features*
status	*String*	–	*Product status (active, inactive, discontinued)*
created_at	*Timestamp*	–	*Date and time when the record was created*
updated_at	*Timestamp*	–	*Date and time of the latest product update*

*Note: Data dictionary for the **Products** collection. This table outlines the structure of product-related data stored in MongoDB, including descriptive fields, media assets, inventory information, and metadata used by the sales and quotation modules.*

Table 6.3 Quotes Collection

Field Name	***Data Type***	***Size***	***Description***
_id	*ObjectId*	-	*MongoDB primary key*
id	*String*	*UUID*	*Unique ID for external references*
customer_name	*String*	-	*Customer's full name*
customer_email	*String*	-	*Customer's email address*
company	*String*	-	*Company name (optional)*
phone	*String*	-	*Phone number (optional)*
items	*Array*	-	*Array of quote items with product details*
total_amount	*Number*	-	*Total quotation amount*
status	*String*	-	*Quote status (pending, approved, rejected, expired)*
notes	*Text*	-	*Customer notes*
admin_notes	*Text*	-	*Admin response/notes*

expires_at	*Timestamp*	*-*	*Quote expiration date*
created_at	*Timestamp*	*-*	*Record creation timestamp*
updated_at	*Timestamp*	*-*	*Last update timestamp*

Note: Data dictionary for the Quotes collection. This table outlines the structure of quotation-related data stored in MongoDB, including customer details, quote items, pricing information, status tracking, and metadata used by the sales and quotation modules

Table: 7.4 Tickets Collection

Field Name	***Data Type***	***Size***	***Description***
_id	*ObjectId*	*-*	*MongoDB primary key*
id	*String*	*UUID*	*Unique ID for external references*
customer_name	*String*	*-*	*Customer's full name*
customer_email	*String*	*-*	*Customer's email address*
subject	*String*	*-*	*Ticket subject*
issue_type	*String*	*-*	*Issue type (technical, billing, general, product, other)*
description	*Text*	*-*	*Issue description (minimum 5 characters)*
priority	*String*	*-*	*Priority level (low, medium, high, urgent)*
status	*String*	*-*	*Ticket status (open, in_progress, resolved, closed)*
attachments	*Array*	*-*	*Array of attachment URLs*
admin_notes	*Text*	*-*	*Admin response/notes*
assigned_to	*String*	*-*	*Assigned admin/team member*
resolved_at	*Timestamp*	*-*	*Resolution timestamp*
closed_at	*Timestamp*	*-*	*Closure timestamp*
created_at	*Timestamp*	*-*	*Record creation timestamp*
updated_at	*Timestamp*	*-*	*Last update timestamp*

Note: Data dictionary for the Tickets collection. This table describes the structure of support ticket data stored in MongoDB, including customer information, issue details,

status tracking, assignee information, and timestamps used by the support and admin modules

6.2.3 Use Case Diagram

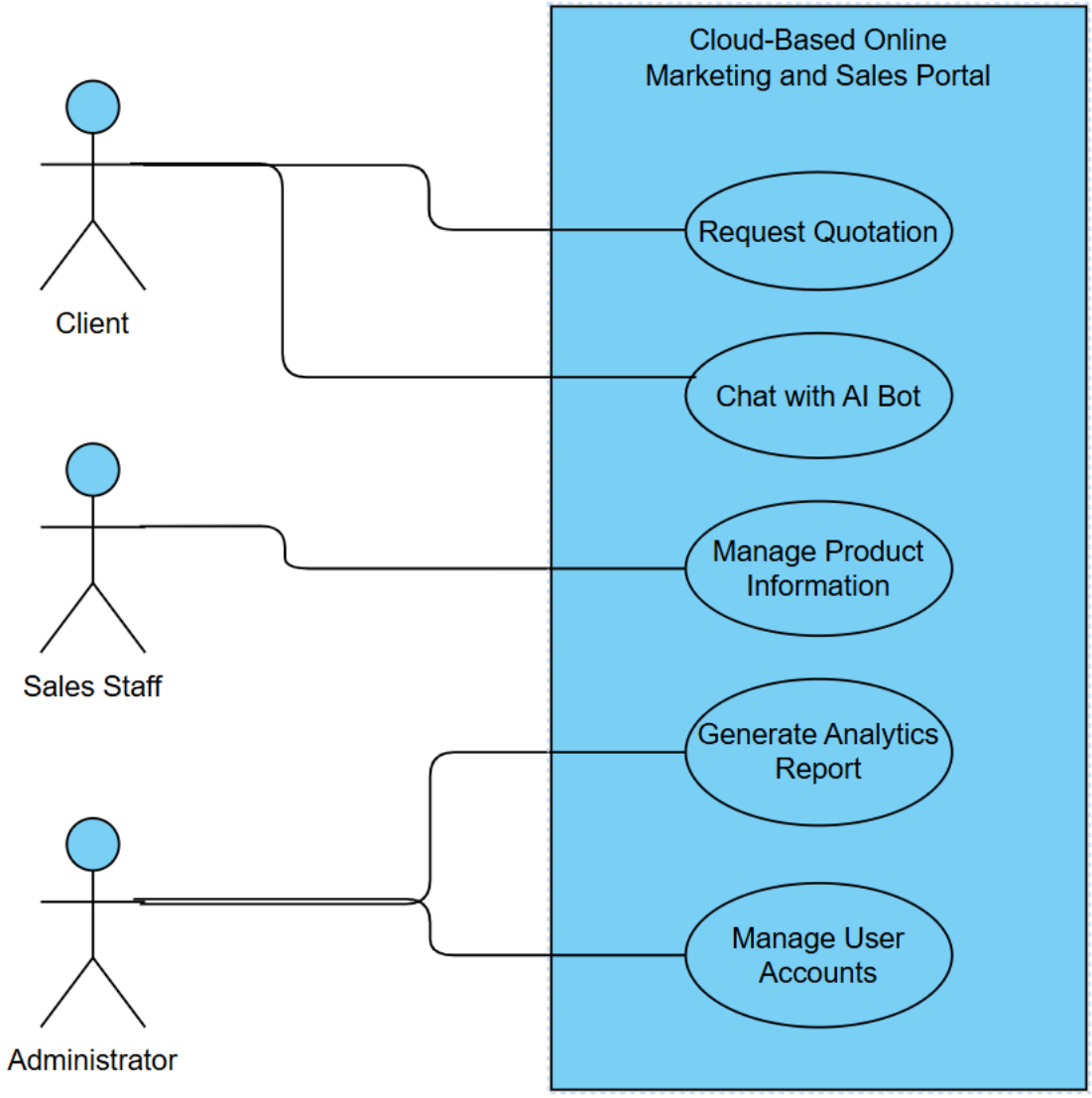


Figure 8. Use Case Diagram of the Proposed System

This case diagram illustrates the system's main interactions among three actors: clients, sales staff, and administrators. Clients request quotations and communicate with an AI bot, sales staff manage product details and quotations, while administrators handle user accounts, generate analytics reports, and oversee overall system operations.

Chapter VII: System Design Specification

7.1 Software Design Specification

P2.3: Generate_Product_Quotation

1. Process Name and ID

Process ID: P2.3

Process Name: Generate_Product_Quotation

Description: This process computes the quotation amount for IT products requested by a client and prepares the pricing summary used by sales representatives.

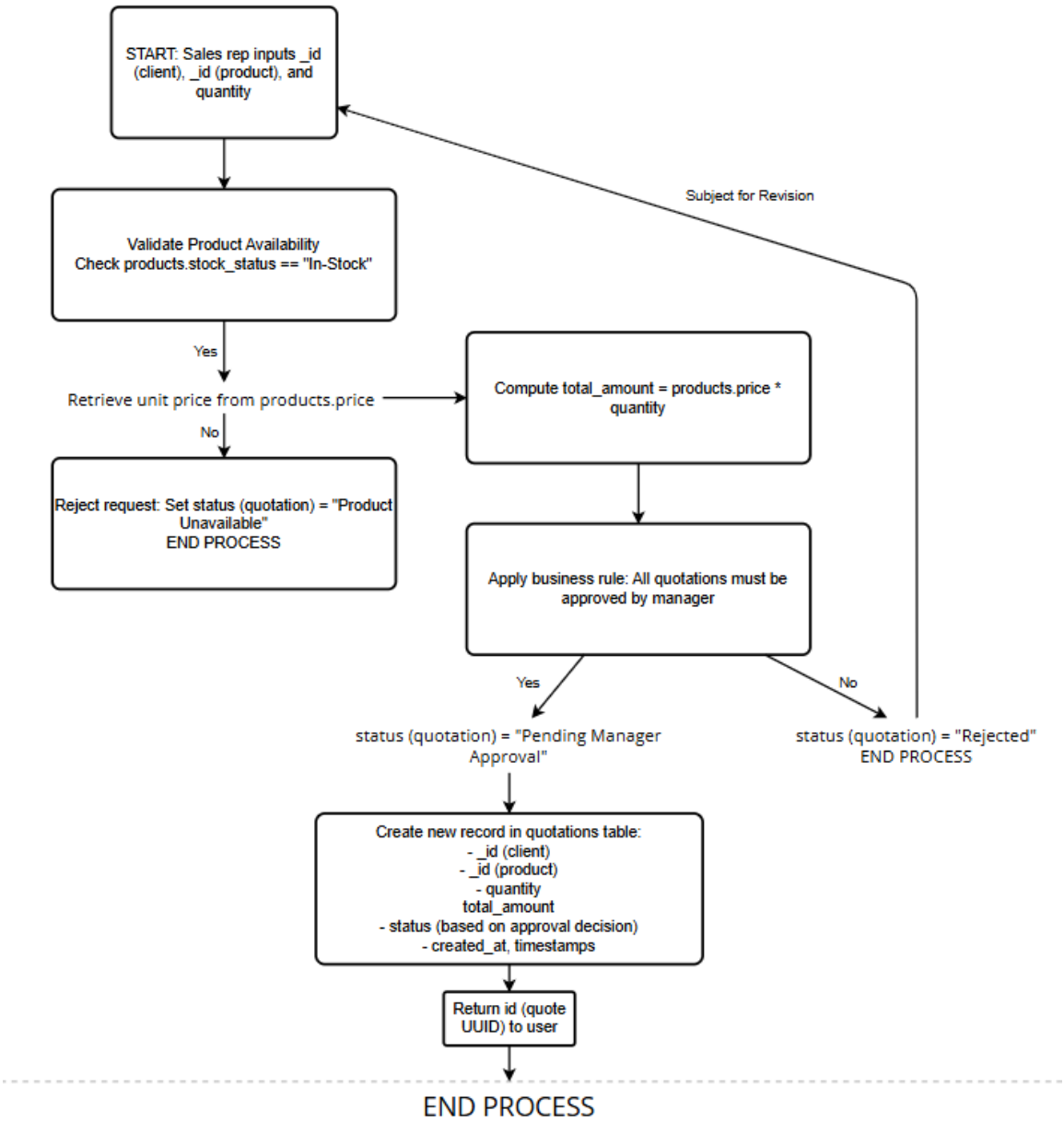
2. Input Data Flows

- **_id (client)** – unique identifier of the customer making the request
- **products entity** – IT items selected (_id (product), price, category)

3. Output Data Flows

- **_id (quote) –** system-generated unique quotation number
- **total_amount** – sum of all item prices (unit_price × quantity)
- **status** – default set to “Pending Approval”

4. Logic Description



5. Business Rules

- **BR1:** All quotations must be reviewed and approved by the Sales Manager.
- **BR2:** Only active IT products marked as "In-Stock" can be included in a quotation.
- **BR3:** Quotation validity period defaults to **30 days** unless specified otherwise.

7.2 Database Design

Below is a **Data Specification** section aligned with the Sales & Marketing Quotation Process **(P2.3 Generate Product Quotation).** This includes a **Data Dictionary** defining all core data elements used in the previous section / process.

Data Specification: **Unicom Sales & Marketing System (Quotation Module)**

Our Data Specification provides a detailed description of all data elements used, stored, or generated by the system. It ensures that every value—from client information to computed totals—follows consistent, precise, and well-defined standards.

The heart of this specification is the Data Dictionary, which outlines each data element's name, meaning, type, format, origin, and usage within the quotation process we are developing.

1. _id (client)

Attribute	Description	Example
Name/ID	_id	C1023
Description	Unique identifier for the customer requesting a quotation.	—
Type	VARCHAR(10)	—
Size/Length	Up to 10 characters	—
Format	Alphanumeric	C1023
Source	Retrieved from Customer Master Database	—
Usage	Input; used to fetch client details	—

2. _id (product)

Attribute	Description	Example
Name/ID	_id	P10120
Description	IT products selected by the client for quotation.	—
Type	VARCHAR(10)	—
Size/Length	10 characters	—
Format	Alphanumeric	—
Source	User input from Sales Representative	—

Usage	Input; used to compute gross amount	—

3. total_amount, price

Attribute	**Description**	**Example**
Name/ID	total_amount, price	45,000.00
Description	Sum of all item totals.	—
Type	Decimal (12,2)	—
Size/Length	12 digits incl. 2 decimals	—
Format	Numeric currency	—
Source	Computed by system	—
Usage	Internal calculation; stored in quotation record	—

4. status

Attribute	**Description**	**Example**
Name/ID	status	"Pending"
Description	Current approval state of the quotation.	—
Type	VARCHAR(10)	—
Size/Length	10 characters	—
Format	Predefined statuses: "Pending" and “Approved"	—
Source	Determined by decision rules	—
Usage	Workflow routing and status tracking	—

5. _id (quote)

Attribute	**Description**	**Example**
Name/ID	_id	b518e281-0b18-4d92-93c2-7ab76c91f145
Description	System-generated unique identifier for each quotation.	—
Type	VARCHAR(40)	—
Size/Length	40 characters max	—
Format	Alphanumeric	—
Source	Automatically generated by system	—
Usage	Primary key; used for tracking and retrieval	—

7.3 System Interface Design

This section presents the complete visual design of the Cloud-Based Sales and Marketing Portal, including all major user interfaces for both client-side and admin-side functionalities. Each interface was designed following modern UI/UX principles with emphasis on usability, accessibility, and responsive design using Next.js, React, and Tailwind CSS.interface.

1. Client Login Interface

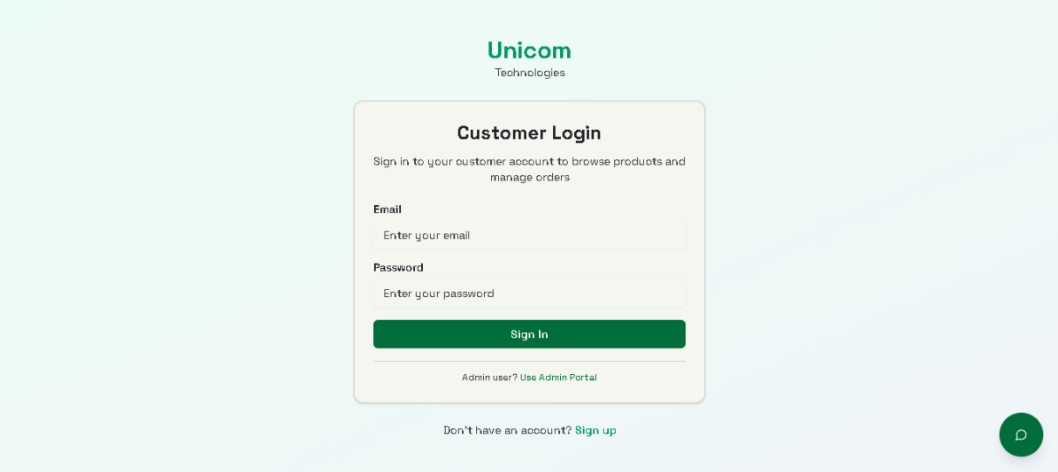


Figure 7.3 Client Login Interface

The client login interface features a minimalist design with "Unicom Technologies" branding at the top. It contains email and password input fields, a green "Sign In" button, and links for admin portal access and new user registration. A chatbot widget in the bottom-right provides immediate assistance. The design prioritizes simplicity with clear typography and white space while implementing password masking, email validation, and HTTPS security.

2. Admin Login Interface

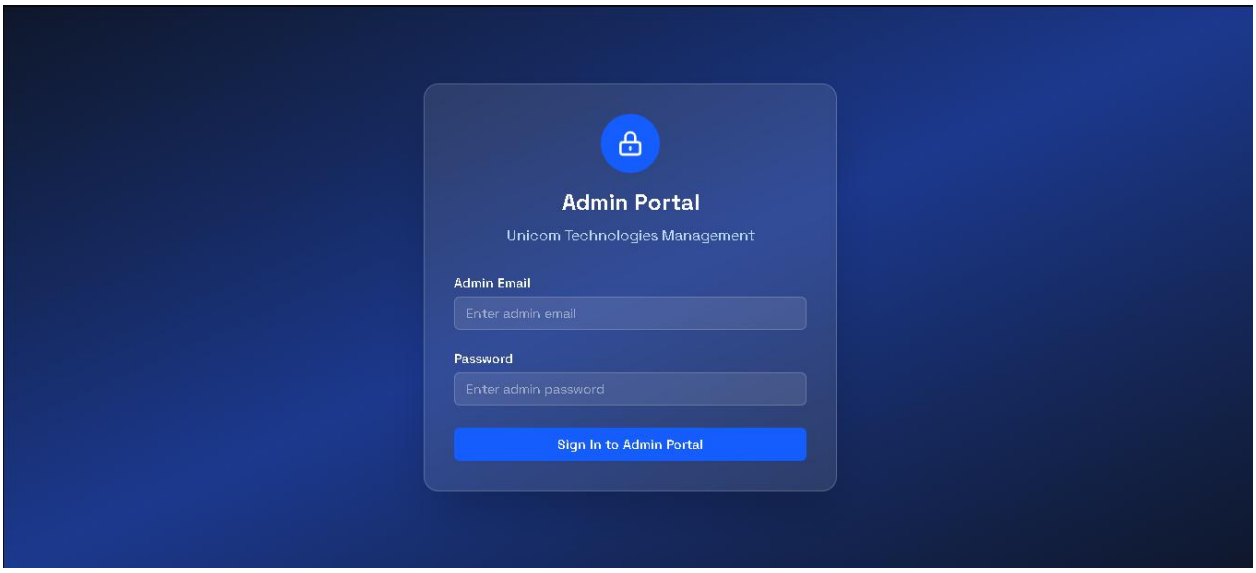


Figure 7.4 Login Dashboard Interface

The admin portal uses a dark blue gradient background to distinguish it from the customer interface. A lockicon reinforces security, with "Admin Portal" and "Unicom Technologies Management" titles clearly identifying restricted access. Semi-transparent fields for admin email and password lead to a bright blue sign-in button. The interface includes role-based authentication, session timeout monitoring, and login attempt tracking.

3. Admin Dashboard

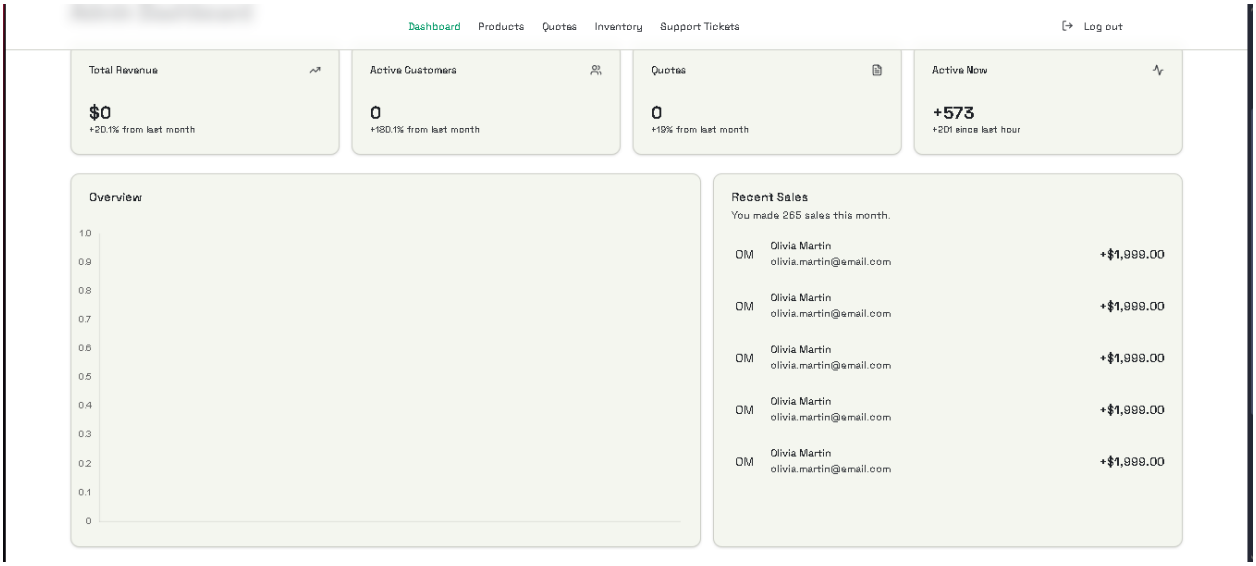


Figure 7.5 Admin Dashboard/Analytics Login Interface

The dashboard displays four KPI cards showing Total Revenue, Active Customers, Quotes, and Active Now metrics with percentage changes. The main area contains an "Overview" chart on the left for trend visualization and a "Recent Sales" panel on the right listing five transactions with customer details and amounts. Top navigation includes Dashboard, Products, Quotes, Inventory, and Support Tickets with a logout option.

4. Quotation Module interface

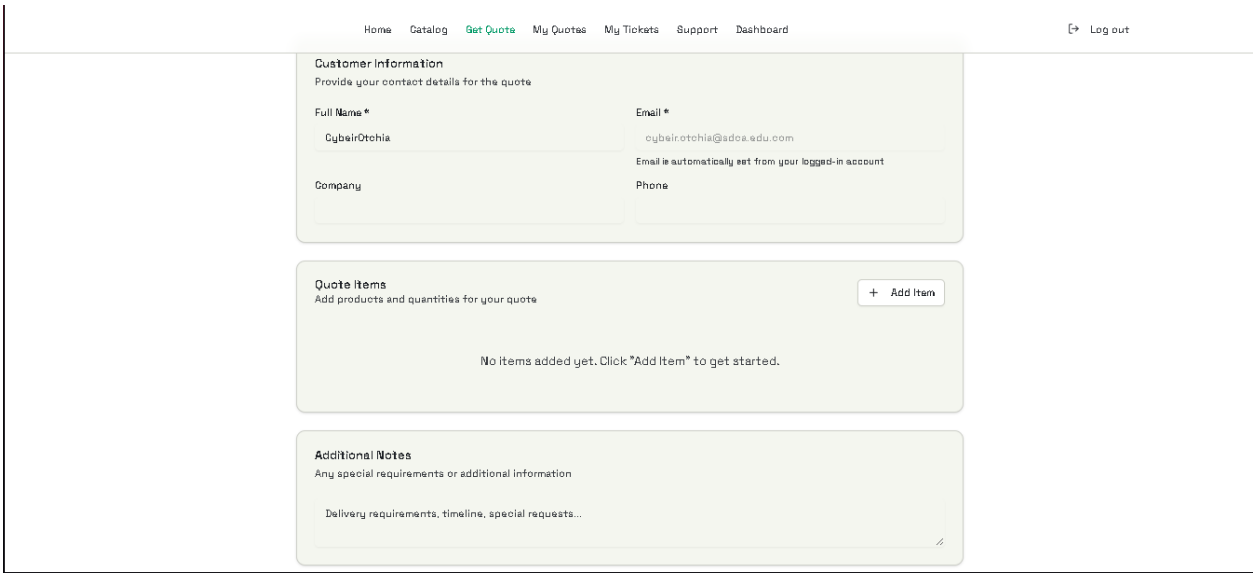


Figure 7.5 Quotation Form Module

The form is organized into three sections: Customer Information (with Full Name, Email, Company, Phone fields), Quote Items (featuring a "+ Add Item" button and empty state), and Additional Notes (for special requirements). Email is auto-filled from the logged-in account. The form includes real-time validation, auto-save functionality, and draft saving capabilities.

5. Ai Chatbot Pabel Interface

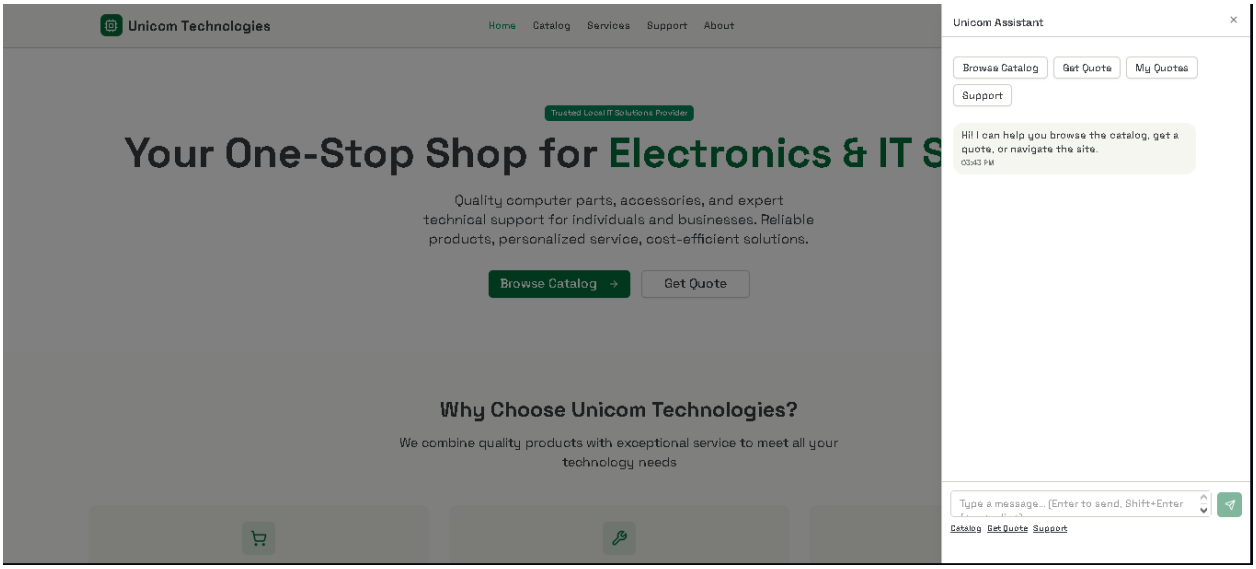


Figure 7.6 Chatbot Interaction Panel

The "Unicom Assistant" chatbot appears as a right-side panel with quick action buttons for Browse Catalog, Get Quote, My Quotes, and Support. The chat displays a welcome message offering help with catalog browsing, quotes, and navigation. A text input field at the bottom allows users to type questions. The chatbot handles product searches, navigation, FAQs, and order status inquiries.

7.4 Program Module Specification

Module: P2.3 – Generate Product Quotation

General Information

Project Identification

- **Project Name:** Unicom, Inc. IT Sales and Marketing Management System
- **Sponsor:** Unicom Finance Department
- **Project Manager:** Assigned Systems Analyst

Business Problem Description

The sales team needs a fast, accurate way to generate price quotations for IT products. Currently, quotations are prepared manually in Excel, resulting in errors in computation, historical status tracking, and approval routing. This slows down transactions and affects sales performance.

Project Purpose

To automate the generation, calculation, and approval routing of IT product quotations—ensuring accuracy, faster processing, and standardization across all sales representatives.

Includes:

- Generating quotations based on selected IT products
- Automatic calculation of total amount and final payable amount
- Workflow routing for approvals based on amount thresholds
- Producing organized quotation summaries

Excludes:

- Inventory management
- Shipping logistics
- Full customer onboarding or CRM functions
- eCommerce (Future Enhancement)

Project Requirements

Overall Description

The system will provide an automated module that computes quotation totals, applies business rules, generates IDs, determines approval status through decision logic, and produces structured quotation summaries. It interacts with product data, customer data, and user inputs with an AI chat support system.

System Features

- Automatic calculation of gross amount and total
- Rule-based approval status assignment
- System-generated quotation ID and summary
- Validation of active and in-stock products
- Integration with client and product master data
- Chatbot Support
- Data Analytics

System Features (Module-Level Functional Requirements)

System Feature 1: Compute Product Quotation

Description: Takes a list of selected IT products and computes all necessary amounts.

Functional Requirements:

1. The system shall compute **total_price** based on unit price × quantity.

System Feature 2: Apply Business Rules & Determine Approval Status

Description: Uses decision logic to select the correct approval path.

Functional Requirements:

1. The system shall check if the quotation is approved by the manager.
2. If yes, the system shall assign status = **"Approved"**.

If no, the system shall assign status = **"Rejected"**.

System Feature 3: Generate Quotation Output

Description: The module outputs a structured summary based on computed values.

Functional Requirements:

1. The system shall generate a unique **id (quote)** per transaction.
2. The system shall create a **quotation summary** containing client info, product list, and computed totals.
3. The system shall store the quotation for retrieval, analytics, and reporting.

Nonfunctional Requirements

Performance Requirements

- The module must compute totals and generate a quotation in **under 2 seconds**, even with up to 50 products.
- The system must handle up to **200 concurrent users** during peak sales activity.

Security Requirements

- Only authenticated sales representatives can generate quotations.

- Admin officers / Sales Supervisors can review all quotes for approval and otherwise.
- Sensitive financial data must be encrypted during transmission and storage.
- Data Analytics are available only for admin-level clearance.

Usability Requirements

- The quotation form must be easy to fill out, with clear input fields.
- All computed fields must refresh automatically without page reloads.
- Users should be able to download or preview the quotation summary.

Reliability Requirements

- System uptime must be **99%** during business hours.
- All computations must achieve **100% mathematical accuracy**.
- Failure recovery must allow regeneration of lost quotations using stored data.

Chapter VIII: Software Implementation

8.1 Programming Considerations, issue, and Tools.

- This system was built using **TypeScript, Tailwind CSS, Shadcn/Radix UI, pnpm, Lucide icons, JavaScript, Git + GitHub, React 18, Next.js 14, MongoDB Atlas, and Vercel** which together support cloud-based architecture. Key considerations included **Programming Considerations included:**

Programming Considerations

- Ensuring fast response times for quotation generation
- Securing user data using **bcrypt and secure cookies**
- Preventing API misuse through role-based access control
- Designing reusable react components for maintainability
- Ensuring database queries are optimized with proper indexing

Common issue Encountered

- Delays in fetching large product list
- Integrating the AI chatbot API with timeouts from external request
- Handling CORS conflicts during deployment

- Managing inconsistent data formats from user inputs
- UI responsiveness issues on lower-end devices

Tools used

- TypeScript - for developing large-scale, maintainable JavaScript applications
- Tailwind CSS – for rapidly building custom web interfaces directly within HTML markup
- Shadcn/Radix UI – for creating customizable, pre-styled UI components
- Pnpm – for a fast, disk-space-efficient package manager for Node.js
- Lucide icons – for adding scalable and consistent icons
- JavaScript – for creating interactive and dynamic web experiences
- React 18 – for building user interfaces (UIs) for web and native applications
- Next.js 14 – for building fast, full-stack web applications
- Visual Studio Code – for coding
- Github – version control and collaboration
- Postman/Bruno – API testing
- MongoDB Atlas – cloud database
- Vercel – hosting and CI/CD
- ClickUp – sprint planning and task management

8.2 System Requirements Specification

This section describes the minimum requirements needed to operate, test, and deploy the system,

Table 8.1 Hardware requirements

Device	Minimum Specification
Development Machine	Intel i5 / Ryzen 5, 8GB RAM, SSD storage
Server (Cloud)	MongoDB Atlas Free Tier, Vercel Cloud Functions
Client Devices	Any device with modern browser (Chrome, Edge, Firefox)

Note: Development requires Intel i5/Ryzen 5 processor, 8GB RAM, and SSD storage. The system is deployed on MongoDB Atlas Free Tier and Vercel Cloud Functions, accessible from any device with a modern browser (Chrome, Edge, Firefox).

8.2.2 Software Requirements

Table 8.2

Category	Required Software
Operating System	Windows 10+, macOS, Linux
Development Tools	VS Code, Node.js 18+, Git
Browser	Chrome/Edge latest version
Database	MongoDB Atlas
Deployment	Vercel

Note: The system requires Windows 10+, macOS, or Linux as the operating system; VS Code, Node.js 18+, and Git for development; Chrome/Edge latest version for browsing; MongoDB Atlas for database management; and Vercel for deployment.

8.2.3 Human Resource Requirements

Table 8.3

Role	Responsibilities
System Analyst	Documentation, diagrams, testing plans
Frontend Developer	UI/UX, Next.js pages, Tailwind components
Backend Developer	API routes, authentication, database logic
Scrum Master	Sprint planning, task delegation, monitoring

Note: The System Analyst handled documentation, diagrams, and testing plans; the Frontend Developer managed UI/UX design and implementation using Next.js and Tailwind; the Backend Developer built API routes, authentication, and database logic; and the Scrum Master coordinated sprint planning, task delegation, and progress monitoring.

8.3 Software Testing

8.3.1 Testing Activities

A combination of black-box testing, Agile sprint Testing and manual exploratory testing was applied.

Testing activities included:

- Unit testing – Checking individual backend endpoints
- Integration testing verifying communication between UI and API
- System Testing – full workflow execution (login – create quote – analytics)

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

- Chatbot Testing – checking accuracy of responses
- UAT (User Acceptance Testing) – Unicom staff validating real use cases
- Cross-browser testing – Chrome, Edge, Firefox
- Responsiveness Testing – Mobile and tablet views

8.3.2 Test Cases

Table 8.4 Test Case summary

Below is a concise but complete test-case table aligned with our system functions

Test Case	Description	Expected Result	Actual Result	Status
TC01	Login with Credentials	System grants Access	Passed	☑
TC02	Create new quotation	Quotation saved to DB	Passed	☑
TC03	Chatbot inquiry	AI returns correct response	Passed	☑
TC04	View analytics	Dashboard renders charts	Passed	☑
TC05	Add new product	Product added to DB	Passed	☑
TC06	Update quotation status	Status updated and displayed	Passed	☑
TC07	Attempt access without login	System denies access	Passed	☑

Note: *Seven test cases were executed with a 100% pass rate. All functionalities including login authentication (TC01), quotation creation (TC02), chatbot inquiries (TC03), analytics dashboard (TC04), product management (TC05), status updates (TC06), and access control (TC07) performed as expected, confirming the system is ready for deployment.*

8.4 Installation Process

This section shows the deployment and installation of the website step by step.

Step-by-step Deployment/Installation:

1. Clone Repository
   - git clone https://github.com/JustPres/unicom-backup.git
2. Install dependencies
   - pnpm install
3. Confiure Environment Variables
   - Create a .env file containing
     - MONGODB_URI
     - MONGO_DB
     - GEMINI_KEY_API
4. Run Development Server
   - pnpm dev
5. Test API Routes
   - Use Bruno/Postman to verify endpoints
6. Deploy to Vercel
   - Connect repository
   - Vercel automatically builds and deploys
   - Add environments variables in Vercel dashboard
7. Test Production Build
   - Login works
   - Test Quotation Generation and Chatbot responds and Analytics loads correctly

Chapter IX: Summary, Conclusion, and Recommendations

9.1 Software Maintenance Plan

Our Software Maintenance Plan outlines how the IT Sales and Marketing Management System,including the Quotation Module,will be updated, deployed, monitored, and maintained after initial release. The goal is to ensure continuous reliability, accuracy of computations (totals), compliance with business rules, and smooth user experience for sales representatives and managers.

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

Release and Deployment Strategy

9.1.1 Release Cadence

To ensure system stability while continuously improving features, the following release schedule will be adopted:

a. Weekly Hotfixes

- Address urgent issues such as:
- Incorrect quotation calculations (totals)
- Broken approval routing logic
- Bugs preventing quotation generation or product selection
- Limited to code changes with minimal system-wide impact.
- AI logic and functionality tests.

b. Monthly Minor Updates

- Improvements to module usability and UI/UX.
- Adjustments to computational logic and business logic.
- Optimization of calculation algorithms for efficiency.
- Minor integrations with product or customer data modules.
- AI logic response tests.

c. Quarterly Major Releases

- Introduction of new features such as:
    - Enhanced reporting for quotations, data analytics
    - Additional approval levels
- Integration with CRM and Sales Systems
- Expansion of data validation, security, and automation
- May include database schema changes or full module upgrades.

9.1.2 Environment Strategy

A multi-environment pipeline ensures that every change undergoes testing and validation before reaching production.

a. Development Environment

- Used by the programmer and system analyst for coding, debugging, and unit testing.

- All quotation-related logic (e.g., mathematical & approval logic) is tested here first.

b. Staging / QA Environment

- Mirrors the production environment as closely as possible.
- Used for:
  - Functional testing of the quotation module
  - Data validation using mock or anonymized customer/product data
  - Testing decision table logic under multiple scenarios
  - User Acceptance Testing (UAT) by sales representatives
- All major defects must be resolved before deployment.

c. Production Environment

- Live environment used by sales representatives and managers.
- Contains real customers, products, and price configurations.
- Only tested and approved builds are deployed here.
- Monitored continuously for performance, uptime, and error logs.

Promotion Process:

Developer → Git Repository → QA Review → Staging Validation → Manager Approval → Production Deployment

9.1.3 Rollback Plan

To protect system stability—especially for price computations and approval workflows—a clear rollback plan is required.

a. Pre-Release Backup

Before deploying updates, the following will be archived:

- Previous application build
- Database snapshot (quotation tables)
- Configuration files

b. Automated Rollback Triggers

Rollback is initiated if:

- Calculations produce incorrect totals
- Quotation generation fails or returns blank data
- Approval routing malfunctions (e.g., bypassing manager approval)
- Critical performance degradation (more than 2s per computation)

c. Rollback Procedure

1. Halt deployment and reactive monitoring.
2. Switch the system to “Maintenance Mode.”
3. Revert to the previously stable version stored in the repository.
4. Restore backed-up database snapshot if structural changes were made.
5. Validate functionality in production:
    - Generate a test quotation
    - Validate totals, and approval status
6. Disable “Maintenance Mode” and notify users of system restoration.

d. Post-Rollback Review

- Identify root cause of failure
- Document findings in maintenance logs
- Patch or refactor defective module in Development
- Schedule re-deployment after QA approval

9.2 Summary

Unicom Technologies' cloud-based sales and marketing platform was created to improve customer communication, automate laborious quote procedures, and offer real-time statistics. The team developed and improved modules including quotation management, product catalog, analytics dashboard, and AI chatbot support repeatedly using Agile methodology. Administrators and employees may effectively manage operations thanks to the system's consolidation of all corporate workflows onto a single web platform. The project effectively provided a simplified and structured solution through this integration, which addressed the operational inefficiencies, data fragmentation, and delays present in the organization's prior manual approach.

9.3 Conclusion

By updating the company's fundamental business procedures, the finished system successfully met its goals. Faster quotation preparation, better data accuracy, and increased client engagement through the AI chatbot function were all made possible by the cloud-hosted platform. Decision-makers also received quick information into customer behavior and sales success from the analytics module.

Moving to a cloud-based solution can provide Unicom Technologies with considerable operational benefits, as evidenced by the automation of workflow from product management to quotation approval, which showed notable gains in efficiency and dependability.

9.4 Recommendation

Future work may include enabling client registration and quotation monitoring to give customers additional visibility and self-service choices in order to further improve the system's performance and usability. The order processing procedure might be enhanced by incorporating safe online payment methods. We are also considering adding the discount and VAT fields on the sales quotation GUI for applicable clients of Unicom. Adding natural language refinement to the AI chatbot could result in more context-aware and customized conversations. Furthermore, creating a mobile application would enhance accessibility for users who are constantly on the go, and putting in place more robust security measures like audit logs and two-factor authentication would strengthen system security. Finally, to guarantee long-term system scalability and stability, it is highly advised to keep frequent upgrades and documentation.

Appendices

Appendix A

Screenshot: Successful Quote Creation

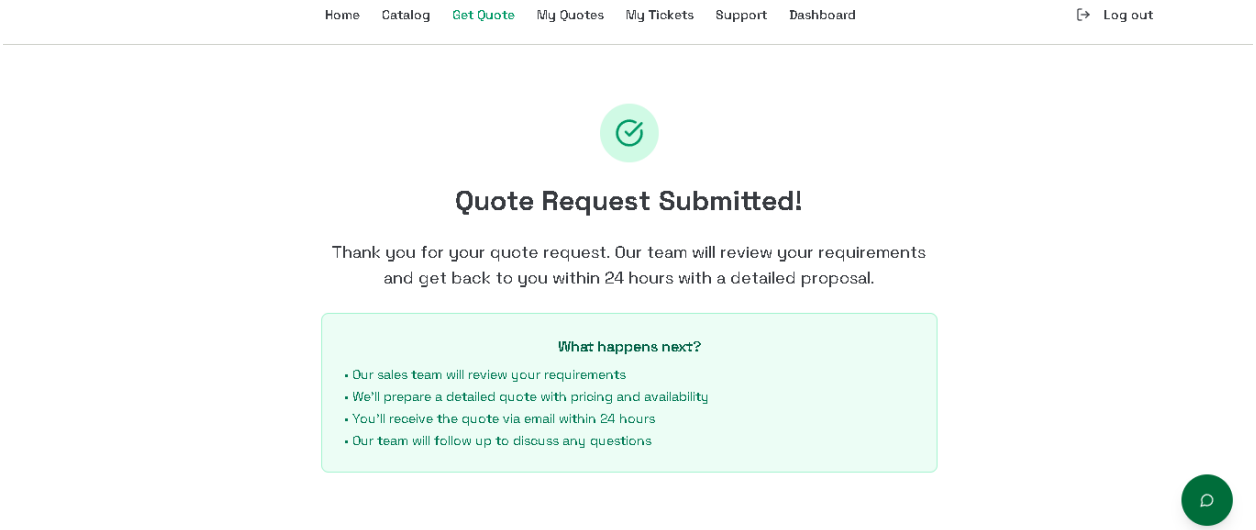


Figure A. Screenshot of Quote Creation Confirmation

This figure shows the successful creation of a quotation in the system, displaying the confirmation message and generated quote ID.

Appendix B

Exported PDF Quotation

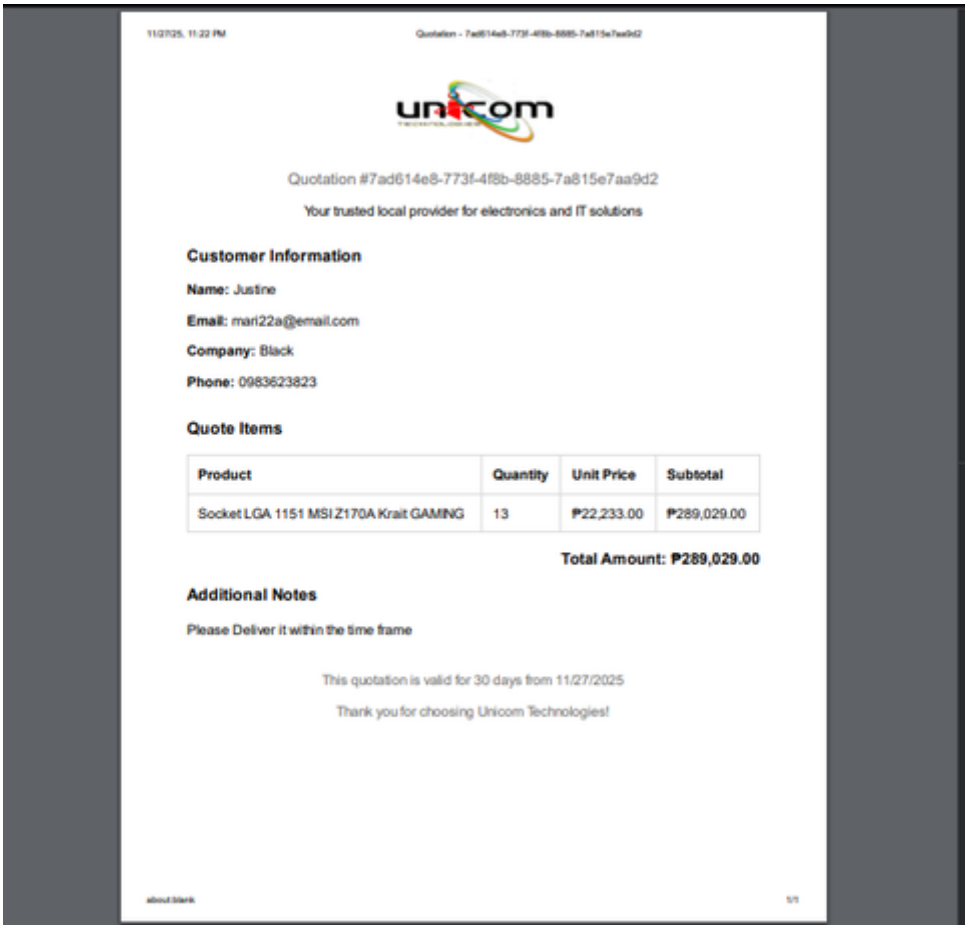


Figure B1. Generated PDF Quotation Template

This figure presents the PDF format of a generated quotation as it appears when exported from the system for client delivery

Appendix C

Google From response from the respondents

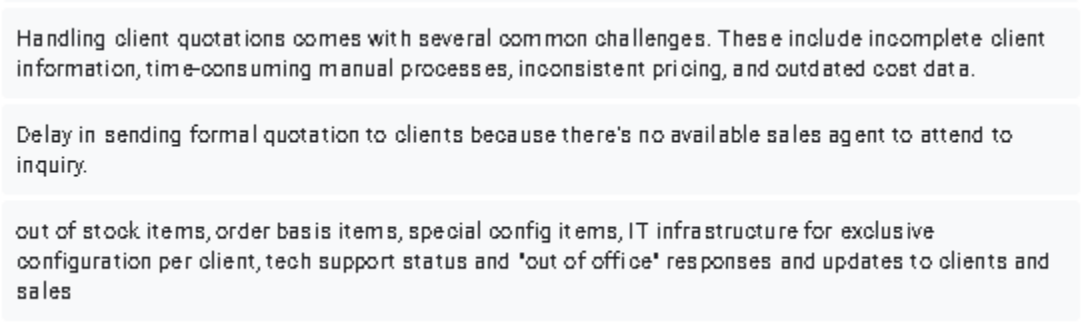


Figure C. Google Forms Response

This figure displays the aggregated responses from the stakeholder survey conducted via Google Forms, showing response distribution and completion rate.

Appendix D

Sample Google form questions

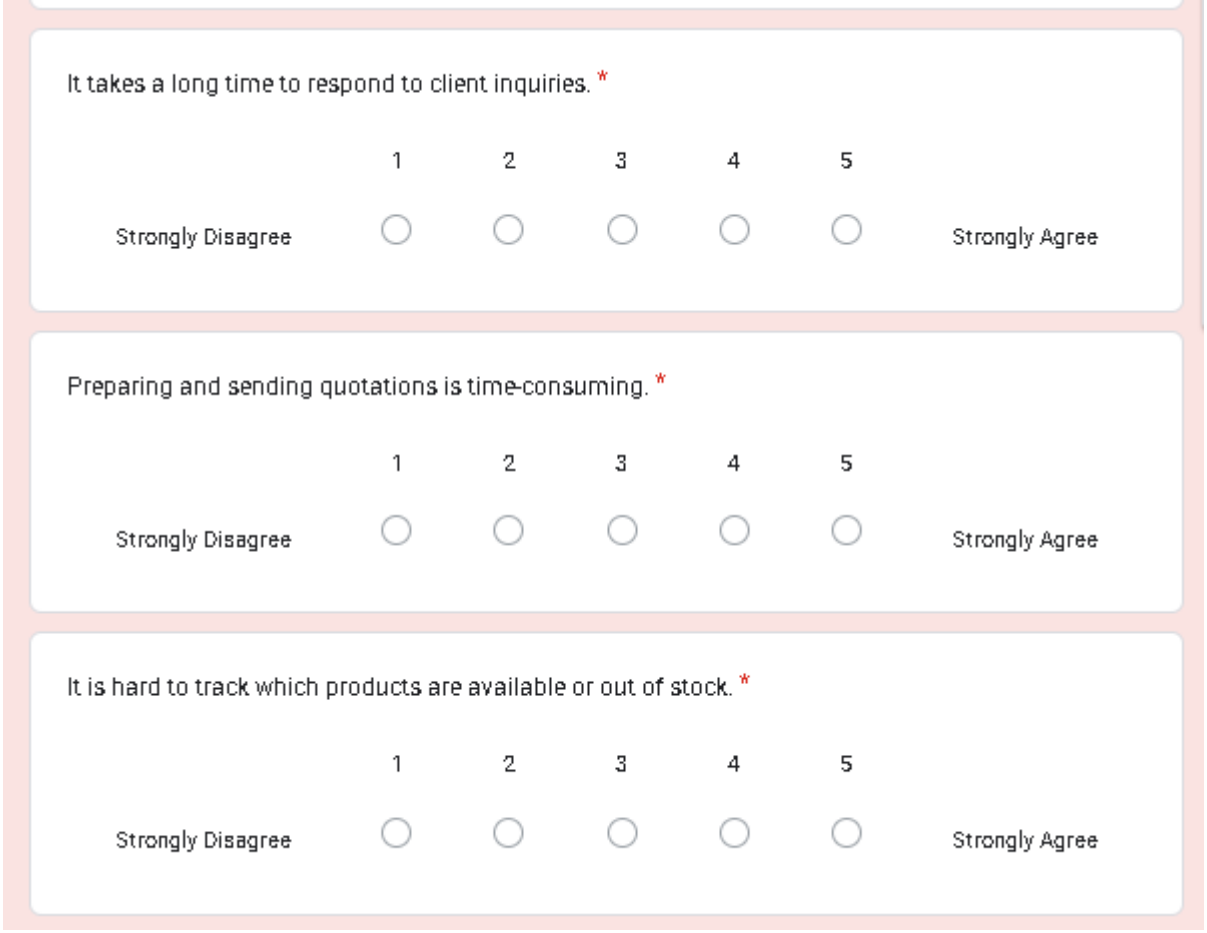


Figure D1. Sample Questions from Stakeholder Survey

This figure shows a portion of the survey questionnaire distributed to Unicom Technologies staff to gather requirements and feedback

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

Appendix E

Complete Interview Questionnaire

1. It takes a long time to respond to client inquiries.
2. Preparing and sending quotations is time-consuming.
3. It is hard to track which products are available or out of stock.
4. We do not have enough data or reports to guide our sales/marketing strategies.
5. We often lose or misplace important client quotation records.
6. We are open to using an AI chatbot to answer basic customer questions.
7. We need a central system to manage products, client information, and quotations.
8. What are the biggest problems you face in handling client quotations?
9. How do you currently prepare and send quotations to clients?
10. What kind of reports or analytics would help improve your work?
11. What features would you want in a new sales and marketing portal?

Appendix F

Curriculum Vitae



JUSTINE M. LOPEZ

Address: Lot 2 BLK 14 Rimas Street, Meadowood Executive Village, Bacoor City, Cavite 4102

Email: justine.lopez@sdca.edu.ph

Mobile: +63 970 398 660

EDUCATIONAL BACKGROUND

- **2024 - Present:** St. Dominic College of Asia, School of Communication, Multimedia, and Computer Studies
- **2022 - 2023:** Theresian School of Cavite, Senior High School, Grade 12 - ICT Strand
- **2021 - 2022:** Theresian School of Cavite, Senior High School, Grade 11 - ICT Strand
- **2017 - 2021:** General Emilio Aguinaldo National High School
- **2012 - 2017:** Governor PF Espiritu Elementary School
- **2011 - 2012:** Habay I Kinder School
- **2010 - 2011:** Panapaan V Kinder School

SKILLS

- Quick problem-solving
- Proficient with AI-driven software tools
- Video editing (educational content)
- Leadership

DENNIS CARLOS
Address: Blk 3 Lot 28, Ph2, Celina Royale Subdivision, Bacoor City

Email: dc.unicomtec@gmail.com
Mobile: +63 925 5000 493
LinkedIn/Facebook: www.facebook.com/unicomtec



PROFESSIONAL PROFILE

Motivated Account Executive with expertise in client relationship management, sales strategy, and business development.

EDUCATION

- **2023 - Present:** St. Dominic College of Asia, Bachelor of Science in Information Technology
- **2007 - 2010:** Arellano University, Bachelor of Science in Accountancy (Undergraduate, 3rd Year Standing)
- **2004 - 2007:** STI College, Certificate in Business Systems Programming (Mainframe, CICS)

WORK EXPERIENCE

- **2012 - Present:** Unicom Technologies Asia Pacific, Inc. - Project Development Manager
- **2007 - 2012:** SIM Computer Sales, Inc. - Senior Account Executive
- **2005 - 2007:** Micro Pacific Corporation - Corporate Account Executive
- **2004-2005 Mactelecom Corporation – Junior Computer Technician / Tech Sales.**

SKILLS

- Account Management
- Microsoft Suite
- QuickBooks, Peachtree, RDBMS
- CCTV/ZKTeko/ MS PowerPoint Certified
- Business Negotiations
- Team Leadership
- Revenue & Business Growth Strategy
- Client Engagement
- Long-term business relationship

OTHER SKILLS (Non-IT Related Certificates during COVID-19 Pandemic and other Course)

- HealthCare Services NC II
- Basic Nursing Skills Enhancement Program
- Basic IV Canulation & Injection Administration
- Practicum, Nursing Assistant (120Hrs), Velasco Hospital
- Practicing Covid-19 Preventive Measures in the Workplace, TESDA

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

- American College of Surgeons Committee on Trauma, Stop the Bleed Course
- Save a Life, Basic Life Support Certification Course
- Community Basic Life Support & First Aid Training
- Community Firefighter and First Responder Training Course (CFFR)
- Basic Rope Rescue Training
- JPIA, Junior Philippine Institute of Accountants Training on SFAS 1, Arellano University

RECOGNITIONS

- SIM Computer Sales, Top Sales of the Year Award, 2006
- Most Compassionate Scholar, Beatitudes Technological & Theological College
- Leadership Award, Beatitudes Technological & Theological College
- 9th Achiever Award, Beatitudes Technological & Theological College

REFERENCES

- Roselle Claros – SouthCity Medical Center / CNA, 0932 861 3774colan53@yahoo.com
- Armand Cruz – Systems Admin, CARE SSC, +63 923 959 6652, armand.cruz@care.org
- Mr. Jun Jamias – Country Director, CARE SSC, jun.jamias@care.org
- Mr. Jonathan Te- Vice President, JT Triumph Manila, jonathan_te95@yahoo.com

CHRISTIAN IVAN A. LOS BAÑOS
Address: Blk 7 Lot 68, Lessandra Phase 3, Barangay Salinas II, Bacoor, Cavite
Email: christian.losbanos@sdca.edu.ph
Mobile: +63 927 833 6509
Portfolio: http://christian-lb.github.io/portfolio/index



EDUCATIONAL BACKGROUND
2023 – Present
Saint Dominic College of Asia
Bachelor of Science in Information Technology
2016
Emilio Aguinaldo College
Bachelor of Science in Nursing

PROFESSIONAL PROFILE

I am a 27-year-old student currently studying at Saint Dominic College of Asia. I am passionate about helping others, programming, and creative design. I see learning as a lifelong journey driven by both personal growth and professional development. With strong faith and dedication, I strive to apply my skills to make a meaningful impact, especially through technology and design.

HARD SKILLS

- Computer Programming
- MS Word / PowerPoint Proficiency
- Playing the Guitar
- Singing

SOFT SKILLS

- Good Listener
- Persistent
- Dedicated



MARIA ROLLAINNE R. LINDOG

Block 9 Lot 16 Southern City 1 Subdivision, Tanzang Luma IV, Imus City, Cavite 4103

Email address: mariarollainne.lindog@sdca.edu.ph

Home phone number: +63 468514054

Mobile number: +63 960 389 1981

EDUCATIONAL ATTAINMENT:

St. Dominic College of Asia	Bachelor of Science in Information Technology AY 2023-2024 up to present
AMA Computer Learning Center	System Design And Programming Graduated June 2016 to April 2018
St. Dominic College of Asia	Bachelor of Science in Hotel Management Undergraduate June 2015 to April 2016
Jesus Good Shepherd School	Secondary June 2014 to April 2015
La Asuncion Montessori of Cavite	Secondary June 2012 to April 2014 Primary & Pre School June 2005 to April 2012

EMPLOYMENT HISTORY:

January 2019 to January 2020: Assistant Staff at Paredes Dental Clinic Nueno Street, Imus City, Cavite

PERSONAL DETAILS:

Date of Birth: February 24, 1998 Place of Birth: Manila

Gender: Female Status: Single Height: 160 cm

Weight: 58 kg

Religion: Roman Catholic

Father's Name: Raul M. Lindog

Mother's Name: Maria Leonor R. Lindog

CHARACTER REFERENCES:

ESTER PAREDES D.M.D**.**

Paredes Dental Clinic Nueno Street, Imus Cavite

WENCY PROVIDO

Jesus Good Shepherd School Palico St. Imus City, Cavite

Appendix G

Codes

About/page

```
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
"@/components/ui/card"
import { Button } from "@/components/ui/button"
import { Users, Award, Clock, MapPin, Target, Heart } from "lucide-react"
import { VisitorHeader } from "@/components/visitor-header"

export default function AboutPage() {
  const stats = [
    { icon: <Users className="h-6 w-6" />, label: "Happy Customers", value:
"2,500+" },
    { icon: <Clock className="h-6 w-6" />, label: "Years in Business", value:
"15+" },
    { icon: <Award className="h-6 w-6" />, label: "Certified Technicians", value:
"8" },
    { icon: <MapPin className="h-6 w-6" />, label: "Service Areas", value: "5" },
  ]

  const values = [
    {
      icon: <Target className="h-8 w-8" />,
      title: "Quality First",
      description: "We never compromise on quality. Every product and service meets
our high standards.",
    },
    {
      icon: <Heart className="h-8 w-8" />,
      title: "Customer Care",
      description: "Your satisfaction is our priority. We build lasting relationships with
personalized service.",
    },
    {
      icon: <Award className="h-8 w-8" />,
      title: "Expertise",
      description: "Our certified technicians stay current with the latest technology and
best practices.",
    },
  ]

  return (
    <div className="min-h-screen bg-background">
```

```
<VisitorHeader />
<main className="container mx-auto px-4 py-8">
{/* Hero Section */}
<div className="text-center mb-12">
 <h1 className="text-4xl font-bold mb-4 text-balance">About Unicom
Technologies</h1>
 <p className="text-xl text-muted-foreground max-w-3xl mx-auto text-pretty">
  Your trusted local partner for electronics, IT solutions, and technical support.
Serving the community with
  reliable technology solutions since 2009.
 </p>
</div>

{/* Stats */}
<div className="grid grid-cols-2 md:grid-cols-4 gap-4 mb-12">
 {stats.map((stat, index) => (
  <Card key={index} className="text-center">
   <CardContent className="pt-6">
    <div className="text-emerald-600 mb-2 flex justify-
center">{stat.icon}</div>
    <div className="text-2xl font-bold mb-1">{stat.value}</div>
    <div className="text-sm text-muted-foreground">{stat.label}</div>
   </CardContent>
  </Card>
 ))}
</div>

{/* Story Section */}
<div className="grid grid-cols-1 lg:grid-cols-2 gap-8 mb-12">
 <Card>
  <CardHeader>
   <CardTitle>Our Story</CardTitle>
  </CardHeader>
  <CardContent className="space-y-4">
   <p className="text-muted-foreground">
    Founded in 2009 by tech enthusiasts who saw a need for reliable, affordable
technology solutions in our
    community, Unicom Technologies has grown from a small repair shop to a
comprehensive IT solutions
    provider.
   </p>
   <p className="text-muted-foreground">
    We started with a simple mission: make technology accessible and reliable
for everyone. Today, we serve
```

```
            over 2,500 satisfied customers, from individual consumers to small
businesses, providing everything from
            computer parts to complete IT infrastructure solutions.
          </p>
          <p className="text-muted-foreground">
            Our team of certified technicians combines years of experience with ongoing
training to stay current with
            the latest technology trends and best practices.
          </p>
        </CardContent>
      </Card>

      <Card>
        <CardHeader>
          <CardTitle>Our Mission</CardTitle>
        </CardHeader>
        <CardContent className="space-y-4">
          <p className="text-muted-foreground">
            To be the most trusted technology partner in our community by delivering
reliable products, exceptional
            service, and cost-effective solutions that help our customers succeed in an
increasingly digital world.
          </p>
          <div className="space-y-2">
            <h4 className="font-medium">What Sets Us Apart:</h4>
            <ul className="space-y-1 text-sm text-muted-foreground">
              <li className="flex items-center">
                <div className="w-1.5 h-1.5 bg-emerald-600 rounded-full mr-2" />
                Personalized service and local expertise
              </li>
              <li className="flex items-center">
                <div className="w-1.5 h-1.5 bg-emerald-600 rounded-full mr-2" />
                Competitive pricing without compromising quality
              </li>
              <li className="flex items-center">
                <div className="w-1.5 h-1.5 bg-emerald-600 rounded-full mr-2" />
                Comprehensive warranty and support
              </li>
              <li className="flex items-center">
                <div className="w-1.5 h-1.5 bg-emerald-600 rounded-full mr-2" />
                Community-focused approach
              </li>
            </ul>
          </div>
```

```
          </CardContent>
        </Card>
      </div>

      {/* Values */}
      <div className="mb-12">
        <h2 className="text-3xl font-bold text-center mb-8">Our Values</h2>
        <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
          {values.map((value, index) => (
            <Card key={index} className="text-center">
              <CardHeader>
                <div className="text-emerald-600 mb-2 flex justify-
center">{value.icon}</div>
                <CardTitle>{value.title}</CardTitle>
              </CardHeader>
              <CardContent>
                <p className="text-muted-foreground">{value.description}</p>
              </CardContent>
            </Card>
          ))}
        </div>
      </div>

      </main>
    </div>
  )
}
```

Admin/login

```
export default function AdminLoginPage() {
  return (
    <div className="min-h-screen bg-gradient-to-br from-slate-900 via-blue-900 to-slate-900 flex items-center justify-center p-4">
      <div className="w-full max-w-md">
        <div className="bg-white/10 backdrop-blur-md rounded-2xl p-8 border border-white/20 shadow-2xl">
          <div className="text-center mb-8">
            <div className="w-16 h-16 bg-blue-600 rounded-full flex items-center justify-center mx-auto mb-4">
              <svg className="w-8 h-8 text-white" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                <path
                  strokeLinecap="round"
                  strokeLinejoin="round"
                  strokeWidth={2}
                  d="M12 15v2m-6 4h12a2 2 0 002-2v-6a2 2 0 00-2-2H6a2 2 0 00-2 2v6a2 2 0 002 2zm10-10V7a4 4 0 00-8 0v4h8z"
                />
              </svg>
            </div>
            <h1 className="text-2xl font-bold text-white mb-2">Admin Portal</h1>
            <p className="text-blue-200">Unicom Technologies Management</p>
          </div>

          <AdminLoginForm />
        </div>
      </div>
    </div>
  )
}

import { AdminLoginForm } from "@/components/auth/admin-login-form"
```

Admin/Ticket

```
"use client"

import { useEffect, useState } from "react"
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card"
import { Badge } from "@/components/ui/badge"
import { Button } from "@/components/ui/button"
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from
"@/components/ui/select"
import { Textarea } from "@/components/ui/textarea"
import { Navigation } from "@/components/navigation"
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger } from
"@/components/ui/dialog"
import { toast } from "sonner"
import type { SupportTicket } from "@/lib/tickets"

export default function AdminTicketsPage() {
  const [tickets, setTickets] = useState<SupportTicket[]>([])
  const [loading, setLoading] = useState(true)
  const [selectedTicket, setSelectedTicket] = useState<SupportTicket | null>(null)
  const [updateData, setUpdateData] = useState({
    status: "",
    adminNotes: "",
    assignedTo: "",
  })
  const [admins, setAdmins] = useState<Array<{ id: string; name: string; email:
string }>>([])

  useEffect(() => {
    const fetchAdmins = async () => {
      try {
        const response = await fetch('/api/users/admins')
        if (response.ok) {
          const data = await response.json()
          setAdmins(data.users || [])
        }
      } catch (error) {
        console.error('Failed to fetch admins:', error)
      }
    }

    fetchTickets()
    fetchAdmins()
  }, [])
```

```
const fetchTickets = async () => {
  try {
    const response = await fetch("/api/tickets")
    if (response.ok) {
      const data = await response.json()
      setTickets(data.tickets)
    }
  } catch (error) {
    console.error("Failed to fetch tickets:", error)
  } finally {
    setLoading(false)
  }
}

const handleUpdateTicket = async (ticketId: string) => {
  try {
    const response = await fetch(`/api/tickets/${ticketId}`, {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(updateData),
    })

    if (response.ok) {
      toast.success("Ticket updated successfully!")
      fetchTickets() // Refresh the list
      setSelectedTicket(null)
      setUpdateData({ status: "", adminNotes: "", assignedTo: "" })
    } else {
      toast.error("Failed to update ticket")
    }
  } catch (error) {
    toast.error("Failed to update ticket")
  }
}

const getStatusColor = (status: string) => {
  switch (status) {
    case "open": return "bg-blue-100 text-blue-800"
    case "in_progress": return "bg-yellow-100 text-yellow-800"
    case "resolved": return "bg-green-100 text-green-800"
    case "closed": return "bg-gray-100 text-gray-800"
    default: return "bg-gray-100 text-gray-800"
  }
```

```
}

const getPriorityColor = (priority: string) => {
  switch (priority) {
    case "urgent": return "bg-red-100 text-red-800"
    case "high": return "bg-orange-100 text-orange-800"
    case "medium": return "bg-yellow-100 text-yellow-800"
    case "low": return "bg-green-100 text-green-800"
    default: return "bg-gray-100 text-gray-800"
  }
}

return (
  <div className="min-h-screen bg-gray-50">
    <Navigation centered />

    <div className="container mx-auto py-8">
      <div className="mb-6">
        <h1 className="text-3xl font-bold text-gray-900">Support Tickets</h1>
        <p className="text-gray-600 mt-2">Manage customer support requests</p>
      </div>

      {loading ? (
        <div className="text-center py-8">Loading tickets...</div>
      ) : tickets.length === 0 ? (
        <Card>
          <CardContent className="text-center py-8">
            <p className="text-gray-500">No support tickets found.</p>
          </CardContent>
        </Card>
      ) : (
        <div className="space-y-4">
          {tickets.map((ticket) => (
            <Card key={ticket.id}>
              <CardHeader>
                <div className="flex justify-between items-start">
                  <div>
                    <CardTitle className="text-lg">{ticket.subject}</CardTitle>
                    <p className="text-sm text-gray-600 mt-1">
                      From: {ticket.customerName} ({ticket.customerEmail})
                    </p>
                    <p className="text-sm text-gray-600">
                      Created: {new Date(ticket.createdAt).toLocaleDateString()}
                    </p>
```

```
            </div>
            <div className="flex gap-2">
              <Badge className={getStatusColor(ticket.status)}>
                {ticket.status.replace("_", " ")}
              </Badge>
              <Badge className={getPriorityColor(ticket.priority)}>
                {ticket.priority}
              </Badge>
            </div>
          </div>
        </CardHeader>
        <CardContent>
          <p className="text-gray-700 mb-4">{ticket.description}</p>

          <div className="flex gap-2">
            <Dialog>
              <DialogTrigger asChild>
                <Button
                  variant="outline"
                  onClick={() => {
                    setSelectedTicket(ticket)
                    setUpdateData({
                      status: ticket.status,
                      adminNotes: ticket.adminNotes || "",
                      assignedTo: ticket.assignedTo || "",
                    })
                  }}
                >
                  Update Ticket
                </Button>
              </DialogTrigger>
              <DialogContent className="max-w-md">
                <DialogHeader>
                  <DialogTitle>Update Ticket</DialogTitle>
                </DialogHeader>
                <div className="space-y-4">
                  <div>
                    <label className="text-sm font-medium">Status</label>
                    <Select
                      value={updateData.status}
                      onValueChange={(value) => setUpdateData({ ...updateData,
status: value })}
                    >
                      <SelectTrigger>
```

```
                    <SelectValue />
                  </SelectTrigger>
                  <SelectContent>
                    <SelectItem value="open">Open</SelectItem>
                    <SelectItem value="in_progress">In Progress</SelectItem>
                    <SelectItem value="resolved">Resolved</SelectItem>
                    <SelectItem value="closed">Closed</SelectItem>
                  </SelectContent>
                </Select>
              </div>

              <div>
                <label className="text-sm font-medium">Admin Notes</label>
                <Textarea
                  value={updateData.adminNotes}
                  onChange={(e) => setUpdateData({ ...updateData, adminNotes:
e.target.value })}
                  placeholder="Add admin notes..."
                  rows={3}
                />
              </div>

              <div>
                <label className="text-sm font-medium">Assigned To</label>
                <Select
                  value={updateData.assignedTo}
                  onValueChange={(value) => setUpdateData({ ...updateData,
assignedTo: value })}
                >
                  <SelectTrigger>
                    <SelectValue placeholder="Unassigned" />
                  </SelectTrigger>
                  <SelectContent>
                    <SelectItem value="unassigned">Unassigned</SelectItem>
                    {admins.map((admin) => (
                      <SelectItem key={admin.id} value={admin.id}>
                        {admin.name} ({admin.email})
                      </SelectItem>
                    ))}
                  </SelectContent>
                </Select>
              </div>

              <Button
```

```
                    onClick={() => handleUpdateTicket(ticket.id)}
                    className="w-full"
                  >
                    Update Ticket
                  </Button>
                </div>
              </DialogContent>
            </Dialog>
          </div>
        </CardContent>
      </Card>
    ))}
  </div>
 ))}
 </div>
 </div>
 )
}
```

Api\Analytics

```
import { NextResponse } from "next/server"
import { getDb } from "@/lib/db"
import { startOfDay, subDays } from 'date-fns'

export async function GET() {
 console.log('Analytics overview endpoint hit')
 try {
  const db = await getDb()
  console.log('Database connection successful')
  const now = new Date()
  const thirtyDaysAgo = subDays(now, 30)
  console.log('Fetching data from', thirtyDaysAgo, 'to', now)

  // Get sales data
  const salesData = await db.collection('orders').aggregate([
   { $match: { createdAt: { $gte: thirtyDaysAgo } } },
   {
    $group: {
     _id: { $dateToString: { format: "%Y-%m-%d", date: "$createdAt" } },
     total: { $sum: "$totalAmount" },
     count: { $sum: 1 }
    }
   },
   { $sort: { _id: 1 } }
```

```
]).toArray()

// Get quote conversion data
const quotesData = await db.collection('quotes').aggregate([
  { $match: { createdAt: { $gte: thirtyDaysAgo } } },
  {
    $group: {
      _id: "$status",
      count: { $sum: 1 }
    }
  }
]).toArray()

// Get customer data
const customersData = await db.collection('users').aggregate([
  { $match: { role: 'customer' } },
  { $count: 'total' }
]).toArray()

// Get support tickets data
const ticketsData = await db.collection('tickets').aggregate([
  {
    $group: {
      _id: "$status",
      count: { $sum: 1 }
    }
  }
]).toArray()

// For testing - return sample data
console.log('Using sample data for testing')
const sampleData = [
  {
    _id: '1',
    name: 'Sample Product 1',
    price: 100,
    quoteCount: 5,
    totalQuotedValue: 500
  },
  {
    _id: '2',
    name: 'Sample Product 2',
    price: 200,
    quoteCount: 3,
```

```
    totalQuotedValue: 600
  }
]

console.log('Returning sample data response')
const responseData = {
  sales: {
    daily: [],
    total: 0,
    orderCount: 0
  },
  customers: {
    total: 0,
    newThisMonth: 0
  },
  quotes: {
    byStatus: {},
    total: 0
  },
  tickets: {
    byStatus: {},
    total: 0
  },
  products: {
    mostQuoted: sampleData
  }
}

console.log('Returning sample data:', JSON.stringify(responseData, null, 2))
return NextResponse.json(responseData)

// Uncomment below to restore database query
/*
// Database diagnostics
console.log('=== DATABASE DIAGNOSTICS ===')
console.log('Database name:', db.databaseName)

// List all collections
const collections = await db.listCollections().toArray()
console.log('Available collections:', collections.map(c => c.name))

// Check if quotes collection exists
const quotesCollection = db.collection('quotes')
const quotesCount = await quotesCollection.estimatedDocumentCount()
```

```
console.log(`Found ${quotesCount} documents in 'quotes' collection`)

let mostQuotedProducts = [];

if (quotesCount > 0) {
  // Get a sample quote to check structure
  const sampleQuote = await quotesCollection.findOne({})
  console.log('Sample quote structure:', JSON.stringify({
    _id: sampleQuote?._id,
    items: sampleQuote?.items?.map((i: any) => ({
      productId: i.productId,
      quantity: i.quantity,
      price: i.price
    }))
  }, null, 2))

  console.log('Running aggregation pipeline...')
  mostQuotedProducts = await quotesCollection.aggregate([
    // Unwind the items array to process each product in each quote
    { $unwind: "$items" },
    {
      $group: {
        _id: "$items.productId",
        quoteCount: { $sum: 1 },
        totalQuotedValue: { $sum: { $multiply: ["$items.quantity", "$items.price"] } }
      }
    },
    { $sort: { quoteCount: -1 } },
    { $limit: 5 },
    {
      $lookup: {
        from: "products",
        localField: "_id",
        foreignField: "_id",
        as: "product"
      }
    },
    { $unwind: "$product" },
    {
      $project: {
        _id: "$product._id",
        name: "$product.name",
        price: "$product.price",
        quoteCount: 1,
```

```
        totalQuotedValue: 1
      }
    }
  ]).toArray()

  console.log('Most quoted products from database:', {
    count: mostQuotedProducts?.length || 0,
    sample: mostQuotedProducts?.slice(0, 2) // Show first 2 items if available
  })
} else {
  console.log('No quotes found in the database')
}
*/

return NextResponse.json({
  sales: {
    daily: salesData.map(item => ({
      date: item._id,
      total: item.total,
      count: item.count
    })),
    total: salesData.reduce((sum, item) => sum + (item.total || 0), 0),
    orderCount: salesData.reduce((sum, item) => sum + (item.count || 0), 0)
  },
  quotes: {
    byStatus: quotesData.reduce((acc, item) => ({
      ...acc,
      [item._id]: item.count
    }), {}),
    total: quotesData.reduce((sum, item) => sum + (item.count || 0), 0)
  },
  customers: {
    total: customersData[0]?.total || 0,
    newThisMonth: 0 // Would need registration date in user schema
  },
  tickets: {
    byStatus: ticketsData.reduce((acc, item) => ({
      ...acc,
      [item._id]: item.count
    }), {}),
    total: ticketsData.reduce((sum, item) => sum + (item.count || 0), 0)
  },
  products: {
    mostQuoted: [] // Using empty array as fallback
```

```
    }
   })
  } catch (error) {
   console.error('Error in analytics overview:', error)
   const errorMessage = error instanceof Error ? error.message : 'Unknown error
occurred'
   const errorStack = error instanceof Error && process.env.NODE_ENV ===
'development' ? error.stack : undefined

   return NextResponse.json(
    {
     error: 'Failed to fetch analytics data',
     details: errorMessage,
     stack: errorStack
    },
    { status: 500 }
   )
  }
}
```

Api\auth\login

```
import { NextResponse } from "next/server"
import { getDb } from "@/lib/db"
import { startOfDay, subDays } from 'date-fns'

export async function GET() {
 console.log('Analytics overview endpoint hit')
 try {
  const db = await getDb()
  console.log('Database connection successful')
  const now = new Date()
  const thirtyDaysAgo = subDays(now, 30)
  console.log('Fetching data from', thirtyDaysAgo, 'to', now)

  // Get sales data
  const salesData = await db.collection('orders').aggregate([
   { $match: { createdAt: { $gte: thirtyDaysAgo } } },
   {
    $group: {
     _id: { $dateToString: { format: "%Y-%m-%d", date: "$createdAt" } },
     total: { $sum: "$totalAmount" },
     count: { $sum: 1 }
    }
   },
```

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

```
  { $sort: { _id: 1 } }
]).toArray()

// Get quote conversion data
const quotesData = await db.collection('quotes').aggregate([
  { $match: { createdAt: { $gte: thirtyDaysAgo } } },
  {
   $group: {
    _id: "$status",
    count: { $sum: 1 }
   }
  }
]).toArray()

// Get customer data
const customersData = await db.collection('users').aggregate([
  { $match: { role: 'customer' } },
  { $count: 'total' }
]).toArray()

// Get support tickets data
const ticketsData = await db.collection('tickets').aggregate([
  {
   $group: {
    _id: "$status",
    count: { $sum: 1 }
   }
  }
]).toArray()

// For testing - return sample data
console.log('Using sample data for testing')
const sampleData = [
  {
   _id: '1',
   name: 'Sample Product 1',
   price: 100,
   quoteCount: 5,
   totalQuotedValue: 500
  },
  {
   _id: '2',
   name: 'Sample Product 2',
   price: 200,
```

```
    quoteCount: 3,
    totalQuotedValue: 600
  }
]

console.log('Returning sample data response')
const responseData = {
  sales: {
    daily: [],
    total: 0,
    orderCount: 0
  },
  customers: {
    total: 0,
    newThisMonth: 0
  },
  quotes: {
    byStatus: {},
    total: 0
  },
  tickets: {
    byStatus: {},
    total: 0
  },
  products: {
    mostQuoted: sampleData
  }
}

console.log('Returning sample data:', JSON.stringify(responseData, null, 2))
return NextResponse.json(responseData)

// Uncomment below to restore database query
/*
// Database diagnostics
console.log('=== DATABASE DIAGNOSTICS ===')
console.log('Database name:', db.databaseName)

// List all collections
const collections = await db.listCollections().toArray()
console.log('Available collections:', collections.map(c => c.name))

// Check if quotes collection exists
const quotesCollection = db.collection('quotes')
```

```
const quotesCount = await quotesCollection.estimatedDocumentCount()
console.log(`Found ${quotesCount} documents in 'quotes' collection`)

let mostQuotedProducts = [];

if (quotesCount > 0) {
  // Get a sample quote to check structure
  const sampleQuote = await quotesCollection.findOne({})
  console.log('Sample quote structure:', JSON.stringify({
    _id: sampleQuote?._id,
    items: sampleQuote?.items?.map((i: any) => ({
      productId: i.productId,
      quantity: i.quantity,
      price: i.price
    }))
  }, null, 2))

  console.log('Running aggregation pipeline...')
  mostQuotedProducts = await quotesCollection.aggregate([
    // Unwind the items array to process each product in each quote
    { $unwind: "$items" },
    {
      $group: {
        _id: "$items.productId",
        quoteCount: { $sum: 1 },
        totalQuotedValue: { $sum: { $multiply: ["$items.quantity", "$items.price"] } }
      }
    },
    { $sort: { quoteCount: -1 } },
    { $limit: 5 },
    {
      $lookup: {
        from: "products",
        localField: "_id",
        foreignField: "_id",
        as: "product"
      }
    },
    { $unwind: "$product" },
    {
      $project: {
        _id: "$product._id",
        name: "$product.name",
        price: "$product.price",
```

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

```
        quoteCount: 1,
        totalQuotedValue: 1
      }
    }
  ]).toArray()

  console.log('Most quoted products from database:', {
    count: mostQuotedProducts?.length || 0,
    sample: mostQuotedProducts?.slice(0, 2) // Show first 2 items if available
  })
} else {
  console.log('No quotes found in the database')
}
*/

return NextResponse.json({
  sales: {
    daily: salesData.map(item => ({
      date: item._id,
      total: item.total,
      count: item.count
    })),
    total: salesData.reduce((sum, item) => sum + (item.total || 0), 0),
    orderCount: salesData.reduce((sum, item) => sum + (item.count || 0), 0)
  },
  quotes: {
    byStatus: quotesData.reduce((acc, item) => ({
      ...acc,
      [item._id]: item.count
    }), {}),
    total: quotesData.reduce((sum, item) => sum + (item.count || 0), 0)
  },
  customers: {
    total: customersData[0]?.total || 0,
    newThisMonth: 0 // Would need registration date in user schema
  },
  tickets: {
    byStatus: ticketsData.reduce((acc, item) => ({
      ...acc,
      [item._id]: item.count
    }), {}),
    total: ticketsData.reduce((sum, item) => sum + (item.count || 0), 0)
  },
  products: {
```

```
        mostQuoted: [] // Using empty array as fallback
      }
    })
  } catch (error) {
    console.error('Error in analytics overview:', error)
    const errorMessage = error instanceof Error ? error.message : 'Unknown error
occurred'
    const errorStack = error instanceof Error && process.env.NODE_ENV ===
'development' ? error.stack : undefined

    return NextResponse.json(
      {
        error: 'Failed to fetch analytics data',
        details: errorMessage,
        stack: errorStack
      },
      { status: 500 }
    )
  }
}
```

Api\auth\logout

```
import { NextResponse } from "next/server"
import { NextRequest } from "next/server"

export async function POST(request: NextRequest) {
  try {
    const response = NextResponse.json({ success: true })

    // Clear auth cookies
    response.cookies.set({
      name: 'auth-token',
      value: '',
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'strict',
      maxAge: 0, // Expire immediately
      path: '/',
    })

    response.cookies.set({
      name: 'user-role',
      value: '',
      httpOnly: true,
```

```
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'strict',
      maxAge: 0, // Expire immediately
      path: '/',
    })

    return response
  } catch (error) {
    console.error('Logout error:', error)
    return NextResponse.json(
      { error: 'Failed to log out' },
      { status: 500 }
    )
  }
}
```

Api\auth\register

```
import { NextResponse } from "next/server"
import { z } from "zod"
import bcrypt from "bcryptjs"
import clientPromise from "@/lib/db"

const RegisterSchema = z.object({
  email: z.string().email(),
  password: z.string().min(6),
  name: z.string().min(1),
  role: z.enum(["admin", "customer"]).optional().default("customer"),
})

export async function POST(request: Request) {
  try {
    const body = await request.json()
    const { email, password, name, role } = RegisterSchema.parse(body)

    const client = await clientPromise!
    const db = client.db(process.env.MONGODB_DB || "unicom")
    const users = db.collection("users")
    const existing = await users.findOne({ email })
    if (existing) {
      return NextResponse.json({ error: "Email already in use" }, { status: 409 })
    }

    const passwordHash = await bcrypt.hash(password, 10)
    const id = crypto.randomUUID()
```

```
      await users.insertOne({ id, email, name, password_hash: passwordHash, role,
created_at: new Date() })

      const user = { id, email, name, role }
      return NextResponse.json({ user }, { status: 201 })
    } catch (error: unknown) {
      if (error instanceof z.ZodError) {
        return NextResponse.json({ error: error.flatten() }, { status: 400 })
      }
      return NextResponse.json({ error: "Internal Server Error" }, { status: 500 })
    }
}
```

Api\chat

```
import { GoogleGenerativeAI, SchemaType } from "@google/generative-ai"
import { searchProducts, getProductById } from "@/lib/products"
import { quotes } from "@/lib/quotes"
import { supportTickets } from "@/lib/tickets"
import { NextResponse } from "next/server"

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY || "")

const searchProductsTool = {
  name: "searchProducts",
  description: "Search for products by keyword",
  parameters: {
    type: SchemaType.OBJECT,
    properties: {
      query: {
        type: SchemaType.STRING,
        description: "Search term",
      },
    },
    required: ["query"],
  },
}

const getProductDetailsTool = {
  name: "getProductDetails",
  description: "Get detailed product specs",
  parameters: {
    type: SchemaType.OBJECT,
    properties: {
      productId: {
```

```
            type: SchemaType.STRING,
            description: "Product ID",
          },
        },
        required: ["productId"],
    },
}

const getMyQuotesTool = {
    name: "getMyQuotes",
    description: "Get user quotes",
    parameters: {
        type: SchemaType.OBJECT,
        properties: {},
        required: [],
    },
}

const getMyTicketsTool = {
    name: "getMyTickets",
    description: "Get user support tickets",
    parameters: {
        type: SchemaType.OBJECT,
        properties: {},
        required: [],
    },
}

export async function POST(req: Request) {
    try {
        const { message, history, user } = await req.json()

        if (!process.env.GEMINI_API_KEY) {
            return NextResponse.json({ error: "API Key not configured" }, { status:
500 })
        }

        let activeTools: any[] = [searchProductsTool, getProductDetailsTool]

        if (user?.email) {
            activeTools = [searchProductsTool, getProductDetailsTool,
getMyQuotesTool, getMyTicketsTool]
        }
```

```
    const model = genAI.getGenerativeModel({
      model: "gemini-2.0-flash",
      systemInstruction: "You are a helpful assistant for Unicom. Always answer
questions clearly.",
      tools: [{ functionDeclarations: activeTools }] as any,
    })

    let validHistory = history.map((msg: any) => (

      {
        role: msg.isBot ? "model" : "user",
        parts: [{ text: msg.text }],
      }))

    if (validHistory.length > 0 && validHistory[0].role === "model") {
      validHistory = validHistory.slice(1)
    }

    const chat = model.startChat({ history: validHistory })

    const result = await chat.sendMessage(message)
    const response = result.response
    const calls = response.functionCalls?.() || null

    let finalText = ""
    let productsFound: any[] = []

    if (calls && calls.length > 0) {
      const call = calls[0]
      const functionResponseParts = []

      if (call.name === "searchProducts") {
        productsFound = await searchProducts((call.args as any).query)
        functionResponseParts.push({
          functionResponse: {
            name: "searchProducts",
            response: { products: productsFound },
          },
        })
      } else if (call.name === "getProductDetails") {
        const product = await getProductById((call.args as any).productId)
        if (product) productsFound = [product]
        functionResponseParts.push({
          functionResponse: {
```

```
                name: "getProductDetails",
                response: { product: product || {} },
              },
            })
          } else if (call.name === "getMyQuotes") {
            const userQuotes = user?.email ? quotes.filter(q => q.customerEmail ===
user.email) : []
            functionResponseParts.push({
              functionResponse: {
                name: "getMyQuotes",
                response: { quotes: userQuotes },
              },
            })
          } else if (call.name === "getMyTickets") {
            const userTickets = user?.email ? supportTickets.filter(t => t.customerEmail
=== user.email) : []
            functionResponseParts.push({
              functionResponse: {
                name: "getMyTickets",
                response: { tickets: userTickets },
              },
            })
          }

          if (functionResponseParts.length > 0) {
            const finalResult = await chat.sendMessage(functionResponseParts)
            finalText = finalResult.response.text?.() || "Here's what I found."
          }
        } else {
          finalText = response.text?.() || "How can I help you today?"
        }

        return NextResponse.json({
          text: finalText,
          products: productsFound.slice(0, 3)
        })

      } catch (error) {
        console.error("Chat API Error:", error)
        return NextResponse.json(
          { text: "I'm here to help! Ask me anything about our products." },
          { status: 200 }
        )
      }
```

```
}
```

Api\products

```
import { NextResponse } from "next/server"
import clientPromise from "@/lib/db"
import { z } from "zod"
import { ObjectId } from "mongodb"

const ProductSchema = z.object({
  name: z.string().min(1),
  description: z.string().optional().default(""),
  price: z.number().nonnegative(),
  category: z.string().min(1),
  brand: z.string().min(1),
  image: z.string().optional().default(""),
  inStock: z.boolean().default(true),
  specifications: z.record(z.string()).optional(),
  rating: z.number().min(0).max(5).optional().default(0),
  reviews: z.number().int().nonnegative().optional().default(0),
})

export async function GET(request: Request) {
  try {
    const { searchParams } = new URL(request.url)
    const q = searchParams.get("q")?.toLowerCase() || ""
    const category = searchParams.get("category") || undefined
    const idsParam = searchParams.get("ids") || undefined

    const client = await clientPromise!
    const db = client.db(process.env.MONGODB_DB || "unicom")
    const col = db.collection("products")

    const filter: any = {}
    if (category) filter.category = category
    if (q) {
      filter.$or = [
        { name: { $regex: q, $options: "i" } },
        { description: { $regex: q, $options: "i" } },
        { brand: { $regex: q, $options: "i" } },
      ]
    }
    if (idsParam) {
      const ids = idsParam.split(",").map((s) => s.trim()).filter(Boolean)
      const objectIds = ids
        .filter((x) => ObjectId.isValid(x))
```

```
      .map((x) => new ObjectId(x))
    filter.$or = [
      ...(filter.$or || []),
      { id: { $in: ids } },
      ...(objectIds.length ? [{ _id: { $in: objectIds } }] : []),
    ]
  }

  const docs = await col.find(filter).sort({ _id: -1 }).toArray()

  return NextResponse.json({ products: docs.map((d) => ({
    id: d.id ?? String(d._id),
    name: d.name,
    description: d.description ?? "",
    price: d.price ?? 0,
    category: d.category,
    image: d.image ?? "",
    inStock: d.inStock ?? true,
    specifications: d.specifications ?? {},
    brand: d.brand ?? "",
    rating: d.rating ?? 0,
    reviews: d.reviews ?? 0,
  })) })
 } catch (e) {
  return NextResponse.json({ error: "Internal Server Error" }, { status: 500 })
 }
}

export async function POST(request: Request) {
 try {
  const json = await request.json()
  const parsed = ProductSchema.parse(json)

  const client = await clientPromise!
  const db = client.db(process.env.MONGODB_DB || "unicom")
  const col = db.collection("products")

  const now = new Date()
  const toInsert = {
    ...parsed,
    createdAt: now,
    updatedAt: now,
  }
```

```
    const result = await col.insertOne(toInsert)
    const id = String(result.insertedId)
    await col.updateOne({ _id: result.insertedId }, { $set: { id } })

    return NextResponse.json({ product: { id, ...parsed } })
  } catch (error: any) {
    if (error?.issues) {
      return NextResponse.json({ error }, { status: 400 })
    }
    return NextResponse.json({ error: "Internal Server Error" }, { status: 500 })
  }
}
```

Api\products\[id]

```
import { NextResponse } from "next/server"
import clientPromise from "@/lib/db"
import { ObjectId } from "mongodb"

export async function GET(_req: Request, { params }: { params: { id: string } }) {
  try {
    const client = await clientPromise!
    const db = client.db(process.env.MONGODB_DB || "unicom")
    const col = db.collection("products")
    const id = params.id
    const doc = ObjectId.isValid(id)
      ? await col.findOne({ $or: [{ id }, { _id: new ObjectId(id) }] })
      : await col.findOne({ id })
    if (!doc) return NextResponse.json({ error: "Not found" }, { status: 404 })
    return NextResponse.json({
      product: {
        id: doc.id ?? String(doc._id),
        name: doc.name,
        description: doc.description ?? "",
        price: doc.price ?? 0,
        category: doc.category,
        image: doc.image ?? "",
        inStock: doc.inStock ?? true,
        specifications: doc.specifications ?? {},
        brand: doc.brand ?? "",
        rating: doc.rating ?? 0,
        reviews: doc.reviews ?? 0,
      },
    })
```

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

```
  } catch {
    return NextResponse.json({ error: "Internal Server Error" }, { status: 500 })
  }
}

export async function PUT(request: Request, { params }: { params: { id: string } }) {
  try {
    const body = await request.json()
    const client = await clientPromise!
    const db = client.db(process.env.MONGODB_DB || "unicom")
    const col = db.collection("products")
    const id = params.id
    const filter = ObjectId.isValid(id) ? { $or: [{ id }, { _id: new ObjectId(id) }] } :
{ id }
    const update: any = { ...body, updatedAt: new Date() }
    delete update._id
    const res = await col.updateOne(filter, { $set: update })
    if (res.matchedCount === 0) return NextResponse.json({ error: "Not found" },
{ status: 404 })
    return NextResponse.json({ ok: true })
  } catch {
    return NextResponse.json({ error: "Internal Server Error" }, { status: 500 })
  }
}

export async function DELETE(_request: Request, { params }: { params: { id:
string } }) {
  try {
    const client = await clientPromise!
    const db = client.db(process.env.MONGODB_DB || "unicom")
    const col = db.collection("products")
    const id = params.id
    const filter = ObjectId.isValid(id) ? { $or: [{ id }, { _id: new ObjectId(id) }] } :
{ id }
    const res = await col.deleteOne(filter)
    if (res.deletedCount === 0) return NextResponse.json({ error: "Not found" },
{ status: 404 })
    return NextResponse.json({ ok: true })
  } catch {
    return NextResponse.json({ error: "Internal Server Error" }, { status: 500 })
  }
}
```

Api\quotes

```
import { NextResponse } from "next/server"
import { z } from "zod"
import clientPromise from "@/lib/db"
import type { Quote, QuoteItem } from "@/lib/quotes"

const QuoteItemSchema = z.object({
  productId: z.string(),
  productName: z.string(),
  quantity: z.number().min(1),
  unitPrice: z.number().min(0),
  customSpecs: z.string().optional(),
})

const CreateQuoteSchema = z.object({
  customerName: z.string().min(1),
  customerEmail: z.string().email(),
  company: z.string().optional(),
  phone: z.string().optional(),
  items: z.array(QuoteItemSchema).min(1),
  notes: z.string().optional(),
})

export async function POST(request: Request) {
  try {
    const body = await request.json()
    const { customerName, customerEmail, company, phone, items, notes } =
CreateQuoteSchema.parse(body)

    const client = await clientPromise!
    const db = client.db(process.env.MONGODB_DB || "unicom")
    const quotes = db.collection("quotes")

    // Ensure quotes collection exists and create indexes
    await quotes.createIndex({ customerEmail: 1 })
    await quotes.createIndex({ status: 1 })
    await quotes.createIndex({ createdAt: -1 })

    const totalAmount = items.reduce((sum, item) => sum + item.quantity *
item.unitPrice, 0)
    const id = crypto.randomUUID()
    const now = new Date()
    const expiresAt = new Date(now.getTime() + 30 * 24 * 60 * 60 * 1000) // 30
days
```

```
        const quote: Quote = {
            id,
            customerName,
            customerEmail,
            company,
            phone,
            items,
            totalAmount,
            status: "pending",
            notes,
            createdAt: now,
            expiresAt,
        }

        await quotes.insertOne(quote)

        return NextResponse.json({ quote }, { status: 201 })
    } catch (error: unknown) {
        if (error instanceof z.ZodError) {
            return NextResponse.json({ error: error.flatten() }, { status: 400 })
        }
        return NextResponse.json({ error: "Internal Server Error" }, { status: 500 })
    }
}

export async function GET(request: Request) {
    try {
        const { searchParams } = new URL(request.url)
        const customerEmail = searchParams.get("customerEmail")
        const status = searchParams.get("status")

        const client = await clientPromise!
        const db = client.db(process.env.MONGODB_DB || "unicom")
        const quotes = db.collection("quotes")

        let filter: any = {}
        if (customerEmail) {
            filter.customerEmail = customerEmail
        }
        if (status) {
            filter.status = status
        }

        const quotesList = await quotes.find(filter).sort({ createdAt: -1 }).toArray()
```

```
    return NextResponse.json({ quotes: quotesList })
  } catch (error: unknown) {
    return NextResponse.json({ error: "Internal Server Error" }, { status: 500 })
  }
}
```

Api\quotes\[id]

```
import { NextResponse } from "next/server"
import clientPromise from "@/lib/db"

export async function GET(request: Request, { params }: { params: { id: string } }) {
  try {
    const client = await clientPromise!
    const db = client.db(process.env.MONGODB_DB || "unicom")
    const quotes = db.collection("quotes")

    const quote = await quotes.findOne({ id: params.id })
    if (!quote) {
      return NextResponse.json({ error: "Quote not found" }, { status: 404 })
    }

    return NextResponse.json({ quote })
  } catch (error: unknown) {
    return NextResponse.json({ error: "Internal Server Error" }, { status: 500 })
  }
}

export async function DELETE(request: Request, { params }: { params: { id:
string } }) {
  try {
    const client = await clientPromise!
    const db = client.db(process.env.MONGODB_DB || "unicom")
    const quotes = db.collection("quotes")

    const result = await quotes.deleteOne({ id: params.id })
    if (result.deletedCount === 0) {
      return NextResponse.json({ error: "Quote not found" }, { status: 404 })
    }

    return NextResponse.json({ success: true })
  } catch (error: unknown) {
    return NextResponse.json({ error: "Internal Server Error" }, { status: 500 })
  }
}
```

```
export async function PATCH(request: Request, { params }: { params: { id:
string } }) {
  try {
    const body = await request.json()
    const { status, adminNotes } = body

    const client = await clientPromise!
    const db = client.db(process.env.MONGODB_DB || "unicom")
    const quotes = db.collection("quotes")

    const updateData: any = {}
    if (status) updateData.status = status
    if (adminNotes !== undefined) updateData.adminNotes = adminNotes

    const result = await quotes.updateOne({ id: params.id }, { $set: updateData })
    if (result.matchedCount === 0) {
      return NextResponse.json({ error: "Quote not found" }, { status: 404 })
    }

    return NextResponse.json({ success: true })
  } catch (error: unknown) {
    return NextResponse.json({ error: "Internal Server Error" }, { status: 500 })
  }
}
```

Api\test-db

```
import { NextRequest, NextResponse } from "next/server"
import { getTicketsCollection } from "@/lib/db"

export async function GET(request: NextRequest) {
 try {
  console.log("=== TESTING MONGODB CONNECTION ===")

  const tickets = await getTicketsCollection()
  console.log("Got tickets collection successfully")

  // Try to count documents
  const count = await tickets.countDocuments()
  console.log("Document count:", count)

  return NextResponse.json({
   success: true,
   message: "MongoDB connection successful",
```

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

```
      documentCount: count
    })
  } catch (error) {
    console.error("=== MONGODB CONNECTION ERROR ===")
    console.error("Error:", error)

    return NextResponse.json({
      error: "MongoDB connection failed",
      details: error instanceof Error ? error.message : "Unknown error"
    }, { status: 500 })
  }
}
```

Api\Tickets

```
import { NextRequest, NextResponse } from "next/server"
import { getTicketsCollection } from "@/lib/db"
import { z } from "zod"
import { randomUUID } from "crypto"

const createTicketSchema = z.object({
  customerName: z.string().min(1),
  customerEmail: z.string().email(),
  subject: z.string().min(1),
  issueType: z.enum(["technical", "billing", "general", "product", "other"]),
  description: z.string().min(5, "Description must be at least 5 characters"),
  priority: z.enum(["low", "medium", "high", "urgent"]).default("medium"),
})

export async function POST(request: NextRequest) {
  try {
    console.log("=== TICKET CREATION START ===")
    const body = await request.json()
    console.log("Request body:", body)

    const validatedData = createTicketSchema.parse(body)
    console.log("Validated data:", validatedData)

    const tickets = await getTicketsCollection()
    console.log("Got tickets collection")

    const ticket = {
      ...validatedData,
      id: randomUUID(),
      status: "open" as const,
      createdAt: new Date(),
```

```
      updatedAt: new Date(),
    }
    console.log("Created ticket object:", ticket)

    const result = await tickets.insertOne(ticket)
    console.log("Insert result:", result)

    console.log("=== TICKET CREATION SUCCESS ===")
    return NextResponse.json({ success: true, ticket, insertId: result.insertedId })
  } catch (error) {
    console.error("=== TICKET CREATION ERROR ===")
    console.error("Error type:", typeof error)
    console.error("Error message:", error instanceof Error ? error.message : "Unknown
error")
    console.error("Error stack:", error instanceof Error ? error.stack : "No stack")
    console.error("Full error:", error)

    return NextResponse.json({
      error: "Failed to create ticket",
      details: error instanceof Error ? error.message : "Unknown error",
      type: typeof error
    }, { status: 500 })
  }
}

export async function GET(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url)
    const customerEmail = searchParams.get("customerEmail")
    const status = searchParams.get("status")

    const tickets = await getTicketsCollection()
    let query = {}

    if (customerEmail) {
      query = { customerEmail }
    }
    if (status) {
      query = { ...query, status }
    }

    const ticketList = await tickets.find(query).sort({ createdAt: -1 }).toArray()

    return NextResponse.json({ tickets: ticketList })
```

```
  } catch (error) {
    console.error("Error fetching tickets:", error)
    return NextResponse.json({ error: "Failed to fetch tickets" }, { status: 500 })
  }
}
```

Api\tickets\[id]

```
import { NextRequest, NextResponse } from "next/server"
import { getTicketsCollection } from "@/lib/db"
import { z } from "zod"

const updateTicketSchema = z.object({
  status: z.enum(["open", "in_progress", "resolved", "closed"]).optional(),
  adminNotes: z.string().optional(),
  assignedTo: z.string().optional(),
})

export async function PUT(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  try {
    const body = await request.json()
    const validatedData = updateTicketSchema.parse(body)

    const tickets = await getTicketsCollection()
    const updateData = {
      ...validatedData,
      updatedAt: new Date(),
    }

    if (validatedData.status === "resolved") {
      updateData.resolvedAt = new Date()
    }
    if (validatedData.status === "closed") {
      updateData.closedAt = new Date()
    }

    const result = await tickets.updateOne(
      { id: params.id },
      { $set: updateData }
    )

    if (result.matchedCount === 0) {
```

```
      return NextResponse.json({ error: "Ticket not found" }, { status: 404 })
    }

    return NextResponse.json({ success: true })
  } catch (error) {
    console.error("Error updating ticket:", error)
    return NextResponse.json({ error: "Failed to update ticket" }, { status: 500 })
  }
}

export async function GET(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  try {
    const tickets = await getTicketsCollection()
    const ticket = await tickets.findOne({ id: params.id })

    if (!ticket) {
      return NextResponse.json({ error: "Ticket not found" }, { status: 404 })
    }

    return NextResponse.json({ ticket })
  } catch (error) {
    console.error("Error fetching ticket:", error)
    return NextResponse.json({ error: "Failed to fetch ticket" }, { status: 500 })
  }
}
```

Api\users\admins

```
import { NextResponse } from "next/server"
import { getDb } from "@/lib/db"

export async function GET() {
  try {
    const db = await getDb()
    const users = await db.collection('users')
      .find({ role: 'admin' })
      .project({ _id: 0, id: 1, name: 1, email: 1 })
      .sort({ name: 1 })
      .toArray()

    return NextResponse.json({ users })
  } catch (error) {
```

```
    console.error('Error fetching admin users:', error)
    return NextResponse.json(
      { error: 'Failed to fetch admin users' },
      { status: 500 }
    )
  }
}
```

App\catalog

```
"use client"

import { useState, useEffect } from "react"
import { VisitorHeader } from "@/components/visitor-header"
import { Navigation } from "@/components/navigation"
import { useAuth } from "@/lib/auth"
import { ProductCard } from "@/components/product-card"
import { Button } from "@/components/ui/button"
import { Input } from "@/components/ui/input"
import { Badge } from "@/components/ui/badge"
import { Card, CardContent } from "@/components/ui/card"
import { Checkbox } from "@/components/ui/checkbox"
import { Search, Grid, List, GitCompare, X } from "lucide-react"
import { categories, fetchProducts, type Product } from "@/lib/products"
import Link from "next/link"

export default function CatalogPage() {
  const { user } = useAuth()
  const [searchQuery, setSearchQuery] = useState("")
  const [selectedCategory, setSelectedCategory] = useState<string | null>(null)
  const [viewMode, setViewMode] = useState<"grid" | "list">("grid")
  const [compareProducts, setCompareProducts] = useState<string[]>([])
  const [items, setItems] = useState<Product[]>([])
  const [loading, setLoading] = useState(false)

  const load = async () => {
    setLoading(true)
    try {
      const list = await fetchProducts({ q: searchQuery || undefined, category:
selectedCategory || undefined })
      setItems(list)
    } finally {
      setLoading(false)
    }
  }
```

```
  // load when query or category changes
  useEffect(() => {
    void load()
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [searchQuery, selectedCategory])

  const handleCompareToggle = (productId: string) => {
    setCompareProducts((prev) => {
      if (prev.includes(productId)) {
        return prev.filter((id) => id !== productId)
      } else if (prev.length < 3) {
        return [...prev, productId]
      }
      return prev
    })
  }

  const getComparedProducts = () => items.filter((p) =>
compareProducts.includes(p.id))

  return (
    <div className="min-h-screen bg-background">
      {user ? <Navigation centered /> : <VisitorHeader />}

      <main className="container mx-auto py-8 px-4">
        {user?.role === 'admin' && (
          <div className="mb-6 bg-yellow-50 border-l-4 border-yellow-400 p-4">
            <div className="flex">
              <div className="flex-shrink-0">
                <svg className="h-5 w-5 text-yellow-400" viewBox="0 0 20 20"
fill="currentColor">
                  <path fillRule="evenodd" d="M8.257 3.099c.765-1.36 2.722-1.36 3.486
0l5.58 9.92c.75 1.334-.213 2.98-1.742 2.98H4.42c-1.53 0-2.493-1.646-1.743-
2.98l5.58-9.92zM11 13a1 1 0 11-2 0 1 1 0 012 0zm-1-8a1 1 0 00-1 1v3a1 1 0 002
0V6a1 1 0 00-1-1z" clipRule="evenodd" />
                </svg>
              </div>
              <div className="ml-3">
                <p className="text-sm text-yellow-700">
                  You are in <span className="font-medium">View Only</span> mode as
an administrator. Some actions are restricted.
                </p>
              </div>
```

```
        </div>
       </div>
      )}
      {/* Header */}
      <div className="mb-8">
       <h1 className="text-4xl font-bold text-balance mb-4">Product Catalog</h1>
       <p className="text-lg text-muted-foreground">Discover our wide range of
electronics and IT solutions</p>
      </div>

      {/* Search and Filters */}
      <div className="mb-8 space-y-4">
       <div className="flex flex-col md:flex-row gap-4">
        <div className="relative flex-1">
         <Search className="absolute left-3 top-1/2 transform -translate-y-1/2 text-
muted-foreground h-4 w-4" />
         <Input
          placeholder="Search products..."
          value={searchQuery}
          onChange={(e) => setSearchQuery(e.target.value)}
          className="pl-10"
         />
        </div>
        <div className="flex items-center space-x-2">
         <Button
          variant={viewMode === "grid" ? "default" : "outline"}
          size="icon"
          onClick={() => setViewMode("grid")}
         >
          <Grid className="h-4 w-4" />
         </Button>
         <Button
          variant={viewMode === "list" ? "default" : "outline"}
          size="icon"
          onClick={() => setViewMode("list")}
         >
          <List className="h-4 w-4" />
         </Button>
        </div>
       </div>

       {/* Categories */}
       <div className="flex flex-wrap gap-2">
        <Button
```

```
            variant={selectedCategory === null ? "default" : "outline"}
            onClick={() => setSelectedCategory(null)}
            className="h-auto py-2"
          >
            All Products
          </Button>
          {categories.map((category) => (
            <Button
              key={category.id}
              variant={selectedCategory === category.id ? "default" : "outline"}
              onClick={() => setSelectedCategory(category.id)}
              className="h-auto py-2"
            >
              <span className="mr-2">{category.icon}</span>
              {category.name}
            </Button>
          ))}
        </div>
      </div>

      {compareProducts.length > 0 && (
        <Card className="mb-6 border-emerald-200 bg-emerald-50">
          <CardContent className="p-4">
            <div className="flex items-center justify-between">
              <div className="flex items-center space-x-4">
                <GitCompare className="h-5 w-5 text-emerald-600" />
                <span className="font-medium">Compare Products
({compareProducts.length}/3)</span>
                <div className="flex space-x-2">
                  {getComparedProducts().map((product) => (
                    <Badge key={product.id} variant="secondary" className="flex items-
center space-x-1">
                      <span>{product.name}</span>
                      <Button
                        variant="ghost"
                        size="icon"
                        className="h-4 w-4 p-0"
                        onClick={() => handleCompareToggle(product.id)}
                      >
                        <X className="h-3 w-3" />
                      </Button>
                    </Badge>
                  ))}
                </div>
```

```
            </div>
            <div className="flex space-x-2">
              <Button size="sm" disabled={compareProducts.length < 2}
asChild={compareProducts.length >= 2}>
                {compareProducts.length >= 2 ? (
                 <Link
href={`/compare?products=${compareProducts.join(",")}`}>Compare Now</Link>
                ) : (
                 "Select 2+ products"
                )}
              </Button>
              <Button variant="outline" size="sm" onClick={() =>
setCompareProducts([])}>
                Clear All
              </Button>
            </div>
          </div>
        </CardContent>
      </Card>
    )}

    {/* Results */}
    <div className="mb-4">
      <p className="text-muted-foreground">
        {loading ? "Loading products…" : `Showing ${items.length} products`}
        {selectedCategory && (
          <Badge variant="secondary" className="ml-2">
            {categories.find((c) => c.id === selectedCategory)?.name}
          </Badge>
        )}
        {searchQuery && (
          <Badge variant="secondary" className="ml-2">
            Search: "{searchQuery}"
          </Badge>
        )}
      </p>
    </div>

    {/* Products Grid */}
    {items.length > 0 ? (
      <div
        className={`grid gap-6 ${
          viewMode === "grid" ? "grid-cols-1 md:grid-cols-2 lg:grid-cols-3 xl:grid-
cols-4" : "grid-cols-1"
```

```
          }`}
        >
          {items.map((product) => (
            <div key={product.id} className="relative">
              <div className="absolute top-2 left-2 z-10">
                <div className="flex items-center space-x-2 bg-white/90 backdrop-blur-
sm rounded-md p-2 shadow-sm">
                  <Checkbox
                    id={`compare-${product.id}`}
                    checked={compareProducts.includes(product.id)}
                    onCheckedChange={() => handleCompareToggle(product.id)}
                    disabled={!compareProducts.includes(product.id) &&
compareProducts.length >= 3}
                  />
                  <label htmlFor={`compare-${product.id}`} className="text-xs font-
medium cursor-pointer">
                    Compare
                  </label>
                </div>
              </div>
              <ProductCard product={product} />
            </div>
          ))}
        </div>
      ) : (
        <Card className="text-center py-12">
          <CardContent>
            <div className="space-y-4">
              <div className="text-6xl">🔍</div>
              <h3 className="text-xl font-semibold">No products found</h3>
              <p className="text-muted-foreground">Try adjusting your search or filter
criteria</p>
              <Button
                onClick={() => {
                  setSearchQuery("")
                  setSelectedCategory(null)
                }}
              >
                Clear Filters
              </Button>
            </div>
          </CardContent>
        </Card>
```

```
      )}
    </main>
  </div>
 )
```

Catalog\[id]

```
"use client"

import Image from "next/image"
import { useEffect, useState } from "react"
import { Navigation } from "@/components/navigation"
import { Button } from "@/components/ui/button"
import { Badge } from "@/components/ui/badge"
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card"
import { Separator } from "@/components/ui/separator"
import { Star, Share2, Quote, ArrowLeft } from "lucide-react"
import { toast } from "sonner"
import { getProductById, type Product } from "@/lib/products"
import Link from "next/link"
import { useAuth } from "@/lib/auth"
import { useRouter } from "next/navigation"

interface ProductPageProps {
 params: {
  id: string
 }
}

export default function ProductPage({ params }: ProductPageProps) {
 const { user } = useAuth()
 const [product, setProduct] = useState<Product | null>(null)
 const [loading, setLoading] = useState(true)
 const router = useRouter()

 const handleShare = async () => {
  try {
   await navigator.share({
    title: product?.name,
    text: product?.description,
    url: window.location.href,
   })
  } catch (err) {
   // Fallback for browsers that don't support Web Share API
   await navigator.clipboard.writeText(window.location.href)
   toast.success('Link copied to clipboard')
```

```
  }
}

const handleRequestQuote = () => {
  if (user?.role === 'admin') {
    toast.info('Admin mode: View only')
  } else {
    router.push(`/quote?productId=${product?.id}`)
  }
}

useEffect(() => {
  let cancelled = false
  const load = async () => {
    try {
      const p = await getProductById(params.id)
      if (!cancelled) setProduct(p ?? null)
    } finally {
      if (!cancelled) setLoading(false)
    }
  }
  void load()
  return () => {
    cancelled = true
  }
}, [params.id])

return (
  <div className="min-h-screen bg-background">
    {user ? <Navigation centered /> : <Navigation />}

    <main className="container mx-auto py-8 px-4">
      {/* Breadcrumb */}
      <div className="mb-6">
        <Button variant="ghost" asChild className="mb-4">
          <Link href="/catalog">
            <ArrowLeft className="mr-2 h-4 w-4" />
            Back to Catalog
          </Link>
        </Button>
      </div>

      {loading ? (
        <div className="py-12 text-center">Loading product…</div>
```

```
      ) : !product ? (
        <Card className="text-center py-12">
          <CardContent>
            <h2 className="text-2xl font-bold mb-2">Product not found</h2>
            <p className="text-muted-foreground">The product you are looking for
does not exist.</p>
          </CardContent>
        </Card>
      ) : (
        <div className="grid gap-8 lg:grid-cols-2">
          {/* Product Image */}
          <div className="space-y-4">
            <div className="relative aspect-square overflow-hidden rounded-lg
border">
              <Image
                src={product.image || "/placeholder.svg"}
                alt={product.name}
                fill
                className="object-cover"
                priority
              />
              {!product.inStock && (
                <Badge variant="destructive" className="absolute top-4 right-4">
                  Out of Stock
                </Badge>
              )}
            </div>
          </div>

          {/* Product Details */}
          <div className="space-y-6">
            <div>
              <div className="flex items-center justify-between mb-2">
                <span className="text-sm text-muted-
foreground">{product.brand}</span>
                <div className="flex items-center space-x-1">
                  <Star className="h-4 w-4 fill-yellow-400 text-yellow-400" />
                  <span className="font-medium">{product.rating}</span>
                  <span className="text-muted-foreground">({product.reviews}
reviews)</span>
                </div>
              </div>
```

```
            <h1 className="text-3xl font-bold text-balance mb-
4">{product.name}</h1>

            <p className="text-lg text-muted-foreground mb-
6">{product.description}</p>

            <div className="flex items-center space-x-4 mb-6">
              <span className="text-4xl font-bold text-emerald-
600">₱{product.price.toLocaleString('en-PH', { minimumFractionDigits: 2,
maximumFractionDigits: 2 })}</span>
              <Badge variant={product.inStock ? "default" : "destructive"}>
                {product.inStock ? "In Stock" : "Out of Stock"}
              </Badge>
            </div>
          </div>

          {/* Actions */}
          <div className="flex gap-4">
            <Button
              size="lg"
              className="flex-1"
              onClick={handleRequestQuote}
            >
              <Quote className="mr-2 h-5 w-5" />
              Request Quote
            </Button>
            <Button
              variant="outline"
              size="icon"
              className="h-11 w-11"
              onClick={handleShare}
            >
              <Share2 className="h-5 w-5" />
            </Button>
          </div>

          {/* Quick Quote */}
          <Card>
            <CardHeader>
              <CardTitle className="text-lg">Need a Custom Quote?</CardTitle>
            </CardHeader>
            <CardContent>
              <p className="text-sm text-muted-foreground mb-4">
                Get personalized pricing for bulk orders or custom configurations
```

```
              </p>
              <Button variant="outline" className="w-full bg-transparent">
                Request Quote
              </Button>
            </CardContent>
          </Card>
        </div>
      </div>
    )}

    {/* Specifications */}
    {!loading && product?.specifications &&
Object.keys(product.specifications).length > 0 && (
      <div className="mt-12">
        <Card>
          <CardHeader>
            <CardTitle>Specifications</CardTitle>
          </CardHeader>
          <CardContent>
            <div className="grid gap-4 md:grid-cols-2">
              {Object.entries(product.specifications).map(([key, value], index) => (
                <div key={key}>
                  <div className="flex justify-between py-2">
                    <span className="font-medium">{key}</span>
                    <span className="text-muted-foreground">{value}</span>
                  </div>
                  {index < Object.entries(product.specifications!).length - 1 &&
<Separator />}
                </div>
              ))}
            </div>
          </CardContent>
        </Card>
      </div>
    )}
  </main>
 </div>
 )
}
```

Compare

```
"use client"

import { Suspense, useEffect, useState } from "react"
```

```
import { Navigation } from "@/components/navigation"
import { Button } from "@/components/ui/button"
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card"
import { Badge } from "@/components/ui/badge"
import { ArrowLeft, X } from "lucide-react"
import { fetchProductsByIds, type Product } from "@/lib/products"
import Link from "next/link"
import { useSearchParams } from "next/navigation"

function CompareContent() {
  const searchParams = useSearchParams()
  const productIds = searchParams.get("products")?.split(",") || []
  const [items, setItems] = useState<Product[]>([])
  const [loading, setLoading] = useState(false)

  useEffect(() => {
    let cancelled = false
    const load = async () => {
      setLoading(true)
      try {
        const list = await fetchProductsByIds(productIds)
        if (!cancelled) setItems(list)
      } finally {
        if (!cancelled) setLoading(false)
      }
    }
    void load()
    return () => {
      cancelled = true
    }
  }, [productIds.join(",")])

  if (loading) {
    return <div className="text-center py-12">Loading comparison…</div>
  }

  const compareProducts = items

  if (compareProducts.length < 2) {
    return (
      <div className="text-center py-12">
        <h2 className="text-2xl font-bold mb-4">No products to compare</h2>
        <p className="text-muted-foreground mb-6">Please select at least 2 products
from the catalog</p>
```

```
      <Button asChild>
        <Link href="/catalog">Browse Catalog</Link>
      </Button>
    </div>
  )
 }

 const allSpecs = Array.from(new Set(compareProducts.flatMap((p) =>
Object.keys(p.specifications || {}))))

 return (
  <div>
   {/* Header */}
   <div className="mb-8 flex items-center justify-between">
    <div>
     <Button variant="ghost" asChild className="mb-4">
      <Link href="/catalog">
       <ArrowLeft className="h-4 w-4 mr-2" />
       Back to Catalog
      </Link>
     </Button>
     <h1 className="text-4xl font-bold text-balance">Product Comparison</h1>
     <p className="text-lg text-muted-foreground">Compare
{compareProducts.length} products side by side</p>
    </div>
   </div>

   {/* Comparison Table */}
   <div className="overflow-x-auto">
    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6 min-w-
full">
     {compareProducts.map((product) => (
      <Card key={product.id} className="min-w-[300px]">
       <CardHeader className="text-center">
        <div className="aspect-square bg-muted rounded-lg mb-4 overflow-
hidden">
         <img
          src={product.image || "/placeholder.svg"}
          alt={product.name}
          className="w-full h-full object-cover"
         />
        </div>
        <CardTitle className="text-lg">{product.name}</CardTitle>
        <div className="flex items-center justify-center space-x-2">
```

```
            <span className="text-2xl font-bold text-emerald-
600">₱{product.price.toLocaleString('en-PH', { minimumFractionDigits: 2,
maximumFractionDigits: 2 })}</span>
            <Badge variant={product.inStock ? "default" : "secondary"}>
              {product.inStock ? "In Stock" : "Out of Stock"}
            </Badge>
          </div>
        </CardHeader>
        <CardContent className="space-y-4">
          <div>
            <h4 className="font-semibold mb-2">Description</h4>
            <p className="text-sm text-muted-
foreground">{product.description}</p>
          </div>

          <div>
            <h4 className="font-semibold mb-2">Category</h4>
            <Badge variant="outline">{product.category}</Badge>
          </div>

          <div>
            <h4 className="font-semibold mb-2">Specifications</h4>
            <div className="space-y-2">
              {allSpecs.map((spec) => (
                <div key={spec} className="flex justify-between text-sm">
                  <span className="text-muted-foreground">{spec}:</span>
                  <span className="font-medium">
                    {product.specifications?.[spec] || <X className="h-4 w-4 text-
muted-foreground" />}
                  </span>
                </div>
              ))}
            </div>
          </div>

          <Button className="w-full" asChild>
            <Link href={`/catalog/${product.id}`}>View Details</Link>
          </Button>
        </CardContent>
      </Card>
    ))}
  </div>
 </div>
</div>
```

```
  )
}

export default function ComparePage() {
  return (
    <div className="min-h-screen bg-background">
      <Navigation centered />
      <main className="container mx-auto py-8 px-4">
        <Suspense fallback={<div>Loading comparison...</div>}>
          <CompareContent />
        </Suspense>
      </main>
    </div>
  )
}
```

```
}

export default function Loading() {
 return null
}

export default function Loading() {
 return null
}
```

Customer\home

```
"use client"

import { useEffect, useState } from "react"
import { Navigation } from "@/components/navigation"
import { ProductCard } from "@/components/product-card"
import { Button } from "@/components/ui/button"
import { Input } from "@/components/ui/input"
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card"
import { Search, TrendingUp, Star, Clock } from "lucide-react"
import { fetchProducts, categories, type Product } from "@/lib/products"
import Link from "next/link"
import { useAuth } from "@/lib/auth"

export default function CustomerHomePage() {
 const [searchQuery, setSearchQuery] = useState("")
 const { logout } = useAuth()
 const [items, setItems] = useState<Product[]>([])

 const handleLogout = () => {
  logout()
  window.location.href = "/"
 }

 useEffect(() => {
  const load = async () => {
   try {
    const list = await fetchProducts()
    setItems(list)
   } catch (e) {
    // ignore for now
   }
  }
```

```
  void load()
 }, [])

 // Get featured products (first 6)
 const featuredProducts = items.slice(0, 6)

 // Get trending categories
 const trendingCategories = categories.slice(0, 4)

 return (
  <div className="min-h-screen bg-background">
   <Navigation centered />

   <main className="container mx-auto py-8 px-4">
    {/* Welcome Section */}
    <div className="mb-8 text-center">
     <h1 className="text-4xl font-bold text-balance mb-4">Welcome to Your
Electronics Hub</h1>
     <p className="text-lg text-muted-foreground mb-6">Discover the latest in
technology and IT solutions</p>

     {/* Quick Search */}
     <div className="max-w-md mx-auto relative">
      <Search className="absolute left-3 top-1/2 transform -translate-y-1/2 text-
muted-foreground h-4 w-4" />
      <Input
       placeholder="Search products..."
       value={searchQuery}
       onChange={(e) => setSearchQuery(e.target.value)}
       className="pl-10"
       onKeyDown={(e) => {
        if (e.key === "Enter" && searchQuery) {
         window.location.href =
`/catalog?search=${encodeURIComponent(searchQuery)}`
        }
       }}
      />
     </div>
    </div>

    {/* Quick Actions */}
    <div className="grid grid-cols-1 md:grid-cols-3 gap-6 mb-8">
     <Card className="hover:shadow-lg transition-shadow">
      <CardHeader className="text-center">
```

```
            <TrendingUp className="h-8 w-8 mx-auto text-emerald-600 mb-2" />
            <CardTitle>Browse Catalog</CardTitle>
          </CardHeader>
          <CardContent className="text-center">
            <p className="text-muted-foreground mb-4">Explore our full range of
products</p>
            <Button asChild className="w-full">
              <Link href="/catalog">View Catalog</Link>
            </Button>
          </CardContent>
        </Card>

        <Card className="hover:shadow-lg transition-shadow">
          <CardHeader className="text-center">
            <Star className="h-8 w-8 mx-auto text-emerald-600 mb-2" />
            <CardTitle>Get Quote</CardTitle>
          </CardHeader>
          <CardContent className="text-center">
            <p className="text-muted-foreground mb-4">Request custom pricing for
your needs</p>
            <Button asChild className="w-full">
              <Link href="/quote">Request Quote</Link>
            </Button>
          </CardContent>
        </Card>

        <Card className="hover:shadow-lg transition-shadow">
          <CardHeader className="text-center">
            <Clock className="h-8 w-8 mx-auto text-emerald-600 mb-2" />
            <CardTitle>My Quotes</CardTitle>
          </CardHeader>
          <CardContent className="text-center">
            <p className="text-muted-foreground mb-4">Track your quote
requests</p>
            <Button asChild className="w-full">
              <Link href="/quotes">View Quotes</Link>
            </Button>
          </CardContent>
        </Card>
      </div>

      {/* Featured Products */}
      <div className="mb-8">
        <div className="flex items-center justify-between mb-6">
```

```
            <h2 className="text-2xl font-bold">Featured Products</h2>
            <Button variant="outline" asChild>
              <Link href="/catalog">View All</Link>
            </Button>
          </div>
          <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
            {featuredProducts.map((product) => (
              <ProductCard key={product.id} product={product} />
            ))}
          </div>
        </div>

        {/* Categories */}
        <div>
          <h2 className="text-2xl font-bold mb-6">Shop by Category</h2>
          <div className="grid grid-cols-2 md:grid-cols-4 gap-4">
            {trendingCategories.map((category) => (
              <Card key={category.id} className="hover:shadow-lg transition-shadow
cursor-pointer">
                <Link href={`/catalog?category=${category.id}`}>
                  <CardContent className="p-6 text-center">
                    <div className="text-3xl mb-2">{category.icon}</div>
                    <h3 className="font-semibold">{category.name}</h3>
                  </CardContent>
                </Link>
              </Card>
            ))}
          </div>
        </div>

      </main>
    </div>
  )
}

export default function Loading() {
  return null
}
```

Customer\quotes

```
"use client"

import { useAuth } from "@/lib/auth"
```

```
import { useRouter } from "next/navigation"
import { useEffect, useState } from "react"
import { Navigation } from "@/components/navigation"
import { Button } from "@/components/ui/button"
import { Input } from "@/components/ui/input"
import { Badge } from "@/components/ui/badge"
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card"
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from
"@/components/ui/table"
import { Search, Eye, CheckCircle, XCircle, Clock, FileText, Plus } from "lucide-
react"
import type { Quote } from "@/lib/quotes"
import Link from "next/link"
import { QuotationDialog } from "@/components/quotation-dialog"

export default function CustomerQuotesPage() {
  const { user, loading } = useAuth()
  const router = useRouter()
  const [searchQuery, setSearchQuery] = useState("")
  const [quotes, setQuotes] = useState<Quote[]>([])
  const [quotesLoading, setQuotesLoading] = useState(true)
  const [selectedQuote, setSelectedQuote] = useState<Quote | null>(null)
  const [isDialogOpen, setIsDialogOpen] = useState(false)

  useEffect(() => {
    if (!loading && (!user || user.role !== "customer")) {
      router.push("/login")
    }
  }, [user, loading, router])

  useEffect(() => {
    if (user && user.role === "customer") {
      fetchQuotes()
    }
  }, [user])

  const fetchQuotes = async () => {
    try {
      console.log("Fetching quotes for customer:", user?.email)
      const response = await
fetch(`/api/quotes?customerEmail=${encodeURIComponent(user?.email || "")}`)
      if (response.ok) {
        const data = await response.json()
        console.log("Customer quotes response:", data.quotes)
```

```
      setQuotes(data.quotes)
    } else {
      console.error("Failed to fetch customer quotes:", response.status)
    }
  } catch (error) {
    console.error("Error fetching quotes:", error)
  } finally {
    setQuotesLoading(false)
  }
}

const customerQuotes = quotes

const filteredQuotes = customerQuotes.filter(
  (quote) =>
    quote.id.toLowerCase().includes(searchQuery.toLowerCase()) ||
    quote.company?.toLowerCase().includes(searchQuery.toLowerCase()),
)

const getStatusIcon = (status: Quote["status"]) => {
  switch (status) {
    case "pending":
      return <Clock className="h-4 w-4" />
    case "approved":
      return <CheckCircle className="h-4 w-4" />
    case "rejected":
      return <XCircle className="h-4 w-4" />
    case "expired":
      return <FileText className="h-4 w-4" />
  }
}

const getStatusVariant = (status: Quote["status"]) => {
  switch (status) {
    case "pending":
      return "secondary"
    case "approved":
      return "default"
    case "rejected":
      return "destructive"
    case "expired":
      return "outline"
    default:
      return "secondary"
```

```
  }
 }

 if (loading || quotesLoading) {
  return (
   <div className="min-h-screen flex items-center justify-center">
    <div className="animate-spin rounded-full h-32 w-32 border-b-2 border-
emerald-600"></div>
   </div>
  )
 }

 if (!user || user.role !== "customer") {
  return null
 }

 return (
  <div className="min-h-screen bg-background">
   <Navigation centered />

   <main className="container mx-auto py-8 px-4">
    <div className="mb-8">
     <div className="flex items-center justify-between mb-4">
      <div>
       <h1 className="text-3xl font-bold text-balance">My Quotes</h1>
       <p className="text-muted-foreground mt-2">View and track your quote
requests</p>
      </div>
      <Button asChild>
       <Link href="/quote">
        <Plus className="h-4 w-4 mr-2" />
        Request New Quote
       </Link>
      </Button>
     </div>
     <Badge variant="secondary">
      Customer
     </Badge>
    </div>

    {/* Stats */}
    <div className="grid gap-4 md:grid-cols-4 mb-8">
     <Card>
```

```
          <CardHeader className="flex flex-row items-center justify-between space-y-
0 pb-2">
            <CardTitle className="text-sm font-medium">Total Quotes</CardTitle>
            <FileText className="h-4 w-4 text-muted-foreground" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold">{customerQuotes.length}</div>
          </CardContent>
        </Card>

        <Card>
          <CardHeader className="flex flex-row items-center justify-between space-y-
0 pb-2">
            <CardTitle className="text-sm font-medium">Pending</CardTitle>
            <Clock className="h-4 w-4 text-amber-600" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold text-amber-600">
              {customerQuotes.filter((q) => q.status === "pending").length}
            </div>
          </CardContent>
        </Card>

        <Card>
          <CardHeader className="flex flex-row items-center justify-between space-y-
0 pb-2">
            <CardTitle className="text-sm font-medium">Approved</CardTitle>
            <CheckCircle className="h-4 w-4 text-emerald-600" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold text-emerald-600">
              {customerQuotes.filter((q) => q.status === "approved").length}
            </div>
          </CardContent>
        </Card>

        <Card>
          <CardHeader className="flex flex-row items-center justify-between space-y-
0 pb-2">
            <CardTitle className="text-sm font-medium">Total Value</CardTitle>
            <FileText className="h-4 w-4 text-muted-foreground" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold">
```

```
            ₱{customerQuotes.reduce((sum, q) => sum + q.totalAmount,
0).toLocaleString('en-PH', { minimumFractionDigits: 2, maximumFractionDigits:
2 })}
           </div>
          </CardContent>
         </Card>
        </div>

        {/* Search */}
        <div className="mb-6">
         <div className="relative max-w-md">
          <Search className="absolute left-3 top-1/2 transform -translate-y-1/2 text-
muted-foreground h-4 w-4" />
          <Input
           placeholder="Search your quotes..."
           value={searchQuery}
           onChange={(e) => setSearchQuery(e.target.value)}
           className="pl-10"
          />
         </div>
        </div>

        {/* Quotes Table or Empty State */}
        <Card>
         <CardHeader>
          <CardTitle>Your Quote Requests</CardTitle>
         </CardHeader>
         <CardContent>
          {filteredQuotes.length === 0 ? (
           <div className="text-center py-12">
            <FileText className="h-12 w-12 text-muted-foreground mx-auto mb-4" />
            <h3 className="text-lg font-semibold mb-2">No quotes found</h3>
            <p className="text-muted-foreground mb-4">
             {customerQuotes.length === 0
              ? "You haven't submitted any quote requests yet."
              : "No quotes match your search criteria."}
            </p>
            <Button asChild>
             <Link href="/quote">
              <Plus className="h-4 w-4 mr-2" />
              Request Your First Quote
             </Link>
            </Button>
           </div>
```

```
        ) : (
          <Table>
            <TableHeader>
              <TableRow>
                <TableHead>Quote ID</TableHead>
                <TableHead>Company</TableHead>
                <TableHead>Items</TableHead>
                <TableHead>Total Amount</TableHead>
                <TableHead>Status</TableHead>
                <TableHead>Created</TableHead>
                <TableHead>Actions</TableHead>
              </TableRow>
            </TableHeader>
            <TableBody>
              {filteredQuotes.map((quote) => (
                <TableRow key={quote.id}>
                  <TableCell className="font-medium">{quote.id}</TableCell>
                  <TableCell>{quote.company || "-"}</TableCell>
                  <TableCell>{quote.items.length} items</TableCell>
                  <TableCell className="font-
medium">₱{quote.totalAmount.toLocaleString('en-PH', { minimumFractionDigits: 2,
maximumFractionDigits: 2 })}</TableCell>
                  <TableCell>
                    <Badge variant={getStatusVariant(quote.status)}
className="capitalize">
                      {getStatusIcon(quote.status)}
                      <span className="ml-1">{quote.status}</span>
                    </Badge>
                  </TableCell>
                  <TableCell>{new
Date(quote.createdAt).toLocaleDateString()}</TableCell>
                  <TableCell>
                    <Button
                      size="sm"
                      variant="outline"
                      onClick={() => {
                        setSelectedQuote(quote)
                        setIsDialogOpen(true)
                      }}
                    >
                      <Eye className="h-4 w-4 mr-1" />
                      View
                    </Button>
                  </TableCell>
```

```
                </TableRow>
              ))}
            </TableBody>
          </Table>
        )}
      </CardContent>
    </Card>
  </main>

  {/* Quote Details Dialog */}
  {selectedQuote && (
   <QuotationDialog
    open={isDialogOpen}
    onOpenChange={setIsDialogOpen}
    quote={selectedQuote}
    viewOnly={true}
    onConfirm={() => {
     setIsDialogOpen(false)
    }}
    onCancel={() => {
     setIsDialogOpen(false)
    }}
   />
  )}
 </div>
 )
}

export default function Loading() {
 return null
}
```

Home\support

```
"use client"

import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
"@/components/ui/card"
import { Button } from "@/components/ui/button"
import { Input } from "@/components/ui/input"
import { Textarea } from "@/components/ui/textarea"
import { Label } from "@/components/ui/label"
import { Phone, Mail, MapPin, Clock, MessageCircle, FileText } from "lucide-react"
import { Navigation } from "@/components/navigation"
import { useAuth } from "@/lib/auth"
```

```
export default function CustomerSupportPage() {
  const { user } = useAuth()

  return (
    <div className="min-h-screen bg-background">
      <Navigation centered />
      <main className="container mx-auto px-4 py-8">
      <div className="text-center mb-12">
        <h1 className="text-4xl font-bold mb-4 text-balance">Customer Support</h1>
        <p className="text-xl text-muted-foreground max-w-2xl mx-auto text-pretty">
          Get help when you need it. Our expert support team is here to assist you with
all your technical needs.
        </p>
      </div>

      <div className="grid grid-cols-1 lg:grid-cols-2 gap-8 mb-12">
        {/* Contact Information */}
        <div className="space-y-6">
          <Card>
            <CardHeader>
              <CardTitle className="flex items-center gap-2">
                <Phone className="h-5 w-5 text-emerald-600" />
                Contact Information
              </CardTitle>
            </CardHeader>
            <CardContent className="space-y-4">
              <div className="flex items-center gap-3">
                <Phone className="h-4 w-4 text-muted-foreground" />
                <div>
                  <p className="font-medium">Phone Support</p>
                  <p className="text-sm text-muted-foreground">(555) 123-TECH</p>
                </div>
              </div>
              <div className="flex items-center gap-3">
                <Mail className="h-4 w-4 text-muted-foreground" />
                <div>
                  <p className="font-medium">Email Support</p>
                  <p className="text-sm text-muted-
foreground">support@unicomtech.com</p>
                </div>
              </div>
              <div className="flex items-center gap-3">
                <MapPin className="h-4 w-4 text-muted-foreground" />
```

```
            <div>
              <p className="font-medium">Visit Our Store</p>
              <p className="text-sm text-muted-foreground">123 Tech Street, Digital
City, DC 12345</p>
            </div>
          </div>
          <div className="flex items-center gap-3">
            <Clock className="h-4 w-4 text-muted-foreground" />
            <div>
              <p className="font-medium">Business Hours</p>
              <p className="text-sm text-muted-foreground">Mon-Fri: 9AM-6PM,
Sat: 10AM-4PM</p>
            </div>
          </div>
        </CardContent>
      </Card>
    </div>

    {/* Support Form */}
    <Card>
      <CardHeader>
        <CardTitle>Submit Support Request</CardTitle>
        <CardDescription>Describe your issue and we'll get back to you as soon as
possible</CardDescription>
      </CardHeader>
      <CardContent className="space-y-4">
        <div className="grid grid-cols-2 gap-4">
          <div className="space-y-2">
            <Label htmlFor="firstName">First Name</Label>
            <Input id="firstName" placeholder={user?.name?.split(' ')[0] || "John"} />
          </div>
          <div className="space-y-2">
            <Label htmlFor="lastName">Last Name</Label>
            <Input id="lastName" placeholder={user?.name?.split(' ')[1] || "Doe"} />
          </div>
        </div>
        <div className="space-y-2">
          <Label htmlFor="email">Email</Label>
          <Input id="email" type="email" placeholder={user?.email ||
"john@example.com"} />
        </div>
        <div className="space-y-2">
          <Label htmlFor="phone">Phone (Optional)</Label>
          <Input id="phone" type="tel" placeholder="(555) 123-4567" />
```

```
          </div>
          <div className="space-y-2">
            <Label htmlFor="subject">Subject</Label>
            <Input id="subject" placeholder="Brief description of your issue" />
          </div>
          <div className="space-y-2">
            <Label htmlFor="message">Message</Label>
            <Textarea id="message" placeholder="Please describe your issue in detail..."
className="min-h-[120px]" />
          </div>
          <Button className="w-full">Submit Support Request</Button>
        </CardContent>
      </Card>
    </div>

    {/* FAQ Section */}
    <Card>
      <CardHeader>
        <CardTitle>Frequently Asked Questions</CardTitle>
        <CardDescription>Quick answers to common questions</CardDescription>
      </CardHeader>
      <CardContent className="space-y-4">
        <div className="space-y-2">
          <h4 className="font-medium">What are your support hours?</h4>
          <p className="text-sm text-muted-foreground">
            Our support team is available Monday-Friday 9AM-6PM and Saturday
10AM-4PM. Emergency support is available
            24/7 for critical issues.
          </p>
        </div>
        <div className="space-y-2">
          <h4 className="font-medium">Do you offer remote support?</h4>
          <p className="text-sm text-muted-foreground">
            Yes, we provide secure remote support for software issues, troubleshooting,
and system maintenance when
            possible.
          </p>
        </div>
        <div className="space-y-2">
          <h4 className="font-medium">What's included in your warranty?</h4>
          <p className="text-sm text-muted-foreground">
            All products come with manufacturer warranty plus our 30-day satisfaction
guarantee. Labor warranties vary
            by service type.
```

```
            </p>
          </div>
        </CardContent>
      </Card>
      </main>
    </div>
  )
}
```

Home\tickets

```
"use client"

import { useEffect, useState } from "react"
import { useAuth } from "@/lib/auth"
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card"
import { Badge } from "@/components/ui/badge"
import { Button } from "@/components/ui/button"
import { Navigation } from "@/components/navigation"
import { Plus, MessageSquare } from "lucide-react"
import Link from "next/link"
import type { SupportTicket } from "@/lib/tickets"

export default function CustomerTicketsPage() {
  const { user } = useAuth()
  const [tickets, setTickets] = useState<SupportTicket[]>([])
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    if (user) {
      fetchTickets()
    }
  }, [user])

  const fetchTickets = async () => {
    try {
      const response = await fetch(`/api/tickets?customerEmail=${user?.email}`)
      if (response.ok) {
        const data = await response.json()
        setTickets(data.tickets)
      }
    } catch (error) {
      console.error("Failed to fetch tickets:", error)
    } finally {
      setLoading(false)
```

```
  }
 }

 const getStatusColor = (status: string) => {
  switch (status) {
   case "open": return "bg-blue-100 text-blue-800"
   case "in_progress": return "bg-yellow-100 text-yellow-800"
   case "resolved": return "bg-green-100 text-green-800"
   case "closed": return "bg-gray-100 text-gray-800"
   default: return "bg-gray-100 text-gray-800"
  }
 }

 const getPriorityColor = (priority: string) => {
  switch (priority) {
   case "urgent": return "bg-red-100 text-red-800"
   case "high": return "bg-orange-100 text-orange-800"
   case "medium": return "bg-yellow-100 text-yellow-800"
   case "low": return "bg-green-100 text-green-800"
   default: return "bg-gray-100 text-gray-800"
  }
 }

 if (!user) {
  return (
   <div className="min-h-screen bg-gray-50">
    <Navigation centered />
    <div className="container mx-auto py-8">
     <Card className="max-w-2xl mx-auto">
      <CardContent className="text-center py-8">
       <p className="text-gray-500">Please log in to view your support
tickets.</p>
      </CardContent>
     </Card>
    </div>
   </div>
  )
 }

 return (
  <div className="min-h-screen bg-gray-50">
   <Navigation centered />

   <div className="container mx-auto py-8">
```

```
      <div className="flex justify-between items-center mb-6">
        <div>
          <h1 className="text-3xl font-bold text-gray-900">My Support Tickets</h1>
          <p className="text-gray-600 mt-2">Track and manage your support
requests</p>
        </div>
        <Link href="/support-ticket">
          <Button className="flex items-center gap-2">
            <Plus className="h-4 w-4" />
            New Ticket
          </Button>
        </Link>
      </div>

      {loading ? (
        <div className="text-center py-8">Loading tickets...</div>
      ) : tickets.length === 0 ? (
        <Card>
          <CardContent className="text-center py-8">
            <MessageSquare className="h-12 w-12 text-gray-400 mx-auto mb-4" />
            <p className="text-gray-500 mb-4">No support tickets found.</p>
            <Link href="/support-ticket">
              <Button>Create Your First Ticket</Button>
            </Link>
          </CardContent>
        </Card>
      ) : (
        <div className="space-y-4">
          {tickets.map((ticket) => (
            <Card key={ticket.id}>
              <CardHeader>
                <div className="flex justify-between items-start">
                  <div>
                    <CardTitle className="text-lg">{ticket.subject}</CardTitle>
                    <p className="text-sm text-gray-600 mt-1">
                      Created: {new Date(ticket.createdAt).toLocaleDateString()}
                    </p>
                  </div>
                  <div className="flex gap-2">
                    <Badge className={getStatusColor(ticket.status)}>
                      {ticket.status.replace("_", " ")}
                    </Badge>
                    <Badge className={getPriorityColor(ticket.priority)}>
                      {ticket.priority}
```

```
                </Badge>
              </div>
            </div>
          </CardHeader>
          <CardContent>
            <p className="text-gray-700 mb-2">{ticket.description}</p>
            <div className="flex justify-between items-center text-sm text-gray-
500">
              <span>Type: {ticket.issueType}</span>
              <span>Updated: {new
Date(ticket.updatedAt).toLocaleDateString()}</span>
            </div>
            {ticket.adminNotes && (
              <div className="mt-3 p-3 bg-blue-50 rounded-lg">
                <p className="text-sm font-medium text-blue-900">Admin
Response:</p>
                <p className="text-sm text-blue-800 mt-1">{ticket.adminNotes}</p>
              </div>
            )}
          </CardContent>
        </Card>
      ))}
    </div>
  )}
 </div>
 </div>
 )
}
```

App\dashboard

```
"use client"

import { useAuth } from "@/lib/auth"
import { useRouter } from "next/navigation"
import { useEffect, useState } from "react"
import Link from "next/link"
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
"@/components/ui/card"
import { Button } from "@/components/ui/button"
import { Badge } from "@/components/ui/badge"
import { Navigation } from "@/components/navigation"
import { ShoppingCart, Users, TrendingUp, Package, Plus, FileText, Activity } from
"lucide-react"
```

```
import { Overview } from "@/components/overview"
import { RecentSales } from "@/components/recent-sales"
import { TopProducts } from "@/components/top-products"
import { RecentActivity } from "@/components/recent-activity"

export default function DashboardPage() {
  const { user, loading } = useAuth()
  const router = useRouter()

  useEffect(() => {
    if (!loading && !user) {
      router.push("/login")
    }
  }, [user, loading, router])

  if (loading) {
    return (
      <div className="min-h-screen flex items-center justify-center">
        <div className="animate-spin rounded-full h-32 w-32 border-b-2 border-
emerald-600"></div>
      </div>
    )
  }

  if (!user) {
    return null
  }

  const isAdmin = user.role === "admin"

  return (
    <div className="min-h-screen bg-background">
      <Navigation centered />

      <main className="container mx-auto py-8 px-4">
        <div className="mb-8">
          <h1 className="text-3xl font-bold text-balance">Welcome back,
{user.name}!</h1>
          <p className="text-muted-foreground mt-2">
            {isAdmin ? "Manage your store and track performance" : "Track your orders
and manage your account"}
          </p>
          <Badge variant={isAdmin ? "default" : "secondary"} className="mt-2">
            {isAdmin ? "Administrator" : "Customer"}
```

```
        </Badge>
       </div>

       {user?.role === 'admin' ? <AdminDashboard /> : <CustomerDashboard />}
     </main>
    </div>
  )
}

function AdminDashboard() {
  const [data, setData] = useState<any>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  const fetchData = async () => {
    try {
      setLoading(true);
      const response = await fetch('/api/analytics/overview');
      if (!response.ok) {
        throw new Error('Failed to fetch dashboard data');
      }
      const result = await response.json();
      setData(result);
    } catch (err) {
      console.error('Error fetching data:', err);
      setError('Failed to load dashboard data');
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchData();
  }, []);

  return (
    <div className="space-y-6">
      <div className="flex items-center justify-between">
        <h1 className="text-3xl font-bold tracking-tight">Admin Dashboard</h1>
        {loading ? (
          <div className="flex items-center justify-center p-8">
            <div className="animate-spin rounded-full h-8 w-8 border-b-2 border-
emerald-600"></div>
          </div>
```

```
      ) : error ? (
        <div className="text-center p-4 text-red-500">{error}</div>
      ) : null}
    </div>

    <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-4">
      <Card>
        <CardHeader className="flex flex-row items-center justify-between space-y-0
pb-2">
          <CardTitle className="text-sm font-medium">Total Revenue</CardTitle>
          <span className="h-4 w-4 text-muted-foreground">
            <TrendingUp className="h-4 w-4" />
          </span>
        </CardHeader>
        <CardContent>
          <div className="text-2xl font-bold">₱{(data?.sales?.total ||
0).toLocaleString('en-PH', { minimumFractionDigits: 2, maximumFractionDigits:
2 })}</div>
          <p className="text-xs text-muted-foreground">+20.1% from last month</p>
        </CardContent>
      </Card>

      <Card>
        <CardHeader className="flex flex-row items-center justify-between space-y-0
pb-2">
          <CardTitle className="text-sm font-medium">Active
Customers</CardTitle>
          <Users className="h-4 w-4 text-muted-foreground" />
        </CardHeader>
        <CardContent>
          <div className="text-2xl font-bold">{data?.customers?.total || 0}</div>
          <p className="text-xs text-muted-foreground">+180.1% from last
month</p>
        </CardContent>
      </Card>

      <Card>
        <CardHeader className="flex flex-row items-center justify-between space-y-0
pb-2">
          <CardTitle className="text-sm font-medium">Quotes</CardTitle>
          <FileText className="h-4 w-4 text-muted-foreground" />
        </CardHeader>
        <CardContent>
          <div className="text-2xl font-bold">{data?.quotes?.total || 0}</div>
```

```
            <p className="text-xs text-muted-foreground">+19% from last month</p>
          </CardContent>
        </Card>

        <Card>
          <CardHeader className="flex flex-row items-center justify-between space-y-0
pb-2">
            <CardTitle className="text-sm font-medium">Active Now</CardTitle>
            <Activity className="h-4 w-4 text-muted-foreground" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold">+573</div>
            <p className="text-xs text-muted-foreground">+201 since last hour</p>
          </CardContent>
        </Card>
      </div>

      <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-7">
        <Card className="col-span-4">
          <CardHeader>
            <CardTitle>Overview</CardTitle>
          </CardHeader>
          <CardContent className="pl-2">
            <Overview data={data?.sales?.daily || []} />
          </CardContent>
        </Card>
        <Card className="col-span-3">
          <CardHeader>
            <CardTitle>Recent Sales</CardTitle>
            <CardDescription>You made 265 sales this month.</CardDescription>
          </CardHeader>
          <CardContent>
            <RecentSales />
          </CardContent>
        </Card>
      </div>

      <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-2">
        <Card>
          <CardHeader>
            <CardTitle>Top Products</CardTitle>
          </CardHeader>
          <CardContent>
            <TopProducts products={data?.products?.mostQuoted || []} />
```

```
        </CardContent>
      </Card>
      <Card>
        <CardHeader>
          <CardTitle>Recent Activity</CardTitle>
        </CardHeader>
        <CardContent>
          <RecentActivity />
        </CardContent>
      </Card>
    </div>
  </div>
 )
}

function QuickActionCard({ title, description, icon, href }: { title: string, description:
string, icon: React.ReactNode, href: string }) {
 return (
  <Link href={href}>
    <Card className="h-full hover:bg-accent/50 transition-colors">
      <CardHeader className="pb-2">
        <div className="flex items-center justify-between">
          <CardTitle className="text-lg">{title}</CardTitle>
          <div className="p-2 rounded-full bg-primary/10">
            {icon}
          </div>
        </div>
        <CardDescription>{description}</CardDescription>
      </CardHeader>
    </Card>
  </Link>
 )
}

function CustomerDashboard() {
 const { user, logout } = useAuth()
 const [quotes, setQuotes] = useState<any[]>([])
 const [quotesLoading, setQuotesLoading] = useState(true)

 useEffect(() => {
  fetchRecentQuotes()
 }, [])

 const fetchRecentQuotes = async () => {
```

```
    try {
      console.log("Fetching quotes for user:", user?.email)
      const response = await
fetch(`/api/quotes?customerEmail=${encodeURIComponent(user?.email || "")}`)
      if (response.ok) {
        const data = await response.json()
        console.log("Quotes response:", data.quotes)
        setQuotes(data.quotes.slice(0, 3)) // Get only recent 3
      } else {
        console.error("Failed to fetch quotes:", response.status)
      }
    } catch (error) {
      console.error("Error fetching quotes:", error)
    } finally {
      setQuotesLoading(false)
    }
  }

  return (
    <div className="space-y-8">
      {/* Recent Quotes */}
      <Card>
        <CardHeader>
          <CardTitle>Recent Quote Requests</CardTitle>
          <CardDescription>Track your recent quote requests</CardDescription>
        </CardHeader>
        <CardContent>
          {quotesLoading ? (
            <div className="text-center py-4">Loading...</div>
          ) : quotes.length === 0 ? (
            <div className="text-center py-8 text-muted-foreground">
              <p>No quote requests yet.</p>
              <Button asChild className="mt-4">
                <Link href="/quote">Request Your First Quote</Link>
              </Button>
            </div>
          ) : (
            <div className="space-y-4">
              {quotes.map((quote) => (
                <div key={quote.id} className="flex items-center justify-between p-4
border rounded-lg">
                  <div>
                    <p className="font-medium">Quote #{quote.id.slice(0, 8)}</p>
                    <p className="text-sm text-muted-foreground">
```

```
                {quote.items.length} items • ₱{quote.totalAmount.toLocaleString('en-
PH', { minimumFractionDigits: 2, maximumFractionDigits: 2 })}
                </p>
              </div>
              <Badge variant={quote.status === "approved" ? "default" : "secondary"}>
                {quote.status}
              </Badge>
            </div>
          ))}
        </div>
      )}
    </CardContent>
  </Card>

  </div>
 )
}
```

App\inventory

```
 "use client"

import { useEffect, useState } from "react"
import { useAuth } from "@/lib/auth"
import { useRouter } from "next/navigation"
import { Navigation } from "@/components/navigation"
import { Button } from "@/components/ui/button"
import { Input } from "@/components/ui/input"
import { Badge } from "@/components/ui/badge"
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card"
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from
"@/components/ui/table"
import { Search, Plus, Edit, Trash2, Package } from "lucide-react"
import { fetchProducts, createProduct, updateProduct, deleteProduct, type Product,
categories } from "@/lib/products"
import { Dialog, DialogContent, DialogFooter, DialogHeader, DialogTitle,
DialogTrigger } from "@/components/ui/dialog"
import { Label } from "@/components/ui/label"
import { Checkbox } from "@/components/ui/checkbox"
import { useToast } from "@/hooks/use-toast"

export default function InventoryPage() {
  const { user, loading } = useAuth()
  const router = useRouter()
  const [searchQuery, setSearchQuery] = useState("")
```

```
  const [items, setItems] = useState<Product[]>([])
  const [creating, setCreating] = useState(false)
  const [open, setOpen] = useState(false)
  const [editingId, setEditingId] = useState<string | null>(null)
  const { toast } = useToast()

  // New product form state
  const [name, setName] = useState("")
  const [brand, setBrand] = useState("")
  const [price, setPrice] = useState<number | "">("")
  const [category, setCategory] = useState("")
  const [image, setImage] = useState("")
  const [description, setDescription] = useState("")
  const [inStock, setInStock] = useState(true)
  const [specRows, setSpecRows] = useState<Array<{ key: string; value:
string }>>([])

  useEffect(() => {
    if (!loading && (!user || user.role !== "admin")) {
      router.push("/login")
    }
  }, [user, loading, router])

  useEffect(() => {
    if (!loading && user?.role === "admin") {
      void load()
    }
  }, [loading, user])

  const load = async () => {
    try {
      const list = await fetchProducts()
      setItems(list)
    } catch (e) {
      toast({ title: "Failed to load products", variant: "destructive" })
    }
  }

  const filteredProducts = items.filter(
    (product) =>
      product.name.toLowerCase().includes(searchQuery.toLowerCase()) ||
      product.brand.toLowerCase().includes(searchQuery.toLowerCase()),
  )
```

```
 if (loading) {
  return (
   <div className="min-h-screen flex items-center justify-center">
    <div className="animate-spin rounded-full h-32 w-32 border-b-2 border-
emerald-600"></div>
   </div>
  )
 }

 if (!user || user.role !== "admin") {
  return null
 }

 return (
  <div className="min-h-screen bg-background">
   <Navigation centered />

   <main className="container mx-auto py-8 px-4">
    <div className="mb-8">
     <div className="flex items-center justify-between">
      <div>
       <h1 className="text-3xl font-bold text-balance">Inventory
Management</h1>
       <p className="text-muted-foreground mt-2">Manage your product
inventory and stock levels</p>
      </div>
      <Button onClick={() => {
       // reset form for create
       setEditingId(null)
       setName("")
       setBrand("")
       setPrice("")
       setCategory("")
       setImage("")
       setDescription("")
       setInStock(true)
       setSpecRows([])
       setOpen(true)
      }}>
       <Plus className="mr-2 h-4 w-4" />
       Add Product
      </Button>
      <Dialog open={open} onOpenChange={setOpen}>
       <DialogContent className="max-w-4xl max-h-[90vh] overflow-y-auto">
```

```
            <DialogHeader>
              <DialogTitle>{editingId ? "Edit Product" : "Add New
Product"}</DialogTitle>
            </DialogHeader>
            <div className="grid grid-cols-1 lg:grid-cols-2 gap-6 py-4">
              {/* Left Column - Basic Info */}
              <div className="space-y-4">
                <div className="space-y-2">
                  <Label htmlFor="name">Product Name *</Label>
                  <Input id="name" value={name} onChange={(e) =>
setName(e.target.value)} placeholder="Enter product name" />
                </div>

                <div className="grid grid-cols-2 gap-4">
                  <div className="space-y-2">
                    <Label htmlFor="brand">Brand *</Label>
                    <Input id="brand" value={brand} onChange={(e) =>
setBrand(e.target.value)} placeholder="Brand name" />
                  </div>
                  <div className="space-y-2">
                    <Label htmlFor="price">Price *</Label>
                    <Input
                      id="price"
                      type="number"
                      step="0.01"
                      value={price}
                      onChange={(e) => setPrice(e.target.value === "" ? "" :
Number(e.target.value))}
                      placeholder="0.00"
                    />
                  </div>
                </div>

                <div className="space-y-2">
                  <Label htmlFor="category">Category *</Label>
                  <select
                    id="category"
                    className="w-full border rounded-md h-9 px-3 bg-background"
                    value={category}
                    onChange={(e) => setCategory(e.target.value)}
                  >
                    <option value="">Select category</option>
                    {categories.map((c) => (
                      <option key={c.id} value={c.id}>{c.name}</option>
```

```
                ))}
              </select>
            </div>

            <div className="space-y-2">
              <Label htmlFor="description">Description</Label>
              <textarea
                id="description"
                className="w-full border rounded-md bg-background p-3 text-sm
resize-none"
                rows={4}
                value={description}
                onChange={(e) => setDescription(e.target.value)}
                placeholder="Product description..."
              />
            </div>

            <div className="flex items-center space-x-2">
              <Checkbox id="inStock" checked={inStock} onCheckedChange={(v)
=> setInStock(Boolean(v))} />
              <Label htmlFor="inStock">In stock</Label>
            </div>
          </div>

          {/* Right Column - Image & Specs */}
          <div className="space-y-4">
            <div className="space-y-2">
              <Label htmlFor="image">Product Image</Label>
              <Input
                id="image"
                type="file"
                accept="image/*"
                onChange={(e) => {
                  const file = e.target.files?.[0]
                  if (!file) return
                  const reader = new FileReader()
                  reader.onload = () => {
                    setImage(String(reader.result || ""))
                  }
                  reader.readAsDataURL(file)
                }}
              />
              {image && (
                <div className="border rounded-md p-3 bg-muted/30">
```

```
                    <p className="text-sm text-muted-foreground mb-2">Preview</p>
                    <div className="flex justify-center">
                     {/* eslint-disable-next-line @next/next/no-img-element */}
                     <img
                      src={image}
                      alt="Preview"
                      className="max-h-32 max-w-32 object-cover rounded border"
                     />
                    </div>
                   </div>
                  )}
                 </div>

                 <div className="space-y-2">
                  <Label>Specifications (Optional)</Label>
                  {specRows.length === 0 && (
                   <p className="text-xs text-muted-foreground">Add technical
specifications as key-value pairs</p>
                  )}
                  <div className="space-y-2 max-h-40 overflow-y-auto">
                   {specRows.map((row, idx) => (
                    <div key={idx} className="flex gap-2">
                     <Input
                      placeholder="Key (e.g., CPU)"
                      value={row.key}
                      onChange={(e) => {
                       const copy = [...specRows]
                       copy[idx].key = e.target.value
                       setSpecRows(copy)
                      }}
                      className="flex-1"
                     />
                     <Input
                      placeholder="Value (e.g., Ryzen 7)"
                      value={row.value}
                      onChange={(e) => {
                       const copy = [...specRows]
                       copy[idx].value = e.target.value
                       setSpecRows(copy)
                      }}
                      className="flex-1"
                     />
                     <Button
                      type="button"
```

```
                    variant="outline"
                    size="sm"
                    onClick={() => setSpecRows(specRows.filter((_, i) => i !== idx))}
                  >
                    ×
                  </Button>
                </div>
              ))}
            </div>
            <Button
              type="button"
              variant="secondary"
              size="sm"
              onClick={() => setSpecRows([...specRows, { key: "", value: "" }])}
            >
              + Add Specification
            </Button>
          </div>
        </div>
      </div>
      <DialogFooter>
        <Button variant="outline" onClick={() =>
setOpen(false)}>Cancel</Button>
        <Button onClick={async () => {
          if (!name || !brand || !category || price === "") {
            toast({ title: "Please fill all required fields", variant: "destructive" })
            return
          }
          try {
            setCreating(true)
            const specifications = Object.fromEntries(specRows.filter(s => s.key
&& s.value).map(s => [s.key, s.value]))
            if (editingId) {
              await updateProduct(editingId, { name, brand, category, price:
Number(price), image, description, inStock, specifications })
              toast({ title: "Product updated" })
            } else {
              await createProduct({ name, brand, category, price: Number(price),
image, description, inStock, specifications, rating: 0, reviews: 0 })
              toast({ title: "Product created" })
            }
            setOpen(false)
            setName(""); setBrand(""); setPrice(""); setCategory(""); setImage("");
setDescription(""); setInStock(true)
```

```
                setSpecRows([])
                await load()
              } catch (e) {
                toast({ title: "Failed to save product", variant: "destructive" })
              } finally {
                setCreating(false)
              }
            }} disabled={creating}>
              {creating ? (editingId ? "Saving..." : "Creating...") : (editingId ? "Save" :
"Create")}
            </Button>
          </DialogFooter>
        </DialogContent>
      </Dialog>
    </div>
  </div>

  {/* Removed demo stats cards */}

  {/* Search */}
  <div className="mb-6">
    <div className="relative max-w-md">
      <Search className="absolute left-3 top-1/2 transform -translate-y-1/2 text-
muted-foreground h-4 w-4" />
      <Input
        placeholder="Search products..."
        value={searchQuery}
        onChange={(e) => setSearchQuery(e.target.value)}
        className="pl-10"
      />
    </div>
  </div>

  {/* Products Table */}
  <Card>
    <CardHeader>
      <CardTitle>Product Inventory</CardTitle>
    </CardHeader>
    <CardContent>
      <Table>
        <TableHeader>
          <TableRow>
            <TableHead>Product</TableHead>
            <TableHead>Brand</TableHead>
```

```
              <TableHead>Category</TableHead>
              <TableHead>Price</TableHead>
              <TableHead>Status</TableHead>
              <TableHead>Actions</TableHead>
            </TableRow>
          </TableHeader>
          <TableBody>
            {filteredProducts.map((product) => (
              <TableRow key={product.id}>
                <TableCell>
                  <div className="flex items-center space-x-3">
                    <div className="w-10 h-10 bg-muted rounded-md flex items-center
justify-center">
                      <Package className="h-5 w-5 text-muted-foreground" />
                    </div>
                    <div>
                      <p className="font-medium">{product.name}</p>
                      <p className="text-sm text-muted-foreground">ID:
{product.id}</p>
                    </div>
                  </div>
                </TableCell>
                <TableCell>{product.brand}</TableCell>
                <TableCell className="capitalize">{product.category}</TableCell>
                <TableCell>₱{product.price.toLocaleString('en-PH',
{ minimumFractionDigits: 2, maximumFractionDigits: 2 })}</TableCell>
                <TableCell>
                  <Badge variant={product.inStock ? "default" : "destructive"}>
                    {product.inStock ? "In Stock" : "Out of Stock"}
                  </Badge>
                </TableCell>
                <TableCell>
                  <div className="flex items-center space-x-2">
                    <Button
                      size="sm"
                      variant="outline"
                      onClick={() => {
                        // Prefill and open dialog for edit
                        setEditingId(product.id)
                        setName(product.name)
                        setBrand(product.brand)
                        setPrice(product.price)
                        setCategory(product.category)
                        setImage(product.image)
```

```
                    setDescription(product.description)
                    setInStock(product.inStock)
                    const specs = product.specifications || {}
                    setSpecRows(Object.keys(specs).map((k) => ({ key: k, value:
String((specs as any)[k]) })))
                    setOpen(true)
                  }}
                >
                  <Edit className="h-4 w-4" />
                </Button>
                <Button
                  size="sm"
                  variant="outline"
                  onClick={async () => {
                    if (!window.confirm(`Delete ${product.name}?`)) return
                    try {
                      await deleteProduct(product.id)
                      toast({ title: "Product deleted" })
                      await load()
                    } catch (e) {
                      toast({ title: "Failed to delete product", variant: "destructive" })
                    }
                  }}
                >
                  <Trash2 className="h-4 w-4" />
                </Button>
              </div>
            </TableCell>
          </TableRow>
        ))}
      </TableBody>
    </Table>
  </CardContent>
</Card>

{/* Removed demo low stock alert */}
</main>
</div>
)
}
```

App\Login

```
import { LoginForm } from "@/components/auth/login-form"
import Link from "next/link"
```

```
export default function LoginPage() {
  return (
    <div className="min-h-screen bg-gradient-to-br from-emerald-50 to-slate-100
flex items-center justify-center p-4">
      <div className="w-full max-w-md space-y-6">
        <div className="text-center">
          <Link href="/" className="inline-block">
            <h1 className="text-3xl font-bold text-emerald-600">Unicom</h1>
            <p className="text-sm text-muted-foreground">Technologies</p>
          </Link>
        </div>

        <LoginForm />

        <p className="text-center text-sm text-muted-foreground">
          Don't have an account?{" "}
          <Link href="/register" className="text-emerald-600 hover:underline font-
medium">
            Sign up
          </Link>
        </p>
      </div>
    </div>
  )
}
```

App\profile

```
"use client"

import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
"@/components/ui/card"
import { Button } from "@/components/ui/button"
import { Input } from "@/components/ui/input"
import { Label } from "@/components/ui/label"
import { Textarea } from "@/components/ui/textarea"
import { useAuth } from "@/lib/auth"
import { Navigation } from "@/components/navigation"
import { User, MapPin, Edit } from "lucide-react"
import { useState } from "react"

export default function ProfilePage() {
  const { user, loading } = useAuth()
  const [isEditing, setIsEditing] = useState(false)
```

```
  if (loading) {
    return (
      <div className="min-h-screen bg-background">
        <Navigation centered />
        <div className="min-h-screen flex items-center justify-center">
          <div className="animate-spin rounded-full h-32 w-32 border-b-2 border-
emerald-600"></div>
        </div>
      </div>
    )
  }

  if (!user) {
    return (
      <div className="min-h-screen bg-background">
        <Navigation centered />
        <div className="container mx-auto px-4 py-8">
          <Card className="max-w-md mx-auto text-center">
            <CardHeader>
              <CardTitle>Access Denied</CardTitle>
              <CardDescription>Please log in to view your profile</CardDescription>
            </CardHeader>
            <CardContent>
              <Button asChild>
                <a href="/login">Sign In</a>
              </Button>
            </CardContent>
          </Card>
        </div>
      </div>
    )
  }

  return (
    <div className="min-h-screen bg-background">
      <Navigation centered />
      <div className="container mx-auto px-4 py-8">
        <div className="max-w-2xl mx-auto">
          <div className="flex items-center justify-between mb-8">
            <h1 className="text-3xl font-bold">Profile Settings</h1>
            <Button variant="outline" onClick={() => setIsEditing(!isEditing)}
className="flex items-center gap-2">
              <Edit className="h-4 w-4" />
```

```
            {isEditing ? "Cancel" : "Edit Profile"}
          </Button>
        </div>

        <div className="space-y-6">
          {/* Personal Information */}
          <Card>
            <CardHeader>
              <CardTitle className="flex items-center gap-2">
                <User className="h-5 w-5 text-emerald-600" />
                Personal Information
              </CardTitle>
            </CardHeader>
            <CardContent className="space-y-4">
              <div className="grid grid-cols-2 gap-4">
                <div className="space-y-2">
                  <Label htmlFor="firstName">First Name</Label>
                  <Input id="firstName" defaultValue={user.name.split(" ")[0]}
disabled={!isEditing} />
                </div>
                <div className="space-y-2">
                  <Label htmlFor="lastName">Last Name</Label>
                  <Input id="lastName" defaultValue={user.name.split(" ")[1] || ""}
disabled={!isEditing} />
                </div>
              </div>
              <div className="space-y-2">
                <Label htmlFor="email">Email</Label>
                <Input id="email" type="email" defaultValue={user.email}
disabled={!isEditing} />
              </div>
              <div className="space-y-2">
                <Label htmlFor="phone">Phone</Label>
                <Input id="phone" type="tel" placeholder="(555) 123-4567"
disabled={!isEditing} />
              </div>
            </CardContent>
          </Card>

          {/* Address Information */}
          <Card>
            <CardHeader>
              <CardTitle className="flex items-center gap-2">
                <MapPin className="h-5 w-5 text-emerald-600" />
```

```
            Address Information
          </CardTitle>
        </CardHeader>
        <CardContent className="space-y-4">
          <div className="space-y-2">
            <Label htmlFor="address">Street Address</Label>
            <Input id="address" placeholder="123 Main Street" disabled={!isEditing}
/>
          </div>
          <div className="grid grid-cols-2 gap-4">
            <div className="space-y-2">
              <Label htmlFor="city">City</Label>
              <Input id="city" placeholder="Digital City" disabled={!isEditing} />
            </div>
            <div className="space-y-2">
              <Label htmlFor="state">State</Label>
              <Input id="state" placeholder="DC" disabled={!isEditing} />
            </div>
          </div>
          <div className="space-y-2">
            <Label htmlFor="zipCode">ZIP Code</Label>
            <Input id="zipCode" placeholder="12345" disabled={!isEditing} />
          </div>
        </CardContent>
      </Card>

      {/* Preferences */}
      <Card>
        <CardHeader>
          <CardTitle>Preferences</CardTitle>
        </CardHeader>
        <CardContent className="space-y-4">
          <div className="space-y-2">
            <Label htmlFor="notifications">Email Notifications</Label>
            <Textarea
              id="notifications"
              placeholder="Choose your notification preferences..."
              disabled={!isEditing}
              className="min-h-[80px]"
            />
          </div>
        </CardContent>
      </Card>
```

```
          {isEditing && (
           <div className="flex gap-4">
            <Button className="flex-1">Save Changes</Button>
            <Button variant="outline" onClick={() => setIsEditing(false)}>
              Cancel
            </Button>
           </div>
          )}
         </div>
        </div>
       </div>
      </div>
    )
}
```

App\quote

```
"use client"

import { useState, useEffect } from "react"
import { useRouter } from "next/navigation"
import { Navigation } from "@/components/navigation"
import { QuoteForm } from "@/components/quote-form"
import { QuotationDialog } from "@/components/quotation-dialog"
import { Card, CardContent } from "@/components/ui/card"
import { CheckCircle, Clock, Calculator, Users } from "lucide-react"
import { useAuth } from "@/lib/auth"
import type { Quote } from "@/lib/quotes"

export default function QuotePage() {
  const { user, loading } = useAuth()
  const router = useRouter()
  const [quoteSubmitted, setQuoteSubmitted] = useState(false)
  const [showQuotationDialog, setShowQuotationDialog] = useState(false)
  const [pendingQuote, setPendingQuote] = useState<Quote | null>(null)

  useEffect(() => {
    if (!loading && !user) {
      router.push("/login")
    }
  }, [user, loading, router])

  const handleQuoteSubmit = (quoteData: Quote) => {
    console.log("Quote submitted:", quoteData)
    setPendingQuote(quoteData)
```

```
  setShowQuotationDialog(true)
}

const handleConfirmQuote = async () => {
  if (!pendingQuote) return

  try {
    const response = await fetch("/api/quotes", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        customerName: pendingQuote.customerName,
        customerEmail: pendingQuote.customerEmail,
        company: pendingQuote.company,
        phone: pendingQuote.phone,
        items: pendingQuote.items,
        notes: pendingQuote.notes,
      }),
    })

    if (!response.ok) {
      throw new Error("Failed to submit quote")
    }

    setShowQuotationDialog(false)
    setQuoteSubmitted(true)
    setPendingQuote(null)

    // Reset after 5 seconds
    setTimeout(() => {
      setQuoteSubmitted(false)
    }, 5000)
  } catch (error) {
    console.error("Error submitting quote:", error)
    // You might want to show an error message to the user
  }
}

const handleCancelQuote = () => {
  setShowQuotationDialog(false)
  setPendingQuote(null)
}

if (loading) {
```

```
  return (
    <div className="min-h-screen flex items-center justify-center">
      <div className="animate-spin rounded-full h-32 w-32 border-b-2 border-
emerald-600"></div>
    </div>
  )
 }

 if (!user) {
  return null
 }

 if (quoteSubmitted) {
  return (
    <div className="min-h-screen bg-background">
      <Navigation centered />

      <main className="container mx-auto py-16 px-4">
        <div className="max-w-2xl mx-auto text-center space-y-6">
          <div className="w-16 h-16 bg-emerald-100 rounded-full flex items-center
justify-center mx-auto">
            <CheckCircle className="w-8 h-8 text-emerald-600" />
          </div>

          <h1 className="text-3xl font-bold text-balance">Quote Request
Submitted!</h1>

          <p className="text-lg text-muted-foreground">
            Thank you for your quote request. Our team will review your requirements
and get back to you within 24
            hours with a detailed proposal.
          </p>

          <div className="bg-emerald-50 border border-emerald-200 rounded-lg p-6">
            <h3 className="font-semibold text-emerald-800 mb-2">What happens
next?</h3>
            <ul className="text-sm text-emerald-700 space-y-1 text-left">
              <li>• Our sales team will review your requirements</li>
              <li>• We'll prepare a detailed quote with pricing and availability</li>
              <li>• You'll receive the quote via email within 24 hours</li>
              <li>• Our team will follow up to discuss any questions</li>
            </ul>
          </div>
        </div>
```

```
      </main>
    </div>
  )
 }

 return (
  <div className="min-h-screen bg-background">
    <Navigation centered />

    <main className="container mx-auto py-8 px-4">
      {/* Header */}
      <div className="mb-8 text-center">
        <h1 className="text-4xl font-bold text-balance mb-4">Request a Quote</h1>
        <p className="text-lg text-muted-foreground max-w-2xl mx-auto">
          Get personalized pricing for your technology needs. Our team will provide
you with a detailed quote tailored
          to your requirements.
        </p>
      </div>

      {/* Services Offered */}
      <div className="mb-12">
        <Card>
          <CardContent className="pt-6">
            <h3 className="text-xl font-semibold mb-6 text-center">Services
Offered</h3>
            <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-3">
              <div className="text-center p-4">
                <div className="w-12 h-12 bg-emerald-100 rounded-full flex items-
center justify-center mx-auto mb-3">
                  <Calculator className="w-6 h-6 text-emerald-600" />
                </div>
                <h4 className="font-medium mb-2">Custom Solutions</h4>
                <p className="text-sm text-muted-foreground">Tailored technology
solutions for your business needs</p>
              </div>
              <div className="text-center p-4">
                <div className="w-12 h-12 bg-blue-100 rounded-full flex items-center
justify-center mx-auto mb-3">
                  <Clock className="w-6 h-6 text-blue-600" />
                </div>
                <h4 className="font-medium mb-2">Installation Services</h4>
                <p className="text-sm text-muted-foreground">Professional setup and
configuration of your equipment</p>
```

```
            </div>
            <div className="text-center p-4">
              <div className="w-12 h-12 bg-purple-100 rounded-full flex items-center
justify-center mx-auto mb-3">
                <Users className="w-6 h-6 text-purple-600" />
              </div>
              <h4 className="font-medium mb-2">Technical Support</h4>
              <p className="text-sm text-muted-foreground">Ongoing maintenance
and troubleshooting assistance</p>
            </div>
          </div>
        </CardContent>
      </Card>
    </div>

    {/* Quote Form */}
    <div className="max-w-4xl mx-auto">
      <QuoteForm onSubmit={handleQuoteSubmit} />
    </div>

    {/* Quotation Dialog */}
    <QuotationDialog
      open={showQuotationDialog}
      onOpenChange={setShowQuotationDialog}
      quote={pendingQuote}
      onConfirm={handleConfirmQuote}
      onCancel={handleCancelQuote}
      viewOnly={false}
    />
  </main>
 </div>
 )
}
```

App\quotes

```
"use client"

import { useAuth } from "@/lib/auth"
import { useRouter } from "next/navigation"
import { useEffect, useState } from "react"
import { Navigation } from "@/components/navigation"
import { Button } from "@/components/ui/button"
import { Input } from "@/components/ui/input"
import { Badge } from "@/components/ui/badge"
```

```
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card"
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from
"@/components/ui/table"
import { Search, Eye, CheckCircle, XCircle, Clock, FileText, Plus } from "lucide-
react"
import type { Quote } from "@/lib/quotes"
import Link from "next/link"

export default function AdminQuotesPage() {
  const { user, loading } = useAuth()
  const router = useRouter()
  const [searchQuery, setSearchQuery] = useState("")
  const [quotes, setQuotes] = useState<Quote[]>([])
  const [quotesLoading, setQuotesLoading] = useState(true)

  useEffect(() => {
    if (!loading && (!user || user.role !== "admin")) {
      router.push("/login")
    }
  }, [user, loading, router])

  useEffect(() => {
    if (user && user.role === "admin") {
      fetchQuotes()
    }
  }, [user])

  const fetchQuotes = async () => {
    try {
      const response = await fetch("/api/quotes")
      if (response.ok) {
        const data = await response.json()
        setQuotes(data.quotes)
      }
    } catch (error) {
      console.error("Error fetching quotes:", error)
    } finally {
      setQuotesLoading(false)
    }
  }

  const filteredQuotes = quotes.filter(
    (quote) =>
      quote.id.toLowerCase().includes(searchQuery.toLowerCase()) ||
```

```
    quote.customerName.toLowerCase().includes(searchQuery.toLowerCase()) ||
    quote.customerEmail.toLowerCase().includes(searchQuery.toLowerCase()) ||
    quote.company?.toLowerCase().includes(searchQuery.toLowerCase()),
)

const getStatusIcon = (status: Quote["status"]) => {
  switch (status) {
    case "pending":
      return <Clock className="h-4 w-4" />
    case "approved":
      return <CheckCircle className="h-4 w-4" />
    case "rejected":
      return <XCircle className="h-4 w-4" />
    case "expired":
      return <FileText className="h-4 w-4" />
  }
}

const getStatusVariant = (status: Quote["status"]) => {
  switch (status) {
    case "pending":
      return "secondary"
    case "approved":
      return "default"
    case "rejected":
      return "destructive"
    case "expired":
      return "outline"
    default:
      return "secondary"
  }
}

const updateQuoteStatus = async (quoteId: string, status: Quote["status"]) => {
  try {
    const response = await fetch(`/api/quotes/${quoteId}`, {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ status }),
    })

    if (response.ok) {
      await fetchQuotes() // Refresh the list
    }
```

```
    } catch (error) {
      console.error("Error updating quote status:", error)
    }
  }

  if (loading || quotesLoading) {
    return (
      <div className="min-h-screen flex items-center justify-center">
        <div className="animate-spin rounded-full h-32 w-32 border-b-2 border-
emerald-600"></div>
      </div>
    )
  }

  if (!user || user.role !== "admin") {
    return null
  }

  return (
    <div className="min-h-screen bg-background">
      <Navigation centered />

      <main className="container mx-auto py-8 px-4">
        <div className="mb-8">
          <h1 className="text-3xl font-bold text-balance">Quote Management</h1>
          <p className="text-muted-foreground mt-2">Manage customer quote
requests</p>
          <Badge variant="default" className="mt-2">
            Administrator
          </Badge>
        </div>

        {/* Stats */}
        <div className="grid gap-4 md:grid-cols-4 mb-8">
          <Card>
            <CardHeader className="flex flex-row items-center justify-between space-y-
0 pb-2">
              <CardTitle className="text-sm font-medium">Total Quotes</CardTitle>
              <FileText className="h-4 w-4 text-muted-foreground" />
            </CardHeader>
            <CardContent>
              <div className="text-2xl font-bold">{quotes.length}</div>
            </CardContent>
          </Card>
```

```
        <Card>
          <CardHeader className="flex flex-row items-center justify-between space-y-
0 pb-2">
            <CardTitle className="text-sm font-medium">Pending</CardTitle>
            <Clock className="h-4 w-4 text-amber-600" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold text-amber-600">
              {quotes.filter((q) => q.status === "pending").length}
            </div>
          </CardContent>
        </Card>

        <Card>
          <CardHeader className="flex flex-row items-center justify-between space-y-
0 pb-2">
            <CardTitle className="text-sm font-medium">Approved</CardTitle>
            <CheckCircle className="h-4 w-4 text-emerald-600" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold text-emerald-600">
              {quotes.filter((q) => q.status === "approved").length}
            </div>
          </CardContent>
        </Card>

        <Card>
          <CardHeader className="flex flex-row items-center justify-between space-y-
0 pb-2">
            <CardTitle className="text-sm font-medium">Total Value</CardTitle>
            <FileText className="h-4 w-4 text-muted-foreground" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold">
              ₱{quotes.reduce((sum, q) => sum + q.totalAmount, 0).toLocaleString('en-
PH', { minimumFractionDigits: 2, maximumFractionDigits: 2 })}
            </div>
          </CardContent>
        </Card>
      </div>

      {/* Search */}
      <div className="mb-6">
```

```
        <div className="relative max-w-md">
          <Search className="absolute left-3 top-1/2 transform -translate-y-1/2 text-
muted-foreground h-4 w-4" />
          <Input
            placeholder="Search quotes..."
            value={searchQuery}
            onChange={(e) => setSearchQuery(e.target.value)}
            className="pl-10"
          />
        </div>
      </div>

      {/* Quotes Table */}
      <Card>
        <CardHeader>
          <CardTitle>All Quote Requests</CardTitle>
        </CardHeader>
        <CardContent>
          {filteredQuotes.length === 0 ? (
            <div className="text-center py-12">
              <FileText className="h-12 w-12 text-muted-foreground mx-auto mb-4" />
              <h3 className="text-lg font-semibold mb-2">No quotes found</h3>
              <p className="text-muted-foreground">
                {quotes.length === 0
                  ? "No quote requests have been submitted yet."
                  : "No quotes match your search criteria."}
              </p>
            </div>
          ) : (
            <Table>
              <TableHeader>
                <TableRow>
                  <TableHead>Quote ID</TableHead>
                  <TableHead>Customer</TableHead>
                  <TableHead>Company</TableHead>
                  <TableHead>Items</TableHead>
                  <TableHead>Total Amount</TableHead>
                  <TableHead>Status</TableHead>
                  <TableHead>Created</TableHead>
                  <TableHead>Actions</TableHead>
                </TableRow>
              </TableHeader>
              <TableBody>
                {filteredQuotes.map((quote) => (
```

```
              <TableRow key={quote.id}>
                <TableCell className="font-medium">#{quote.id.slice(0,
8)}</TableCell>
                <TableCell>
                  <div>
                    <p className="font-medium">{quote.customerName}</p>
                    <p className="text-sm text-muted-
foreground">{quote.customerEmail}</p>
                  </div>
                </TableCell>
                <TableCell>{quote.company || "-"}</TableCell>
                <TableCell>{quote.items.length} items</TableCell>
                <TableCell className="font-
medium">₱{quote.totalAmount.toLocaleString('en-PH', { minimumFractionDigits: 2,
maximumFractionDigits: 2 })}</TableCell>
                <TableCell>
                  <Badge variant={getStatusVariant(quote.status)}
className="capitalize">
                    {getStatusIcon(quote.status)}
                    <span className="ml-1">{quote.status}</span>
                  </Badge>
                </TableCell>
                <TableCell>{new
Date(quote.createdAt).toLocaleDateString()}</TableCell>
                <TableCell>
                  <div className="flex items-center space-x-2">
                    <Button size="sm" variant="outline">
                      <Eye className="h-4 w-4 mr-1" />
                      View
                    </Button>
                    {quote.status === "pending" && (
                      <>
                        <Button
                          size="sm"
                          variant="outline"
                          onClick={() => updateQuoteStatus(quote.id, "approved")}
                          className="text-emerald-600 hover:text-emerald-700"
                        >
                          <CheckCircle className="h-4 w-4 mr-1" />
                          Approve
                        </Button>
                        <Button
                          size="sm"
                          variant="outline"
```

```
                    onClick={() => updateQuoteStatus(quote.id, "rejected")}
                    className="text-red-600 hover:text-red-700"
                   >
                    <XCircle className="h-4 w-4 mr-1" />
                    Reject
                   </Button>
                  </>
                 )}
                </div>
               </TableCell>
              </TableRow>
             ))}
            </TableBody>
           </Table>
          )}
         </CardContent>
        </Card>
       </main>
      </div>
  )
}

export default function Loading() {
  return null
}
```

App\register

```
import { RegisterForm } from "@/components/auth/register-form"
import Link from "next/link"

export default function RegisterPage() {
  return (
    <div className="min-h-screen bg-gradient-to-br from-emerald-50 to-slate-100
flex items-center justify-center p-4">
     <div className="w-full max-w-md space-y-6">
      <div className="text-center">
       <Link href="/" className="inline-block">
        <h1 className="text-3xl font-bold text-emerald-600">Unicom</h1>
        <p className="text-sm text-muted-foreground">Technologies</p>
       </Link>
      </div>

      <RegisterForm />
```

```
      <p className="text-center text-sm text-muted-foreground">
        Already have an account?{" "}
        <Link href="/login" className="text-emerald-600 hover:underline font-
medium">
          Sign in
        </Link>
      </p>
    </div>
  </div>
 )
}
```

App\services

```
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
"@/components/ui/card"
import { Button } from "@/components/ui/button"
import { Wrench, Monitor, Wifi, HardDrive, Shield, Headphones } from "lucide-
react"
import { VisitorHeader } from "@/components/visitor-header"

export default function ServicesPage() {
 const services = [
   {
    icon: <Monitor className="h-8 w-8" />,
    title: "Computer Repair & Maintenance",
    description: "Professional diagnosis and repair of desktops, laptops, and
workstations",
    features: ["Hardware diagnostics", "Software troubleshooting", "Performance
optimization", "Virus removal"],
   },
   {
    icon: <Wifi className="h-8 w-8" />,
    title: "Network Setup & Support",
    description: "Complete networking solutions for homes and small businesses",
    features: ["Router configuration", "WiFi optimization", "Network security",
"Cable management"],
   },
   {
    icon: <HardDrive className="h-8 w-8" />,
    title: "Data Recovery & Backup",
    description: "Secure data recovery and backup solutions to protect your valuable
information",
    features: ["Hard drive recovery", "Cloud backup setup", "Data migration",
"Storage solutions"],
```

```
    },
    {
      icon: <Shield className="h-8 w-8" />,
      title: "Cybersecurity Solutions",
      description: "Comprehensive security services to protect your digital assets",
      features: ["Antivirus installation", "Firewall configuration", "Security audits",
"Employee training"],
    },
    {
      icon: <Wrench className="h-8 w-8" />,
      title: "Custom PC Building",
      description: "Tailored computer builds for gaming, work, and specialized
applications",
      features: ["Performance consulting", "Component selection", "Assembly &
testing", "Warranty support"],
    },
    {
      icon: <Headphones className="h-8 w-8" />,
      title: "Technical Support",
      description: "Ongoing technical support and consultation for all your IT needs",
      features: ["Remote assistance", "On-site support", "Training sessions",
"Maintenance plans"],
    },
  ]

  return (
    <div className="min-h-screen bg-background">
      <VisitorHeader />
      <main className="container mx-auto px-4 py-8">
      <div className="text-center mb-12">
        <h1 className="text-4xl font-bold mb-4 text-balance">Our Services</h1>
        <p className="text-xl text-muted-foreground max-w-2xl mx-auto text-pretty">
          Comprehensive IT solutions and technical support services for individuals and
small businesses
        </p>
      </div>

      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6 mb-12">
        {services.map((service, index) => (
          <Card key={index} className="h-full">
            <CardHeader>
              <div className="text-emerald-600 mb-2">{service.icon}</div>
              <CardTitle className="text-xl">{service.title}</CardTitle>
              <CardDescription>{service.description}</CardDescription>
```

```
            </CardHeader>
            <CardContent>
              <ul className="space-y-2">
                {service.features.map((feature, idx) => (
                  <li key={idx} className="flex items-center text-sm">
                    <div className="w-1.5 h-1.5 bg-emerald-600 rounded-full mr-2" />
                    {feature}
                  </li>
                ))}
              </ul>
            </CardContent>
          </Card>
        ))}
      </div>

    </main>
  </div>
 )
}
```

App\support

```
"use client"

import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
"@/components/ui/card"
import { Button } from "@/components/ui/button"
import { Input } from "@/components/ui/input"
import { Textarea } from "@/components/ui/textarea"
import { Label } from "@/components/ui/label"
import { Phone, Mail, MapPin, Clock, MessageCircle, FileText, Ticket } from
"lucide-react"
import { VisitorHeader } from "@/components/visitor-header"
import { Navigation } from "@/components/navigation"
import { useAuth } from "@/lib/auth"
import Link from "next/link"

export default function SupportPage() {
 const { user } = useAuth()

 return (
  <div className="min-h-screen bg-background">
    {user ? <Navigation centered /> : <VisitorHeader />}
    <main className="container mx-auto px-4 py-8">
    <div className="text-center mb-12">
      <h1 className="text-4xl font-bold mb-4 text-balance">Technical Support</h1>
```

```
      <p className="text-xl text-muted-foreground max-w-2xl mx-auto text-pretty">
        Get help when you need it. Our expert support team is here to assist you with
all your technical needs.
      </p>
    </div>

    <div className="grid grid-cols-1 lg:grid-cols-2 gap-8 mb-12">
      {/* Contact Information */}
      <div className="space-y-6">
        <Card>
          <CardHeader>
            <CardTitle className="flex items-center gap-2">
              <Phone className="h-5 w-5 text-emerald-600" />
              Contact Information
            </CardTitle>
          </CardHeader>
          <CardContent className="space-y-4">
            <div className="flex items-center gap-3">
              <Phone className="h-4 w-4 text-muted-foreground" />
              <div>
                <p className="font-medium">Phone Support</p>
                <p className="text-sm text-muted-foreground">(555) 123-TECH</p>
              </div>
            </div>
            <div className="flex items-center gap-3">
              <Mail className="h-4 w-4 text-muted-foreground" />
              <div>
                <p className="font-medium">Email Support</p>
                <p className="text-sm text-muted-
foreground">support@unicomtech.com</p>
              </div>
            </div>
            <div className="flex items-center gap-3">
              <MapPin className="h-4 w-4 text-muted-foreground" />
              <div>
                <p className="font-medium">Visit Our Store</p>
                <p className="text-sm text-muted-foreground">123 Tech Street, Digital
City, DC 12345</p>
              </div>
            </div>
            <div className="flex items-center gap-3">
              <Clock className="h-4 w-4 text-muted-foreground" />
              <div>
                <p className="font-medium">Business Hours</p>
```

```
            <p className="text-sm text-muted-foreground">Mon-Fri: 9AM-6PM,
Sat: 10AM-4PM</p>
          </div>
        </div>
      </CardContent>
    </Card>

    {/* Support Ticket Section for Logged-in Users */}
    {user && (
     <Card>
      <CardHeader>
       <CardTitle className="flex items-center gap-2">
        <Ticket className="h-5 w-5 text-emerald-600" />
        Support Tickets
       </CardTitle>
       <CardDescription>
        Submit and track your support requests
       </CardDescription>
      </CardHeader>
      <CardContent className="space-y-4">
       <p className="text-sm text-muted-foreground">
        Need help? Submit a support ticket and we'll get back to you quickly.
       </p>
       <div className="flex gap-2">
        <Button asChild className="flex-1">
         <Link href="/support-ticket">
          <Ticket className="h-4 w-4 mr-2" />
          Submit Ticket
         </Link>
        </Button>
        <Button variant="outline" asChild className="flex-1">
         <Link href="/customer/tickets">
          <FileText className="h-4 w-4 mr-2" />
          My Tickets
         </Link>
        </Button>
       </div>
      </CardContent>
     </Card>
    )}
   </div>

   {/* Support Form */}
   <Card>
```

```
        <CardHeader>
          <CardTitle>Submit Support Request</CardTitle>
          <CardDescription>
            {user
              ? "Describe your issue and we'll get back to you as soon as possible"
              : "Please log in to submit a support ticket, or use the contact information
above"
            }
          </CardDescription>
        </CardHeader>
        <CardContent className="space-y-4">
          {user ? (
            <div className="text-center py-8">
              <p className="text-muted-foreground mb-4">
                Use our ticket system for faster support
              </p>
              <Button asChild className="w-full">
                <Link href="/support-ticket">
                  <Ticket className="h-4 w-4 mr-2" />
                  Submit Support Ticket
                </Link>
              </Button>
            </div>
          ) : (
            <>
              <div className="grid grid-cols-2 gap-4">
                <div className="space-y-2">
                  <Label htmlFor="firstName">First Name</Label>
                  <Input id="firstName" placeholder="John" />
                </div>
                <div className="space-y-2">
                  <Label htmlFor="lastName">Last Name</Label>
                  <Input id="lastName" placeholder="Doe" />
                </div>
              </div>
              <div className="space-y-2">
                <Label htmlFor="email">Email</Label>
                <Input id="email" type="email" placeholder="john@example.com" />
              </div>
              <div className="space-y-2">
                <Label htmlFor="phone">Phone (Optional)</Label>
                <Input id="phone" type="tel" placeholder="(555) 123-4567" />
              </div>
              <div className="space-y-2">
```

```
              <Label htmlFor="subject">Subject</Label>
              <Input id="subject" placeholder="Brief description of your issue" />
             </div>
             <div className="space-y-2">
              <Label htmlFor="message">Message</Label>
              <Textarea id="message" placeholder="Please describe your issue in
detail..." className="min-h-[120px]" />
             </div>
             <Button className="w-full">Submit Support Request</Button>
            </>
           )}
         </CardContent>
        </Card>
       </div>

       {/* FAQ Section */}
       <Card>
        <CardHeader>
         <CardTitle>Frequently Asked Questions</CardTitle>
         <CardDescription>Quick answers to common questions</CardDescription>
        </CardHeader>
        <CardContent className="space-y-4">
         <div className="space-y-2">
          <h4 className="font-medium">What are your support hours?</h4>
          <p className="text-sm text-muted-foreground">
           Our support team is available Monday-Friday 9AM-6PM and Saturday
10AM-4PM. Emergency support is available
           24/7 for critical issues.
          </p>
         </div>
         <div className="space-y-2">
          <h4 className="font-medium">Do you offer remote support?</h4>
          <p className="text-sm text-muted-foreground">
           Yes, we provide secure remote support for software issues, troubleshooting,
and system maintenance when
           possible.
          </p>
         </div>
         <div className="space-y-2">
          <h4 className="font-medium">What's included in your warranty?</h4>
          <p className="text-sm text-muted-foreground">
           All products come with manufacturer warranty plus our 30-day satisfaction
guarantee. Labor warranties vary
           by service type.
```

```
          </p>
         </div>
        </CardContent>
      </Card>
      </main>
    </div>
  )
}
```

App\support-ticket

```
"use client"

import { useState } from "react"
import { useAuth } from "@/lib/auth"
import { Button } from "@/components/ui/button"
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card"
import { Input } from "@/components/ui/input"
import { Label } from "@/components/ui/label"
import { Textarea } from "@/components/ui/textarea"
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from
"@/components/ui/select"
import { Navigation } from "@/components/navigation"
import { useRouter } from "next/navigation"
import { toast } from "sonner"

export default function SupportTicketPage() {
  const { user } = useAuth()
  const router = useRouter()
  const [isSubmitting, setIsSubmitting] = useState(false)
  const [formData, setFormData] = useState({
    subject: "",
    issueType: "",
    description: "",
    priority: "medium",
  })

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault()
    if (!user) {
      toast.error("Please log in to submit a ticket")
      return
    }

    // Validate required fields
```

```
    if (!formData.subject || !formData.issueType || !formData.description) {
      toast.error("Please fill in all required fields")
      return
    }

    if (formData.description.length < 5) {
      toast.error("Description must be at least 5 characters long")
      return
    }

    setIsSubmitting(true)
    try {
      console.log("Submitting ticket:", { ...formData, customerName: user.name,
customerEmail: user.email })

      const response = await fetch("/api/tickets", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          ...formData,
          customerName: user.name,
          customerEmail: user.email,
        }),
      })

      const data = await response.json()
      console.log("API Response:", data)

      if (response.ok) {
        toast.success("Support ticket submitted successfully!")
        router.push("/customer/tickets")
      } else {
        console.error("API Error:", data)
        toast.error(`Failed to submit ticket: ${data.error || 'Unknown error'}`)
      }
    } catch (error) {
      console.error("Submission error:", error)
      toast.error("Failed to submit ticket. Please try again.")
    } finally {
      setIsSubmitting(false)
    }
  }

  if (!user) {
```

```
  return (
    <div className="min-h-screen bg-gray-50">
      <Navigation centered />
      <div className="container mx-auto py-8">
        <Card className="max-w-2xl mx-auto">
          <CardContent className="text-center py-8">
            <p className="text-gray-500">Please log in to submit a support ticket.</p>
          </CardContent>
        </Card>
      </div>
    </div>
  )
}

 return (
   <div className="min-h-screen bg-gray-50">
     <Navigation centered />

     <div className="container mx-auto py-8">
       <div className="mb-6">
         <h1 className="text-3xl font-bold text-gray-900">Submit Support
Ticket</h1>
         <p className="text-gray-600 mt-2">We're here to help! Please provide details
about your issue.</p>
       </div>

       <Card className="max-w-2xl mx-auto">
         <CardHeader>
           <CardTitle>Support Ticket Details</CardTitle>
         </CardHeader>
         <CardContent>
           <form onSubmit={handleSubmit} className="space-y-4">
             <div>
               <Label htmlFor="subject">Subject *</Label>
               <Input
                 id="subject"
                 value={formData.subject}
                 onChange={(e) => setFormData({ ...formData, subject: e.target.value })}
                 placeholder="Brief description of your issue"
                 required
               />
             </div>

             <div>
```

```
            <Label htmlFor="issueType">Issue Type *</Label>
            <Select
              value={formData.issueType}
              onValueChange={(value) => setFormData({ ...formData, issueType:
value })}
            >
              <SelectTrigger>
                <SelectValue placeholder="Select issue type" />
              </SelectTrigger>
              <SelectContent>
                <SelectItem value="technical">Technical Issue</SelectItem>
                <SelectItem value="billing">Billing Question</SelectItem>
                <SelectItem value="general">General Inquiry</SelectItem>
                <SelectItem value="product">Product Question</SelectItem>
                <SelectItem value="other">Other</SelectItem>
              </SelectContent>
            </Select>
          </div>

          <div>
            <Label htmlFor="priority">Priority</Label>
            <Select
              value={formData.priority}
              onValueChange={(value) => setFormData({ ...formData, priority:
value })}
            >
              <SelectTrigger>
                <SelectValue />
              </SelectTrigger>
              <SelectContent>
                <SelectItem value="low">Low</SelectItem>
                <SelectItem value="medium">Medium</SelectItem>
                <SelectItem value="high">High</SelectItem>
                <SelectItem value="urgent">Urgent</SelectItem>
              </SelectContent>
            </Select>
          </div>

          <div>
            <Label htmlFor="description">Description *</Label>
            <Textarea
              id="description"
              value={formData.description}
```

```
                onChange={(e) => setFormData({ ...formData, description:
e.target.value })}
                rows={6}
                required
                placeholder="Please provide detailed information about your issue..."
              />
            </div>

            <div className="flex gap-4">
              <Button type="submit" disabled={isSubmitting} className="flex-1">
                {isSubmitting ? "Submitting..." : "Submit Ticket"}
              </Button>
              <Button
                type="button"
                variant="outline"
                onClick={() => router.back()}
                className="flex-1"
              >
                Cancel
              </Button>
            </div>
          </form>
        </CardContent>
      </Card>
    </div>
  </div>
 )
}
```

App/unauthorized

```
import { Button } from "@/components/ui/button"
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
"@/components/ui/card"
import { Badge } from "@/components/ui/badge"
import { ShoppingCart, Users, Wrench, ArrowRight, Cpu, HardDrive, Monitor,
LogIn } from "lucide-react"
import Link from "next/link"

export default function HomePage() {
 return (
  <div className="min-h-screen bg-background">
   {/* Header */}
   <header className="border-b bg-card/50 backdrop-blur-sm sticky top-0 z-50">
    <div className="container mx-auto px-4 py-4">
```

```
        <div className="flex items-center justify-between">
          <div className="flex items-center gap-2">
            <div className="w-8 h-8 bg-primary rounded-lg flex items-center justify-
center">
              <Cpu className="w-5 h-5 text-primary-foreground" />
            </div>
            <h1 className="text-xl font-bold text-foreground">Unicom
Technologies</h1>
          </div>
          <nav className="hidden md:flex items-center gap-6">
            <Link href="/" className="text-sm text-emerald-600 transition-colors">
              Home
            </Link>
            <Link href="/catalog" className="text-sm text-muted-foreground
hover:text-foreground transition-colors">
              Catalog
            </Link>
            <Link href="/services" className="text-sm text-muted-foreground
hover:text-foreground transition-colors">
              Services
            </Link>
            <Link href="/support" className="text-sm text-muted-foreground
hover:text-foreground transition-colors">
              Support
            </Link>
            <Link href="/about" className="text-sm text-muted-foreground hover:text-
foreground transition-colors">
              About
            </Link>
          </nav>
          <div className="flex items-center gap-3">
            <Link href="/login">
              <Button variant="outline" size="sm">
                Customer Sign In
              </Button>
            </Link>
            <Link href="/register">
              <Button size="sm">
                <LogIn className="w-4 h-4 mr-2" />
                Register
              </Button>
            </Link>
          </div>
        </div>
```

```
      </div>
    </header>

    {/* Hero Section */}
    <section className="py-20 px-4">
      <div className="container mx-auto text-center">
        <Badge variant="secondary" className="mb-4">
          Trusted Local IT Solutions Provider
        </Badge>
        <h2 className="text-4xl md:text-6xl font-bold text-balance mb-6">
          Your One-Stop Shop for <span className="text-primary">Electronics & IT
Solutions</span>
        </h2>
        <p className="text-xl text-muted-foreground text-balance mb-8 max-w-2xl
mx-auto">
          Quality computer parts, accessories, and expert technical support for
individuals and businesses. Reliable
          products, personalized service, cost-efficient solutions.
        </p>
        <div className="flex flex-col sm:flex-row gap-4 justify-center">
          <Button size="lg" className="text-lg px-8" asChild>
            <Link href="/catalog">
              Browse Catalog
              <ArrowRight className="w-5 h-5 ml-2" />
            </Link>
          </Button>
          <Button variant="outline" size="lg" className="text-lg px-8 bg-transparent"
asChild>
            <Link href="/login">
              Get Quote
            </Link>
          </Button>
        </div>
      </div>
    </section>

    {/* Features Grid */}
    <section className="py-16 px-4 bg-muted/30">
      <div className="container mx-auto">
        <div className="text-center mb-12">
          <h3 className="text-3xl font-bold mb-4">Why Choose Unicom
Technologies?</h3>
          <p className="text-muted-foreground text-lg max-w-2xl mx-auto">
```

```
          We combine quality products with exceptional service to meet all your
technology needs
          </p>
        </div>

        <div className="grid md:grid-cols-3 gap-8">
          <Card className="border-0 shadow-sm hover:shadow-md transition-
shadow">
            <CardHeader className="text-center">
              <div className="w-12 h-12 bg-primary/10 rounded-lg flex items-center
justify-center mx-auto mb-4">
                <ShoppingCart className="w-6 h-6 text-primary" />
              </div>
              <CardTitle>Wide Product Range</CardTitle>
            </CardHeader>
            <CardContent>
              <CardDescription className="text-center">
                Comprehensive selection of computer parts, accessories, and IT equipment
from trusted brands
              </CardDescription>
            </CardContent>
          </Card>

          <Card className="border-0 shadow-sm hover:shadow-md transition-
shadow">
            <CardHeader className="text-center">
              <div className="w-12 h-12 bg-primary/10 rounded-lg flex items-center
justify-center mx-auto mb-4">
                <Wrench className="w-6 h-6 text-primary" />
              </div>
              <CardTitle>Expert Support</CardTitle>
            </CardHeader>
            <CardContent>
              <CardDescription className="text-center">
                Professional technical support and personalized service for all your IT
needs
              </CardDescription>
            </CardContent>
          </Card>

          <Card className="border-0 shadow-sm hover:shadow-md transition-
shadow">
            <CardHeader className="text-center">
```

```
            <div className="w-12 h-12 bg-primary/10 rounded-lg flex items-center
justify-center mx-auto mb-4">
              <Users className="w-6 h-6 text-primary" />
            </div>
            <CardTitle>Local Community</CardTitle>
          </CardHeader>
          <CardContent>
            <CardDescription className="text-center">
              Serving individual consumers and small businesses with cost-efficient,
reliable solutions
            </CardDescription>
          </CardContent>
        </Card>
      </div>
    </div>
  </section>

  {/* Product Categories */}
  <section className="py-16 px-4">
    <div className="container mx-auto">
      <div className="text-center mb-12">
        <h3 className="text-3xl font-bold mb-4">Popular Categories</h3>
        <p className="text-muted-foreground text-lg">Explore our most requested
product categories</p>
      </div>

      <div className="grid md:grid-cols-3 gap-6">
        <Card className="group cursor-pointer hover:shadow-lg transition-all
duration-300">
          <CardHeader>
            <div className="w-16 h-16 bg-gradient-to-br from-primary/20 to-
primary/10 rounded-xl flex items-center justify-center mb-4 group-hover:scale-110
transition-transform">
              <Cpu className="w-8 h-8 text-primary" />
            </div>
            <CardTitle>Computer Components</CardTitle>
            <CardDescription>CPUs, GPUs, motherboards, RAM, and
more</CardDescription>
          </CardHeader>
        </Card>

        <Card className="group cursor-pointer hover:shadow-lg transition-all
duration-300">
          <CardHeader>
```

```
            <div className="w-16 h-16 bg-gradient-to-br from-primary/20 to-primary/10 rounded-xl flex items-center justify-center mb-4 group-hover:scale-110 transition-transform">
              <HardDrive className="w-8 h-8 text-primary" />
            </div>
            <CardTitle>Storage Solutions</CardTitle>
            <CardDescription>SSDs, HDDs, external drives, and backup solutions</CardDescription>
          </CardHeader>
        </Card>

        <Card className="group cursor-pointer hover:shadow-lg transition-all duration-300">
          <CardHeader>
            <div className="w-16 h-16 bg-gradient-to-br from-primary/20 to-primary/10 rounded-xl flex items-center justify-center mb-4 group-hover:scale-110 transition-transform">
              <Monitor className="w-8 h-8 text-primary" />
            </div>
            <CardTitle>Peripherals</CardTitle>
            <CardDescription>Monitors, keyboards, mice, and accessories</CardDescription>
          </CardHeader>
        </Card>
      </div>
    </div>
  </section>

  {/* CTA Section */}
  <section className="py-20 px-4 bg-primary text-primary-foreground">
    <div className="container mx-auto text-center">
      <h3 className="text-3xl font-bold mb-4">Ready to Get Started?</h3>
      <p className="text-xl opacity-90 mb-8 max-w-2xl mx-auto">
        Browse our catalog or contact us for personalized recommendations and quotes
      </p>
      <div className="flex justify-center">
        <Button
          size="lg"
          variant="outline"
          className="text-lg px-8 border-primary-foreground text-primary-foreground hover:bg-primary-foreground hover:text-primary bg-transparent"
          asChild
        >
```

```
            <Link href="/support">
              Contact Us
            </Link>
          </Button>
        </div>
      </div>
    </section>

    {/* Footer */}
    <footer className="bg-card border-t py-12 px-4">
      <div className="container mx-auto">
        <div className="grid md:grid-cols-4 gap-8">
          <div>
            <div className="flex items-center gap-2 mb-4">
              <div className="w-6 h-6 bg-primary rounded flex items-center justify-
center">
                <Cpu className="w-4 h-4 text-primary-foreground" />
              </div>
              <span className="font-bold">Unicom Technologies</span>
            </div>
            <p className="text-muted-foreground text-sm">
              Your trusted local provider for electronics and IT solutions.
            </p>
          </div>

          <div>
            <h4 className="font-semibold mb-3">Products</h4>
            <ul className="space-y-2 text-sm text-muted-foreground">
              <li>
                <Link href="#" className="hover:text-foreground transition-colors">
                  Computer Parts
                </Link>
              </li>
              <li>
                <Link href="#" className="hover:text-foreground transition-colors">
                  Accessories
                </Link>
              </li>
              <li>
                <Link href="#" className="hover:text-foreground transition-colors">
                  Storage
                </Link>
              </li>
              <li>
```

```
      <Link href="#" className="hover:text-foreground transition-colors">
        Peripherals
      </Link>
    </li>
  </ul>
</div>

<div>
  <h4 className="font-semibold mb-3">Services</h4>
  <ul className="space-y-2 text-sm text-muted-foreground">
    <li>
      <Link href="#" className="hover:text-foreground transition-colors">
        Technical Support
      </Link>
    </li>
    <li>
      <Link href="#" className="hover:text-foreground transition-colors">
        Installation
      </Link>
    </li>
    <li>
      <Link href="#" className="hover:text-foreground transition-colors">
        Consultation
      </Link>
    </li>
    <li>
      <Link href="#" className="hover:text-foreground transition-colors">
        Maintenance
      </Link>
    </li>
  </ul>
</div>

<div>
  <h4 className="font-semibold mb-3">Company</h4>
  <ul className="space-y-2 text-sm text-muted-foreground">
    <li>
      <Link href="#" className="hover:text-foreground transition-colors">
        About Us
      </Link>
    </li>
    <li>
      <Link href="#" className="hover:text-foreground transition-colors">
        Contact
```

```
              </Link>
             </li>
             <li>
              <Link href="#" className="hover:text-foreground transition-colors">
               Support
              </Link>
             </li>
             <li>
              <Link href="#" className="hover:text-foreground transition-colors">
               Privacy
              </Link>
             </li>
            </ul>
           </div>
          </div>

          <div className="border-t mt-8 pt-8 text-center text-sm text-muted-
foreground">
           <p>&copy; 2025 Unicom Technologies. All rights reserved.</p>
          </div>
         </div>
        </footer>
       </div>
      )
}


import type React from "react"
import type { Metadata } from "next"
import { Space_Grotesk } from "next/font/google"
import { DM_Sans } from "next/font/google"
import { Analytics } from "@vercel/analytics/next"
import { Suspense } from "react"
import { AuthProvider } from "@/lib/auth"
import { Chatbot } from "@/components/chatbot"
import { Toaster } from "sonner"
import "./globals.css"

const spaceGrotesk = Space_Grotesk({
 subsets: ["latin"],
 variable: "--font-space-grotesk",
 display: "swap",
})
```

```
const dmSans = DM_Sans({
  subsets: ["latin"],
  variable: "--font-dm-sans",
  display: "swap",
})

export const metadata: Metadata = {
  title: "Unicom Technologies - Electronics & IT Solutions",
  description:
    "Your trusted local provider for computer parts, accessories, and technical support
services. Quality products and personalized service for consumers and businesses.",
  generator: "v0.app",
  icons: {
    icon: "data:image/svg+xml,<svg xmlns='http://www.w3.org/2000/svg' viewBox='0
0 24 24' fill='%23059669'><path d='M12 2L2 7l10 5 10-5-10-5zM2 17l10 5 10-5M2
12l10 5 10-5'/><circle cx='12' cy='7' r='1.5'/><circle cx='12' cy='12' r='1.5'/><circle
cx='12' cy='17' r='1.5'/></svg>",
  },
}

export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode
}>) {
  return (
    <html lang="en">
      <body className={`font-sans ${spaceGrotesk.variable} ${dmSans.variable}
antialiased`}>
        <AuthProvider>
          <Suspense fallback={<div>Loading...</div>}>
            {children}
            <Analytics />
            <Chatbot />
            <Toaster />
          </Suspense>
        </AuthProvider>
      </body>
    </html>
  )
}
```

Bibliography

Highsmith, J. (2009). *Agile project management: Creating innovative products*. Addison-
Wesley.
Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.
Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach*
(9th ed.). McGraw-Hill.

MongoDB. (2025). *MongoDB Atlas documentation*.
https://www.mongodb.com/docs/atlas/
Next.js. (2025). *Next.js 14 documentation*.
https://nextjs.org/docs
Node.js Foundation. (2025). *Node.js documentation*.
https://nodejs.org/en/docs/
Express.js. (2025). *Express documentation*.
https://expressjs.com/
Tailwind Labs. (2025). *Tailwind CSS documentation*.
https://tailwindcss.com/docs
Vercel. (2025). *Vercel documentation*.
https://vercel.com/docs
Gemini. (2025). *API reference documentation*.
https://ai.google.dev/gemini-api/docs
JWT. (2025). *JSON Web Token specification*.
https://jwt.io/introduction/
bcrypt. (2025). *bcrypt documentation*.
https://www.npmjs.com/package/bcrypt

OMG. (2017). *Unified Modeling Language (UML) specification, version 2.5.1*. Object
Management Group.
Yourdon, E., & Constantine, L. (1979). *Structured design: Fundamentals of a discipline of*
computer program and systems design. Prentice-Hall.

Gane, C., & Sarson, T. (1979). *Structured systems analysis: Tools and techniques*. Improved
Systems Technologies.

ISO/IEC. (2011). *ISO/IEC 25010: Systems and software engineering Systems and software*
quality requirements and evaluation (SQuaRE) System and software quality models
International Organization for Standardization

Resource Persons

Resource Person 1

Name: *Dennis Carlos*

Position: CEO / Project Development Manager, Unicom Technologies

Contribution:

- Explained the overall company workflow structure
- Provided approval flow for quotations and costing
- Clarified organizational roles relevant to system development

Resource Person 2

Name: *Nicole Claros-Casimsiman, Zel Penaflor, Rochelle Aquino*

Position: Corporate Account Executive

Contribution:

- Described the step-by-step quotation process used daily
- Identified delays caused by manual sourcing and manual communication
- Validated pain points such as follow-ups, sourcing delays, and data inconsistency

Resource Person 3

Name: *Roselle Claros*

Position: Financial Officer

Contribution:

- Explained payment coordination and approval flow
- Defined requirements for finance-related processes
- Identified data needed for analytics (e.g., monthly revenue, client status)

Resource Person 4

Name: *Dennis V. Carlos - System Analyst (Student Researcher)*

Contribution:

- Identified technical requirements
- Assisted in effort estimation for documentation and testing
- Helped validate feasibility of timelines (Gantt chart duration, sprint timelines)

Resource Person 5

Name: *Christian Ivan A. Los Baños – Front End (Student Researcher)*

Contribution:

- Designed UI wireframes for Login, Dashboard, Quotation Form, Chatbot, Analytics
- Conducted interface usability feedback

Resource Person 6

Name: *Justine M. Lopez – Back End (Student Researcher)*

Contribution:

- Supported system testing (unit testing, API testing)
- Conducted evaluation for ISO/IEC 25010 criteria

Resource Person 7x

Name: *Dennis Carlos*

Role: User Acceptance Tester

Contribution:

- Tested actual quotation creation, chatbot responses, product viewing
- Provided approval feedback for final UAT